

Manual del Programador Competitivo

Antti Laaksonen

Traducción al español

Borrador 19 de agosto de 2026

Índice general

Prefacio	VII
I Técnicas básicas	1
1. Introducción	3
1.1. Lenguajes de programación	3
1.2. Entrada y salida	4
1.3. Trabajar con números	6
1.4. Acortar el código	8
1.5. Matemáticas	10
1.6. Concursos y recursos	15
2. Complejidad temporal	19
2.1. Reglas de cálculo	19
2.2. Clases de complejidad	22
2.3. Estimar la eficiencia	23
2.4. Suma máxima de subarreglo	24
3. Ordenamiento	27
3.1. Teoría del ordenamiento	27
3.2. Ordenamiento en C++	32
3.3. Búsqueda binaria	34
4. Estructuras de datos	39
4.1. Arreglos dinámicos	39
4.2. Estructuras de conjunto	41
4.3. Estructuras de mapa	42
4.4. Iteradores y rangos	43
4.5. Otras estructuras	46
4.6. Comparación con el ordenamiento	50
5. Búsqueda completa	53
5.1. Generar subconjuntos	53
5.2. Generar permutaciones	55
5.3. Backtracking	56
5.4. Podar la búsqueda	58
5.5. Encuentro en el medio	61

6. Algoritmos voraces	63
6.1. Problema de las monedas	63
6.2. Planificación	64
6.3. Tareas y fechas límite	66
6.4. Minimizar sumas	67
6.5. Compresión de datos	68
7. Programación dinámica	71
7.1. Problema de las monedas	71
7.2. Subsecuencia creciente más larga	76
7.3. Caminos en una cuadrícula	77
7.4. Problemas de la mochila	79
7.5. Distancia de edición	80
7.6. Contar teselados	82
8. Análisis amortizado	85
8.1. Método de los dos punteros	85
8.2. Elementos menores más cercanos	87
8.3. Mínimo de ventana deslizante	89
9. Consultas de rango	91
9.1. Consultas en un arreglo estático	92
9.2. Árbol indexado binario	94
9.3. Árbol de segmentos	97
9.4. Técnicas adicionales	101
10. Manipulación de bits	103
10.1. Representación en bits	103
10.2. Operaciones de bits	104
10.3. Representar conjuntos	106
10.4. Optimizaciones con bits	108
10.5. Programación dinámica	110
II Algoritmos de grafos	117
III Temas avanzados	119
Bibliografía	121

Prefacio

El propósito de este libro es ofrecerte una introducción completa a la programación competitiva. Se asume que ya conoces los fundamentos de la programación, pero no se necesita ninguna experiencia previa en programación competitiva.

El libro está especialmente pensado para estudiantes que quieren aprender algoritmos y posiblemente participar en la Olimpiada Internacional de Informática (IOI) o en el Concurso Internacional de Programación Colegial (ICPC). Por supuesto, el libro también es adecuado para cualquier otra persona interesada en la programación competitiva.

Se necesita mucho tiempo para llegar a ser un buen programador competitivo, pero también es una oportunidad para aprender mucho. Puedes estar seguro de que obtendrás una buena comprensión general de los algoritmos si dedicas tiempo a leer el libro, resolver problemas y participar en concursos.

El libro está en desarrollo continuo. Siempre puedes enviar comentarios sobre el libro a `ahslaaks@cs.helsinki.fi`.

Helsinki, agosto de 2019
Antti Laaksonen

Parte I

Técnicas básicas

Capítulo 1

Introducción

La programación competitiva combina dos temas: (1) el diseño de algoritmos y (2) la implementación de algoritmos.

El **diseño de algoritmos** consiste en la resolución de problemas y el pensamiento matemático. Se requieren habilidades para analizar problemas y resolverlos de forma creativa. Un algoritmo para resolver un problema tiene que ser a la vez correcto y eficiente, y el núcleo del problema suele consistir en idear un algoritmo eficiente.

El conocimiento teórico de los algoritmos es importante para los programadores competitivos. Por lo general, la solución de un problema es una combinación de técnicas bien conocidas e ideas nuevas. Las técnicas que aparecen en la programación competitiva también constituyen la base de la investigación científica de los algoritmos.

La **implementación de algoritmos** requiere buenas habilidades de programación. En la programación competitiva, las soluciones se califican probando un algoritmo implementado mediante un conjunto de casos de prueba. Por lo tanto, no basta con que la idea del algoritmo sea correcta, sino que la implementación también tiene que ser correcta.

Un buen estilo de código en los concursos es directo y conciso. Los programas deben escribirse con rapidez, porque no hay mucho tiempo disponible. A diferencia de la ingeniería de software tradicional, los programas son cortos (normalmente a lo sumo unos pocos cientos de líneas de código), y no necesitan mantenerse después del concurso.

1.1. Lenguajes de programación

En la actualidad, los lenguajes de programación más populares usados en los concursos son C++, Python y Java. Por ejemplo, en el Google Code Jam 2017, entre los mejores 3.000 participantes, el 79% usó C++, el 16% usó Python y el 8% usó Java [29]. Algunos participantes también usaron varios lenguajes.

Mucha gente piensa que C++ es la mejor opción para un programador competitivo, y C++ casi siempre está disponible en los sistemas de concursos. Los beneficios de usar C++ son que es un lenguaje muy eficiente y su biblioteca estándar contiene una gran colección de estructuras de datos y algoritmos.

Por otro lado, es bueno dominar varios lenguajes y comprender sus fortalezas. Por ejemplo, si se necesitan enteros muy grandes en el problema, Python puede ser una buena opción, porque contiene operaciones integradas para realizar cálculos con enteros grandes. Aun así, la mayoría de los problemas en los concursos de programación se plantean de modo que usar un lenguaje de programación específico no sea una ventaja injusta.

Todos los programas de ejemplo de este libro están escritos en C++, y a menudo se usan las estructuras de datos y los algoritmos de la biblioteca estándar. Los programas siguen el estándar C++11, que puede usarse en la mayoría de los concursos hoy en día. Si todavía no sabes programar en C++, ahora es un buen momento para empezar a aprender.

Plantilla de código C++

Una plantilla de código C++ típica para programación competitiva tiene este aspecto:

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    // aquí se escribe la solución
}
```

La línea `#include` al principio del código es una instrucción que le indica al compilador `g++` incluir toda la biblioteca estándar. Así, no es necesario incluir por separado bibliotecas como `iostream`, `vector` y `algorithm`, sino que están disponibles automáticamente.

La línea `using` declara que las clases y funciones de la biblioteca estándar pueden usarse directamente en el código. Sin la línea `using` tendríamos que escribir, por ejemplo, `std::cout`, pero ahora basta con escribir `cout`.

El código puede compilarse usando el siguiente comando:

```
g++ -std=c++11 -O2 -Wall test.cpp -o test
```

Este comando produce un archivo binario `test` a partir del código fuente `test.cpp`. El compilador sigue el estándar C++11 (`-std=c++11`), optimiza el código (`-O2`) y muestra advertencias sobre posibles errores (`-Wall`).

1.2. Entrada y salida

En la mayoría de los concursos, se usan flujos estándar para leer la entrada y escribir la salida. En C++, los flujos estándar son `cin` para la entrada y `cout` para la salida. Además, pueden usarse las funciones de C `scanf` y `printf`.

La entrada del programa normalmente consta de números y cadenas separados por espacios y saltos de línea. Pueden leerse del flujo `cin` de la siguiente

manera:

```
int a, b;
string x;
cin >> a >> b >> x;
```

Este tipo de código siempre funciona, suponiendo que haya al menos un espacio o salto de línea entre cada elemento de la entrada. Por ejemplo, el código anterior puede leer ambas de las siguientes entradas:

```
123 456 monkey
```

```
123    456
monkey
```

El flujo cout se usa para la salida de la siguiente manera:

```
int a = 123, b = 456;
string x = "monkey";
cout << a << " " << b << " " << x << "\n";
```

La entrada y la salida a veces son un cuello de botella en el programa. Las siguientes líneas al principio del código hacen que la entrada y la salida sean más eficientes:

```
ios::sync_with_stdio(0);
cin.tie(0);
```

Nota que el salto de línea "\n" funciona más rápido que endl, porque endl siempre provoca una operación de vaciado (flush).

Las funciones de C scanf y printf son una alternativa a los flujos estándar de C++. Suelen ser un poco más rápidas, pero también son más difíciles de usar. El siguiente código lee dos enteros de la entrada:

```
int a, b;
scanf("%d %d", &a, &b);
```

El siguiente código imprime dos enteros:

```
int a = 123, b = 456;
printf("%d %d\n", a, b);
```

A veces el programa debe leer una línea completa de la entrada, que posiblemente contenga espacios. Esto puede lograrse usando la función getline:

```
string s;
getline(cin, s);
```

Si la cantidad de datos es desconocida, el siguiente bucle es útil:

```
while (cin >> x) {  
    // código  
}
```

Este bucle lee elementos de la entrada uno tras otro, hasta que no hay más datos disponibles en la entrada.

En algunos sistemas de concursos, se usan archivos para la entrada y la salida. Una solución sencilla para esto es escribir el código como de costumbre usando flujos estándar, pero añadir las siguientes líneas al principio del código:

```
freopen("input.txt", "r", stdin);  
freopen("output.txt", "w", stdout);
```

Después de esto, el programa lee la entrada del archivo "input.txt" y escribe la salida en el archivo "output.txt".

1.3. Trabajar con números

Enteros

El tipo entero más usado en programación competitiva es `int`, que es un tipo de 32 bits con un rango de valores de $-2^{31} \dots 2^{31} - 1$ o aproximadamente $-2 \cdot 10^9 \dots 2 \cdot 10^9$. Si el tipo `int` no es suficiente, puede usarse el tipo de 64 bits `long long`. Tiene un rango de valores de $-2^{63} \dots 2^{63} - 1$ o aproximadamente $-9 \cdot 10^{18} \dots 9 \cdot 10^{18}$.

El siguiente código define una variable `long long`:

```
long long x = 123456789123456789LL;
```

El sufijo `LL` significa que el tipo del número es `long long`.

Un error común al usar el tipo `long long` es que el tipo `int` se sigue usando en algún lugar del código. Por ejemplo, el siguiente código contiene un error sutil:

```
int a = 123456789;  
long long b = a*a;  
cout << b << "\n"; // -1757895751
```

Aunque la variable `b` es de tipo `long long`, ambos números en la expresión `a*a` son de tipo `int` y el resultado también es de tipo `int`. Por eso, la variable `b` contendrá un resultado erróneo. El problema puede resolverse cambiando el tipo de `a` a `long long` o cambiando la expresión a `(long long)a*a`.

Normalmente los problemas de los concursos se plantean de modo que el tipo `long long` sea suficiente. Aun así, es bueno saber que el compilador `g++` también proporciona un tipo de 128 bits `__int128_t` con un rango de valores de $-2^{127} \dots 2^{127} - 1$ o aproximadamente $-10^{38} \dots 10^{38}$. Sin embargo, este tipo no está disponible en todos los sistemas de concursos.

Aritmética modular

Denotamos por $x \bmod m$ el residuo cuando x se divide entre m . Por ejemplo, $17 \bmod 5 = 2$, porque $17 = 3 \cdot 5 + 2$.

A veces, la respuesta a un problema es un número muy grande pero basta con darla "módulo m ", es decir, el residuo cuando la respuesta se divide entre m (por ejemplo, "módulo $10^9 + 7$ "). La idea es que aunque la respuesta real sea muy grande, basta con usar los tipos `int` y `long long`.

Una propiedad importante del residuo es que en la suma, la resta y la multiplicación, el residuo puede tomarse antes de la operación:

$$\begin{aligned}(a + b) \bmod m &= (a \bmod m + b \bmod m) \bmod m \\(a - b) \bmod m &= (a \bmod m - b \bmod m) \bmod m \\(a \cdot b) \bmod m &= (a \bmod m \cdot b \bmod m) \bmod m\end{aligned}$$

Así, podemos tomar el residuo después de cada operación y los números nunca se harán demasiado grandes.

Por ejemplo, el siguiente código calcula $n!$, el factorial de n , módulo m :

```
long long x = 1;
for (int i = 2; i <= n; i++) {
    x = (x*i)%m;
}
cout << x%m << "\n";
```

Normalmente queremos que el residuo esté siempre entre $0 \dots m - 1$. Sin embargo, en C++ y otros lenguajes, el residuo de un número negativo es cero o negativo. Una forma sencilla de asegurarse de que no haya residuos negativos es calcular primero el residuo como de costumbre y luego sumar m si el resultado es negativo:

```
x = x%m;
if (x < 0) x += m;
```

Sin embargo, esto solo es necesario cuando hay restas en el código y el residuo puede volverse negativo.

Números de punto flotante

Los tipos de punto flotante habituales en programación competitiva son el `double` de 64 bits y, como extensión en el compilador `g++`, el `long double` de 80 bits. En la mayoría de los casos, `double` es suficiente, pero `long double` es más preciso.

La precisión requerida de la respuesta suele indicarse en el enunciado del problema. Una forma sencilla de mostrar la respuesta es usar la función `printf` e indicar el número de posiciones decimales en la cadena de formato. Por ejemplo, el siguiente código imprime el valor de x con 9 posiciones decimales:

```
printf("%.9f\n", x);
```

Una dificultad al usar números de punto flotante es que algunos números no pueden representarse con exactitud como números de punto flotante, y habrá errores de redondeo. Por ejemplo, el resultado del siguiente código es sorprendente:

```
double x = 0.3*3+0.1;
printf("%.20f\n", x); // 0.99999999999999988898
```

Debido a un error de redondeo, el valor de x es un poco menor que 1, mientras que el valor correcto sería 1.

Es arriesgado comparar números de punto flotante con el operador `==`, porque es posible que los valores debieran ser iguales pero no lo sean debido a errores de precisión. Una mejor forma de comparar números de punto flotante es suponer que dos números son iguales si la diferencia entre ellos es menor que ϵ , donde ϵ es un número pequeño.

En la práctica, los números pueden compararse de la siguiente manera ($\epsilon = 10^{-9}$):

```
if (abs(a-b) < 1e-9) {
    // a and b are equal
}
```

Nota que aunque los números de punto flotante son inexactos, los enteros hasta cierto límite todavía pueden representarse con exactitud. Por ejemplo, usando `double`, es posible representar con exactitud todos los enteros cuyo valor absoluto sea a lo sumo 2^{53} .

1.4. Acortar el código

El código corto es ideal en programación competitiva, porque los programas deben escribirse lo más rápido posible. Por eso, los programadores competitivos a menudo definen nombres más cortos para los tipos de datos y otras partes del código.

Nombres de tipos

Usando el comando `typedef` es posible dar un nombre más corto a un tipo de dato. Por ejemplo, el nombre `long long` es largo, así que podemos definir un nombre más corto `ll`:

```
typedef long long ll;
```

Después de esto, el código

```
long long a = 123456789;
long long b = 987654321;
cout << a*b << "\n";
```

puede acortarse de la siguiente manera:

```
ll a = 123456789;
ll b = 987654321;
cout << a*b << "\n";
```

El comando `typedef` también puede usarse con tipos más complejos. Por ejemplo, el siguiente código da el nombre `vi` a un vector de enteros y el nombre `pi` a un par que contiene dos enteros.

```
typedef vector<int> vi;
typedef pair<int,int> pi;
```

Macros

Otra forma de acortar el código es definir **macros**. Una macro significa que ciertas cadenas del código se cambiarán antes de la compilación. En C++, las macros se definen usando la palabra clave `#define`.

Por ejemplo, podemos definir las siguientes macros:

```
#define F first
#define S second
#define PB push_back
#define MP make_pair
```

Después de esto, el código

```
v.push_back(make_pair(y1,x1));
v.push_back(make_pair(y2,x2));
int d = v[i].first+v[i].second;
```

puede acortarse de la siguiente manera:

```
v.PB(MP(y1,x1));
v.PB(MP(y2,x2));
int d = v[i].F+v[i].S;
```

Una macro también puede tener parámetros lo que hace posible acortar bucles y otras estructuras. Por ejemplo, podemos definir la siguiente macro:

```
#define REP(i,a,b) for (int i = a; i <= b; i++)
```

Después de esto, el código

```
for (int i = 1; i <= n; i++) {
    search(i);
}
```

puede acortarse de la siguiente manera:

```
REP(i,1,n) {  
    search(i);  
}
```

A veces las macros provocan errores que pueden ser difíciles de detectar. Por ejemplo, considera la siguiente macro que calcula el cuadrado de un número:

```
#define SQ(a) a*a
```

Esta macro *no* siempre funciona como se espera. Por ejemplo, el código

```
cout << SQ(3+3) << "\n";
```

corresponde al código

```
cout << 3+3*3+3 << "\n"; // 15
```

Una mejor versión de la macro es la siguiente:

```
#define SQ(a) (a)*(a)
```

Ahora el código

```
cout << SQ(3+3) << "\n";
```

corresponde al código

```
cout << (3+3)*(3+3) << "\n"; // 36
```

1.5. Matemáticas

Las matemáticas desempeñan un papel importante en la programación competitiva, y no es posible llegar a ser un programador competitivo exitoso sin tener buenas habilidades matemáticas. Esta sección analiza algunos conceptos y fórmulas matemáticas importantes que se necesitarán más adelante en el libro.

Fórmulas de sumas

Cada suma de la forma

$$\sum_{x=1}^n x^k = 1^k + 2^k + 3^k + \dots + n^k,$$

donde k es un entero positivo, tiene una fórmula cerrada que es un polinomio de grado $k + 1$. Por ejemplo¹,

$$\sum_{x=1}^n x = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

y

$$\sum_{x=1}^n x^2 = 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}.$$

Una **progresión aritmética** es una sucesión de números donde la diferencia entre dos números consecutivos cualesquiera es constante. Por ejemplo,

$$3, 7, 11, 15$$

es una progresión aritmética con constante 4. La suma de una progresión aritmética puede calcularse usando la fórmula

$$\underbrace{a + \dots + b}_{n \text{ números}} = \frac{n(a+b)}{2}$$

donde a es el primer número, b es el último número y n es la cantidad de números. Por ejemplo,

$$3 + 7 + 11 + 15 = \frac{4 \cdot (3 + 15)}{2} = 36.$$

La fórmula se basa en el hecho de que la suma consta de n números y el valor de cada número es $(a+b)/2$ en promedio.

Una **progresión geométrica** es una sucesión de números donde la razón entre dos números consecutivos cualesquiera es constante. Por ejemplo,

$$3, 6, 12, 24$$

es una progresión geométrica con constante 2. La suma de una progresión geométrica puede calcularse usando la fórmula

$$a + ak + ak^2 + \dots + b = \frac{bk - a}{k - 1}$$

donde a es el primer número, b es el último número y la razón entre números consecutivos es k . Por ejemplo,

$$3 + 6 + 12 + 24 = \frac{24 \cdot 2 - 3}{2 - 1} = 45.$$

Esta fórmula puede deducirse de la siguiente manera. Sea

$$S = a + ak + ak^2 + \dots + b.$$

Multiplicando ambos lados por k , obtenemos

$$kS = ak + ak^2 + ak^3 + \dots + bk,$$

¹ Existe incluso una fórmula general para tales sumas, llamada **fórmula de Faulhaber**, pero es demasiado compleja para presentarla aquí.

y resolviendo la ecuación

$$kS - S = bk - a$$

se obtiene la fórmula.

Un caso especial de la suma de una progresión geométrica es la fórmula

$$1 + 2 + 4 + 8 + \dots + 2^{n-1} = 2^n - 1.$$

Una **suma armónica** es una suma de la forma

$$\sum_{x=1}^n \frac{1}{x} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}.$$

Una cota superior para una suma armónica es $\log_2(n) + 1$. En efecto, podemos modificar cada término $1/k$ de modo que k se convierta en la potencia de dos más cercana que no exceda a k . Por ejemplo, cuando $n = 6$, podemos estimar la suma de la siguiente manera:

$$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} \leq 1 + \frac{1}{2} + \frac{1}{2} + \frac{1}{4} + \frac{1}{4} + \frac{1}{4}.$$

Esta cota superior consta de $\log_2(n) + 1$ partes (1 , $2 \cdot 1/2$, $4 \cdot 1/4$, etc.), y el valor de cada parte es a lo sumo 1 .

Teoría de conjuntos

Un **conjunto** es una colección de elementos. Por ejemplo, el conjunto

$$X = \{2, 4, 7\}$$

contiene los elementos 2 , 4 y 7 . El símbolo $\emptyset$ denota un conjunto vacío, y $|S|$ denota el tamaño de un conjunto S , es decir, el número de elementos del conjunto. Por ejemplo, en el conjunto anterior, $|X| = 3$.

Si un conjunto S contiene un elemento x , escribimos $x \in S$, y en caso contrario escribimos $x \notin S$. Por ejemplo, en el conjunto anterior

$$4 \in X \quad \text{y} \quad 5 \notin X.$$

Pueden construirse nuevos conjuntos usando operaciones de conjuntos:

- La **intersección** $A \cap B$ consta de los elementos que están tanto en A como en B . Por ejemplo, si $A = \{1, 2, 5\}$ y $B = \{2, 4\}$, entonces $A \cap B = \{2\}$.
- La **unión** $A \cup B$ consta de los elementos que están en A o en B o en ambos. Por ejemplo, si $A = \{3, 7\}$ y $B = \{2, 3, 8\}$, entonces $A \cup B = \{2, 3, 7, 8\}$.
- El **complemento** $\bar{A}$ consta de los elementos que no están en A . La interpretación de un complemento depende del **conjunto universal**, que contiene todos los elementos posibles. Por ejemplo, si $A = \{1, 2, 5, 7\}$ y el conjunto universal es $\{1, 2, \dots, 10\}$, entonces $\bar{A} = \{3, 4, 6, 8, 9, 10\}$.
- La **diferencia** $A \setminus B = A \cap \bar{B}$ consta de los elementos que están en A pero no en B . Nota que B puede contener elementos que no están en A . Por ejemplo, si $A = \{2, 3, 7, 8\}$ y $B = \{3, 5, 8\}$, entonces $A \setminus B = \{2, 7\}$.

Si cada elemento de A también pertenece a S , decimos que A es un **subconjunto** de S , denotado por $A \subset S$. Un conjunto S siempre tiene $2^{|S|}$ subconjuntos,

incluido el conjunto vacío. Por ejemplo, los subconjuntos del conjunto $\{2, 4, 7\}$ son

$$\emptyset, \{2\}, \{4\}, \{7\}, \{2, 4\}, \{2, 7\}, \{4, 7\} \text{ y } \{2, 4, 7\}.$$

Algunos conjuntos de uso frecuente son $\mathbb{N}$ (números naturales), $\mathbb{Z}$ (enteros), $\mathbb{Q}$ (números racionales) y $\mathbb{R}$ (números reales). El conjunto $\mathbb{N}$ puede definirse de dos maneras, según la situación: o bien $\mathbb{N} = \{0, 1, 2, \dots\}$ o bien $\mathbb{N} = \{1, 2, 3, \dots\}$.

También podemos construir un conjunto usando una regla de la forma

$$\{f(n) : n \in S\},$$

donde $f(n)$ es alguna función. Este conjunto contiene todos los elementos de la forma $f(n)$, donde n es un elemento de S . Por ejemplo, el conjunto

$$X = \{2n : n \in \mathbb{Z}\}$$

contiene todos los enteros pares.

Lógica

El valor de una expresión lógica es **verdadero** (1) o **falso** (0). Los operadores lógicos más importantes son $\neg$ (**negación**), $\wedge$ (**conjunción**), $\vee$ (**disyunción**), $\Rightarrow$ (**implicación**) y $\Leftrightarrow$ (**equivalencia**). La siguiente tabla muestra los significados de estos operadores:

A	B	$\neg A$	$\neg B$	$A \wedge B$	$A \vee B$	$A \Rightarrow B$	$A \Leftrightarrow B$
0	0	1	1	0	0	1	1
0	1	1	0	0	1	1	0
1	0	0	1	0	1	0	0
1	1	0	0	1	1	1	1

La expresión $\neg A$ tiene el valor opuesto de A . La expresión $A \wedge B$ es verdadera si tanto A como B son verdaderas, y la expresión $A \vee B$ es verdadera si A o B o ambas son verdaderas. La expresión $A \Rightarrow B$ es verdadera si siempre que A es verdadera, también B es verdadera. La expresión $A \Leftrightarrow B$ es verdadera si A y B son ambas verdaderas o ambas falsas.

Un **predicado** es una expresión que es verdadera o falsa según sus parámetros. Los predicados suelen denotarse con letras mayúsculas. Por ejemplo, podemos definir un predicado $P(x)$ que es verdadero exactamente cuando x es un número primo. Usando esta definición, $P(7)$ es verdadero pero $P(8)$ es falso.

Un **cuantificador** conecta una expresión lógica con los elementos de un conjunto. Los cuantificadores más importantes son $\forall$ (**para todo**) y $\exists$ (**existe**). Por ejemplo,

$$\forall x(\exists y(y < x))$$

significa que para cada elemento x del conjunto, existe un elemento y en el conjunto tal que y es menor que x . Esto es verdadero en el conjunto de los enteros, pero falso en el conjunto de los números naturales.

Usando la notación descrita arriba, podemos expresar muchos tipos de proposiciones lógicas. Por ejemplo,

$$\forall x((x > 1 \wedge \neg P(x)) \Rightarrow (\exists a(\exists b(a > 1 \wedge b > 1 \wedge x = ab))))$$

significa que si un número x es mayor que 1 y no es un número primo, entonces existen números a y b que son mayores que 1 y cuyo producto es x . Esta proposición es verdadera en el conjunto de los enteros.

Funciones

La función $\lfloor x \rfloor$ redondea el número x hacia abajo a un entero, y la función $\lceil x \rceil$ redondea el número x hacia arriba a un entero. Por ejemplo,

$$\lfloor 3/2 \rfloor = 1 \quad \text{y} \quad \lceil 3/2 \rceil = 2.$$

Las funciones $\min(x_1, x_2, \dots, x_n)$ y $\max(x_1, x_2, \dots, x_n)$ dan el menor y el mayor de los valores $x_1, x_2, \dots, x_n$. Por ejemplo,

$$\min(1, 2, 3) = 1 \quad \text{y} \quad \max(1, 2, 3) = 3.$$

El **factorial** $n!$ puede definirse

$$\prod_{x=1}^n x = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$$

o recursivamente

$$\begin{aligned} 0! &= 1 \\ n! &= n \cdot (n-1)! \end{aligned}$$

Los **números de Fibonacci** aparecen en muchas situaciones. Pueden definirse recursivamente de la siguiente manera:

$$\begin{aligned} f(0) &= 0 \\ f(1) &= 1 \\ f(n) &= f(n-1) + f(n-2) \end{aligned}$$

Los primeros números de Fibonacci son

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots$$

También existe una fórmula cerrada para calcular los números de Fibonacci, que a veces se llama **fórmula de Binet**:

$$f(n) = \frac{(1 + \sqrt{5})^n - (1 - \sqrt{5})^n}{2^n \sqrt{5}}.$$

Logaritmos

El **logaritmo** de un número x se denota $\log_k(x)$, donde k es la base del logaritmo. Según la definición, $\log_k(x) = a$ exactamente cuando $k^a = x$.

Una propiedad útil de los logaritmos es que $\log_k(x)$ es igual al número de veces que tenemos que dividir x entre k antes de llegar al número 1. Por ejemplo, $\log_2(32) = 5$ porque se necesitan 5 divisiones entre 2:

$$32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

Los logaritmos se usan a menudo en el análisis de algoritmos, porque muchos algoritmos eficientes reducen algo a la mitad en cada paso. Por eso, podemos estimar la eficiencia de tales algoritmos usando logaritmos.

El logaritmo de un producto es

$$\log_k(ab) = \log_k(a) + \log_k(b),$$

y en consecuencia,

$$\log_k(x^n) = n \cdot \log_k(x).$$

Además, el logaritmo de un cociente es

$$\log_k\left(\frac{a}{b}\right) = \log_k(a) - \log_k(b).$$

Otra fórmula útil es

$$\log_u(x) = \frac{\log_k(x)}{\log_k(u)},$$

y usando esto, es posible calcular logaritmos en cualquier base si hay una forma de calcular logaritmos en alguna base fija.

El **logaritmo natural** $\ln(x)$ de un número x es un logaritmo cuya base es $e \approx 2.71828$. Otra propiedad de los logaritmos es que el número de dígitos de un entero x en base b es $\lfloor \log_b(x) \rfloor + 1$. Por ejemplo, la representación de 123 en base 2 es 1111011 y $\lfloor \log_2(123) \rfloor + 1 = 7$.

1.6. Concursos y recursos

IOI

La Olimpiada Internacional de Informática (IOI) es un concurso de programación anual para estudiantes de secundaria. Cada país puede enviar un equipo de cuatro estudiantes al concurso. Suele haber unos 300 participantes de 80 países.

La IOI consta de dos concursos de cinco horas de duración. En ambos concursos, se pide a los participantes que resuelvan tres tareas algorítmicas de dificultad variada. Las tareas se dividen en subtareas, cada una de las cuales tiene una puntuación asignada. Aunque los concursantes se dividen en equipos, compiten como individuos.

El temario de la IOI [41] regula los temas que pueden aparecer en las tareas de la IOI. Casi todos los temas del temario de la IOI están cubiertos por este libro.

Los participantes de la IOI se seleccionan mediante concursos nacionales. Antes de la IOI, se organizan muchos concursos regionales, como la Olimpiada Báltica de Informática (BOI), la Olimpiada Centroeuropa de Informática (CEOI) y la Olimpiada de Informática de Asia-Pacífico (APIO).

Algunos países organizan concursos de práctica en línea para futuros participantes de la IOI, como el Concurso Abierto Croata de Informática [11] y la Olimpiada de Computación de EE. UU. [68]. Además, hay una gran colección de problemas de concursos polacos disponible en línea [60].

ICPC

El Concurso Internacional de Programación Colegial (ICPC) es un concurso de programación anual para estudiantes universitarios. Cada equipo del concurso consta de tres estudiantes, y a diferencia de la IOI, los estudiantes trabajan juntos; solo hay una computadora disponible para cada equipo.

El ICPC consta de varias etapas, y finalmente los mejores equipos son invitados a las Finales Mundiales. Aunque hay decenas de miles de participantes en el concurso, solo hay un pequeño número² de plazas disponibles para la final, así que incluso avanzar a las finales es un gran logro en algunas regiones.

En cada concurso ICPC, los equipos tienen cinco horas de tiempo para resolver unos diez problemas algorítmicos. La solución de un problema se acepta solo si resuelve todos los casos de prueba de forma eficiente. Durante el concurso, los competidores pueden ver los resultados de otros equipos, pero durante la última hora el marcador se congela y no es posible ver los resultados de los últimos envíos.

Los temas que pueden aparecer en el ICPC no están tan bien especificados como los de la IOI. En cualquier caso, está claro que se necesita más conocimiento en el ICPC, especialmente más habilidades matemáticas.

Concursos en línea

También hay muchos concursos en línea que están abiertos para todos. En este momento, el sitio de concursos más activo es Codeforces, que organiza concursos aproximadamente cada semana. En Codeforces, los participantes se dividen en dos divisiones: los principiantes compiten en la Div2 y los programadores más experimentados en la Div1. Otros sitios de concursos son AtCoder, CS Academy, HackerRank y Topcoder.

Algunas empresas organizan concursos en línea con finales presenciales. Ejemplos de tales concursos son el Facebook Hacker Cup, el Google Code Jam y Yandex.Algorithm. Por supuesto, las empresas también usan esos concursos para reclutar: tener un buen desempeño en un concurso es una buena forma de demostrar las habilidades de uno.

²El número exacto de plazas para la final varía de año en año; en 2017, hubo 133 plazas para la final.

Libros

Ya hay algunos libros (además de este) que se centran en la programación competitiva y la resolución algorítmica de problemas:

- S. S. Skiena y M. A. Revilla: *Programming Challenges: The Programming Contest Training Manual* [59]
- S. Halim y F. Halim: *Competitive Programming 3: The New Lower Bound of Programming Contests* [33]
- K. Diks et al.: *Looking for a Challenge? The Ultimate Problem Set from the University of Warsaw Programming Competitions* [15]

Los dos primeros libros están pensados para principiantes, mientras que el último libro contiene material avanzado.

Por supuesto, los libros generales de algoritmos también son adecuados para los programadores competitivos. Algunos libros populares son:

- T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein: *Introduction to Algorithms* [13]
- J. Kleinberg y É. Tardos: *Algorithm Design* [45]
- S. S. Skiena: *The Algorithm Design Manual* [58]

Capítulo 2

Complejidad temporal

La eficiencia de los algoritmos es importante en la programación competitiva. Normalmente, es fácil diseñar un algoritmo que resuelva el problema lentamente, pero el verdadero reto es idear un algoritmo rápido. Si el algoritmo es demasiado lento, obtendrá solo puntos parciales o ningún punto en absoluto.

La **complejidad temporal** de un algoritmo estima cuánto tiempo usará el algoritmo para una entrada dada. La idea es representar la eficiencia como una función cuyo parámetro es el tamaño de la entrada. Al calcular la complejidad temporal, podemos averiguar si el algoritmo es lo bastante rápido sin implementarlo.

2.1. Reglas de cálculo

La complejidad temporal de un algoritmo se denota $O(\dots)$ donde los tres puntos representan alguna función. Normalmente, la variable n denota el tamaño de la entrada. Por ejemplo, si la entrada es un arreglo de números, n será el tamaño del arreglo, y si la entrada es una cadena, n será la longitud de la cadena.

Bucles

Una razón común por la que un algoritmo es lento es que contiene muchos bucles que recorren la entrada. Cuantos más bucles anidados contiene el algoritmo, más lento es. Si hay k bucles anidados, la complejidad temporal es $O(n^k)$.

Por ejemplo, la complejidad temporal del siguiente código es $O(n)$:

```
for (int i = 1; i <= n; i++) {  
    // code  
}
```

Y la complejidad temporal del siguiente código es $O(n^2)$:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        // code  
    }  
}
```

```
}
```

Orden de magnitud

Una complejidad temporal no nos dice el número exacto de veces que se ejecuta el código dentro de un bucle, sino que solo muestra el orden de magnitud. En los siguientes ejemplos, el código dentro del bucle se ejecuta $3n$, $n + 5$ y $\lceil n/2 \rceil$ veces, pero la complejidad temporal de cada código es $O(n)$.

```
for (int i = 1; i <= 3*n; i++) {  
    // code  
}
```

```
for (int i = 1; i <= n+5; i++) {  
    // code  
}
```

```
for (int i = 1; i <= n; i += 2) {  
    // code  
}
```

Como otro ejemplo, la complejidad temporal del siguiente código es $O(n^2)$:

```
for (int i = 1; i <= n; i++) {  
    for (int j = i+1; j <= n; j++) {  
        // code  
    }  
}
```

Fases

Si el algoritmo consta de fases consecutivas, la complejidad temporal total es la mayor complejidad temporal de una sola fase. La razón de esto es que la fase más lenta suele ser el cuello de botella del código.

Por ejemplo, el siguiente código consta de tres fases con complejidades temporales $O(n)$, $O(n^2)$ y $O(n)$. Así, la complejidad temporal total es $O(n^2)$.

```
for (int i = 1; i <= n; i++) {  
    // code  
}  
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        // code  
    }  
}
```

```
for (int i = 1; i <= n; i++) {  
    // code  
}
```

Varias variables

A veces la complejidad temporal depende de varios factores. En este caso, la fórmula de la complejidad temporal contiene varias variables.

Por ejemplo, la complejidad temporal del siguiente código es $O(nm)$:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= m; j++) {  
        // code  
    }  
}
```

Recursión

La complejidad temporal de una función recursiva depende del número de veces que se llama a la función y de la complejidad temporal de una sola llamada. La complejidad temporal total es el producto de esos valores.

Por ejemplo, considera la siguiente función:

```
void f(int n) {  
    if (n == 1) return;  
    f(n-1);  
}
```

La llamada $f(n)$ provoca n llamadas a la función, y la complejidad temporal de cada llamada es $O(1)$. Así, la complejidad temporal total es $O(n)$.

Como otro ejemplo, considera la siguiente función:

```
void g(int n) {  
    if (n == 1) return;  
    g(n-1);  
    g(n-1);  
}
```

En este caso cada llamada a la función genera otras dos llamadas, excepto para $n = 1$. Veamos qué ocurre cuando se llama a g con el parámetro n . La siguiente tabla muestra las llamadas a la función producidas por esta única llamada:

llamada a la función	número de llamadas
$g(n)$	1
$g(n-1)$	2
$g(n-2)$	4
...	...
$g(1)$	2^{n-1}

Con base en esto, la complejidad temporal es

$$1 + 2 + 4 + \dots + 2^{n-1} = 2^n - 1 = O(2^n).$$

2.2. Clases de complejidad

La siguiente lista contiene complejidades temporales comunes de los algoritmos:

- $O(1)$ El tiempo de ejecución de un algoritmo de **tiempo constante** no depende del tamaño de la entrada. Un algoritmo de tiempo constante típico es una fórmula directa que calcula la respuesta.
- $O(\log n)$ Un algoritmo **logarítmico** a menudo reduce a la mitad el tamaño de la entrada en cada paso. El tiempo de ejecución de tal algoritmo es logarítmico, porque $\log_2 n$ es igual al número de veces que n debe dividirse entre 2 para llegar a 1.
- $O(\sqrt{n})$ Un **algoritmo de raíz cuadrada** es más lento que $O(\log n)$ pero más rápido que $O(n)$. Una propiedad especial de las raíces cuadradas es que $\sqrt{n} = n/\sqrt{n}$, así que la raíz cuadrada $\sqrt{n}$ se encuentra, en cierto sentido, en el medio de la entrada.
- $O(n)$ Un algoritmo **lineal** recorre la entrada un número constante de veces. Esta suele ser la mejor complejidad temporal posible, porque normalmente es necesario acceder a cada elemento de la entrada al menos una vez antes de dar la respuesta.
- $O(n \log n)$ Esta complejidad temporal a menudo indica que el algoritmo ordena la entrada, porque la complejidad temporal de los algoritmos de ordenamiento eficientes es $O(n \log n)$. Otra posibilidad es que el algoritmo use una estructura de datos donde cada operación toma $O(\log n)$ de tiempo.
- $O(n^2)$ Un algoritmo **cuadrático** a menudo contiene dos bucles anidados. Es posible recorrer todos los pares de elementos de la entrada en tiempo $O(n^2)$.
- $O(n^3)$ Un algoritmo **cúbico** a menudo contiene tres bucles anidados. Es posible recorrer todas las ternas de elementos de la entrada en tiempo $O(n^3)$.
- $O(2^n)$ Esta complejidad temporal a menudo indica que el algoritmo itera sobre todos los subconjuntos de los elementos de la entrada. Por ejemplo, los subconjuntos de $\{1, 2, 3\}$ son $\emptyset$, $\{1\}$, $\{2\}$, $\{3\}$, $\{1, 2\}$, $\{1, 3\}$, $\{2, 3\}$ y $\{1, 2, 3\}$.

$O(n!)$ Esta complejidad temporal a menudo indica que el algoritmo itera sobre todas las permutaciones de los elementos de la entrada. Por ejemplo, las permutaciones de $\{1,2,3\}$ son $(1,2,3)$, $(1,3,2)$, $(2,1,3)$, $(2,3,1)$, $(3,1,2)$ y $(3,2,1)$.

Un algoritmo es **polinómico** si su complejidad temporal es a lo sumo $O(n^k)$ donde k es una constante. Todas las complejidades temporales anteriores excepto $O(2^n)$ y $O(n!)$ son polinómicas. En la práctica, la constante k suele ser pequeña, y por lo tanto una complejidad temporal polinómica significa a grandes rasgos que el algoritmo es *eficiente*.

La mayoría de los algoritmos de este libro son polinómicos. Aun así, hay muchos problemas importantes para los que no se conoce ningún algoritmo polinómico, es decir, nadie sabe cómo resolverlos de forma eficiente. Los problemas **NP-difíciles** son un conjunto importante de problemas, para los que no se conoce ningún algoritmo polinómico¹.

2.3. Estimar la eficiencia

Al calcular la complejidad temporal de un algoritmo, es posible comprobar, antes de implementar el algoritmo, que es lo bastante eficiente para el problema. El punto de partida para las estimaciones es el hecho de que una computadora moderna puede realizar algunos cientos de millones de operaciones en un segundo.

Por ejemplo, supongamos que el límite de tiempo de un problema es un segundo y el tamaño de la entrada es $n = 10^5$. Si la complejidad temporal es $O(n^2)$, el algoritmo realizará unas $(10^5)^2 = 10^{10}$ operaciones. Esto debería tomar al menos algunas decenas de segundos, así que el algoritmo parece ser demasiado lento para resolver el problema.

Por otro lado, dado el tamaño de la entrada, podemos intentar *adivinar* la complejidad temporal requerida del algoritmo que resuelve el problema. La siguiente tabla contiene algunas estimaciones útiles suponiendo un límite de tiempo de un segundo.

tamaño de la entrada	complejidad temporal requerida
$n \leq 10$	$O(n!)$
$n \leq 20$	$O(2^n)$
$n \leq 500$	$O(n^3)$
$n \leq 5000$	$O(n^2)$
$n \leq 10^6$	$O(n \log n)$ u $O(n)$
n es grande	$O(1)$ u $O(\log n)$

Por ejemplo, si el tamaño de la entrada es $n = 10^5$, probablemente se espere que la complejidad temporal del algoritmo sea $O(n)$ u $O(n \log n)$. Esta información

¹Un libro clásico sobre el tema es *Computers and Intractability: A Guide to the Theory of NP-Completeness*, de M. R. Garey y D. S. Johnson [28].

facilita el diseño del algoritmo, porque descarta enfoques que darían un algoritmo con una peor complejidad temporal.

Aun así, es importante recordar que una complejidad temporal es solo una estimación de la eficiencia, porque oculta los *factores constantes*. Por ejemplo, un algoritmo que se ejecuta en tiempo $O(n)$ puede realizar $n/2$ o $5n$ operaciones. Esto tiene un efecto importante en el tiempo de ejecución real del algoritmo.

2.4. Suma máxima de subarreglo

A menudo hay varios algoritmos posibles para resolver un problema de modo que sus complejidades temporales sean distintas. Esta sección analiza un problema clásico que tiene una solución directa en $O(n^3)$. Sin embargo, al diseñar un algoritmo mejor, es posible resolver el problema en tiempo $O(n^2)$ e incluso en tiempo $O(n)$.

Dado un arreglo de n números, nuestra tarea es calcular la **suma máxima de subarreglo**, es decir, la mayor suma posible de una secuencia de valores consecutivos del arreglo². El problema es interesante cuando puede haber valores negativos en el arreglo. Por ejemplo, en el arreglo

-1	2	4	-3	5	2	-5	2
----	---	---	----	---	---	----	---

el siguiente subarreglo produce la suma máxima 10:

-1	2	4	-3	5	2	-5	2
----	---	---	----	---	---	----	---

Suponemos que se permite un subarreglo vacío, así que la suma máxima de subarreglo es siempre al menos 0.

Algoritmo 1

Una forma directa de resolver el problema es recorrer todos los subarreglos posibles, calcular la suma de los valores de cada subarreglo y mantener la suma máxima. El siguiente código implementa este algoritmo:

```
int best = 0;
for (int a = 0; a < n; a++) {
    for (int b = a; b < n; b++) {
        int sum = 0;
        for (int k = a; k <= b; k++) {
            sum += array[k];
        }
        best = max(best, sum);
    }
}
cout << best << "\n";
```

²El libro *Programming Pearls*, de J. Bentley [8], popularizó el problema.

Las variables a y b fijan el primer y último índice del subarreglo, y la suma de los valores se calcula en la variable sum . La variable $best$ contiene la suma máxima encontrada durante la búsqueda.

La complejidad temporal del algoritmo es $O(n^3)$, porque consta de tres bucles anidados que recorren la entrada.

Algoritmo 2

Es fácil hacer el Algoritmo 1 más eficiente eliminando un bucle de él. Esto es posible calculando la suma al mismo tiempo que se mueve el extremo derecho del subarreglo. El resultado es el siguiente código:

```
int best = 0;
for (int a = 0; a < n; a++) {
    int sum = 0;
    for (int b = a; b < n; b++) {
        sum += array[b];
        best = max(best, sum);
    }
}
cout << best << "\n";
```

Después de este cambio, la complejidad temporal es $O(n^2)$.

Algoritmo 3

Sorprendentemente, es posible resolver el problema en tiempo $O(n)^3$, lo que significa que basta con un solo bucle. La idea es calcular, para cada posición del arreglo, la suma máxima de un subarreglo que termina en esa posición. Después de esto, la respuesta al problema es el máximo de esas sumas.

Considera el subproblema de encontrar el subarreglo de suma máxima que termina en la posición k . Hay dos posibilidades:

1. El subarreglo solo contiene el elemento en la posición k .
2. El subarreglo consta de un subarreglo que termina en la posición $k - 1$, seguido del elemento en la posición k .

En el último caso, como queremos encontrar un subarreglo con suma máxima, el subarreglo que termina en la posición $k - 1$ también debe tener la suma máxima. Así, podemos resolver el problema de forma eficiente calculando la suma máxima de subarreglo para cada posición final, de izquierda a derecha.

El siguiente código implementa el algoritmo:

```
int best = 0, sum = 0;
for (int k = 0; k < n; k++) {
```

³En [8], este algoritmo de tiempo lineal se atribuye a J. B. Kadane, y el algoritmo a veces se llama **algoritmo de Kadane**.

```

    sum = max(array[k], sum+array[k]);
    best = max(best, sum);
}
cout << best << "\n";

```

El algoritmo solo contiene un bucle que recorre la entrada, así que la complejidad temporal es $O(n)$. Esta es también la mejor complejidad temporal posible, porque cualquier algoritmo para el problema tiene que examinar todos los elementos del arreglo al menos una vez.

Comparación de eficiencia

Es interesante estudiar qué tan eficientes son los algoritmos en la práctica. La siguiente tabla muestra los tiempos de ejecución de los algoritmos anteriores para distintos valores de n en una computadora moderna.

En cada prueba, la entrada se generó aleatoriamente. El tiempo necesario para leer la entrada no se midió.

tamaño del arreglo n	Algoritmo 1	Algoritmo 2	Algoritmo 3
10^2	0.0 s	0.0 s	0.0 s
10^3	0.1 s	0.0 s	0.0 s
10^4	> 10.0 s	0.1 s	0.0 s
10^5	> 10.0 s	5.3 s	0.0 s
10^6	> 10.0 s	> 10.0 s	0.0 s
10^7	> 10.0 s	> 10.0 s	0.0 s

La comparación muestra que todos los algoritmos son eficientes cuando el tamaño de la entrada es pequeño, pero las entradas más grandes hacen notar diferencias notables en los tiempos de ejecución de los algoritmos. El Algoritmo 1 se vuelve lento cuando $n = 10^4$, y el Algoritmo 2 se vuelve lento cuando $n = 10^5$. Solo el Algoritmo 3 es capaz de procesar incluso las entradas más grandes de forma instantánea.

Capítulo 3

Ordenamiento

El **ordenamiento** es un problema fundamental del diseño de algoritmos. Muchos algoritmos eficientes usan el ordenamiento como subrutina, porque a menudo es más fácil procesar los datos si los elementos están en orden.

Por ejemplo, el problema "¿un arreglo contiene dos elementos iguales?" es fácil de resolver usando ordenamiento. Si el arreglo contiene dos elementos iguales, quedarán uno junto al otro después de ordenar, así que es fácil encontrarlos. De igual forma, el problema "¿cuál es el elemento más frecuente de un arreglo?" puede resolverse de manera similar.

Hay muchos algoritmos de ordenamiento, y también son buenos ejemplos de cómo aplicar distintas técnicas de diseño de algoritmos. Los algoritmos generales de ordenamiento eficientes funcionan en tiempo $O(n \log n)$, y muchos algoritmos que usan el ordenamiento como subrutina también tienen esta complejidad temporal.

3.1. Teoría del ordenamiento

El problema básico del ordenamiento es el siguiente:

Dado un arreglo que contiene n elementos, tu tarea es ordenar los elementos en orden creciente.

Por ejemplo, el arreglo

1	3	8	2	9	2	5	6
---	---	---	---	---	---	---	---

quedará como sigue después de ordenar:

1	2	2	3	5	6	8	9
---	---	---	---	---	---	---	---

Algoritmos $O(n^2)$

Los algoritmos sencillos para ordenar un arreglo funcionan en tiempo $O(n^2)$. Tales algoritmos son cortos y normalmente constan de dos bucles anidados. Un

famoso algoritmo de ordenamiento de tiempo $O(n^2)$ es el **ordenamiento de burbuja**, donde los elementos "burbujean" en el arreglo según sus valores.

El ordenamiento de burbuja consta de n rondas. En cada ronda, el algoritmo recorre los elementos del arreglo. Siempre que se encuentran dos elementos consecutivos que no están en el orden correcto, el algoritmo los intercambia. El algoritmo puede implementarse de la siguiente manera:

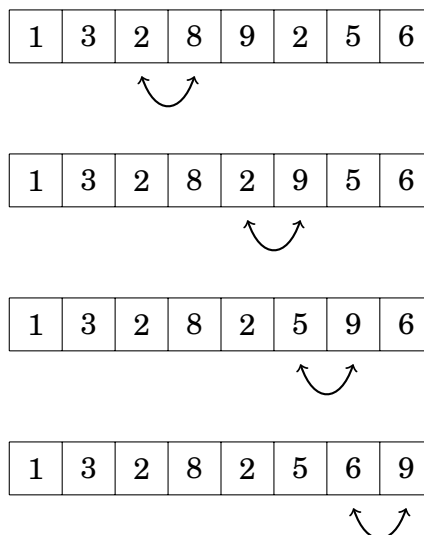
```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n-1; j++) {  
        if (array[j] > array[j+1]) {  
            swap(array[j], array[j+1]);  
        }  
    }  
}
```

Después de la primera ronda del algoritmo, el elemento más grande estará en la posición correcta, y en general, después de k rondas, los k elementos más grandes estarán en las posiciones correctas. Así, después de n rondas, todo el arreglo estará ordenado.

Por ejemplo, en el arreglo

1	3	8	2	9	2	5	6
---	---	---	---	---	---	---	---

la primera ronda del ordenamiento de burbuja intercambia elementos de la siguiente manera:



Inversiones

El ordenamiento de burbuja es un ejemplo de un algoritmo de ordenamiento que siempre intercambia elementos *consecutivos* del arreglo. Resulta que la complejidad temporal de tal algoritmo es *siempre* al menos $O(n^2)$, porque en el peor caso, se requieren $O(n^2)$ intercambios para ordenar el arreglo.

Un concepto útil al analizar algoritmos de ordenamiento es una **inversión**: un par de elementos del arreglo ($\text{array}[a], \text{array}[b]$) tal que $a < b$ y $\text{array}[a] > \text{array}[b]$, es decir, los elementos están en el orden incorrecto. Por ejemplo, el arreglo

1	2	2	6	3	5	9	8
---	---	---	---	---	---	---	---

tiene tres inversiones: (6,3), (6,5) y (9,8). El número de inversiones indica cuánto trabajo se necesita para ordenar el arreglo. Un arreglo está completamente ordenado cuando no hay inversiones. Por otro lado, si los elementos del arreglo están en el orden inverso, el número de inversiones es el mayor posible:

$$1 + 2 + \dots + (n - 1) = \frac{n(n - 1)}{2} = O(n^2)$$

Intercambiar un par de elementos consecutivos que están en el orden incorrecto elimina exactamente una inversión del arreglo. Por lo tanto, si un algoritmo de ordenamiento solo puede intercambiar elementos consecutivos, cada intercambio elimina a lo sumo una inversión, y la complejidad temporal del algoritmo es al menos $O(n^2)$.

Algoritmos $O(n \log n)$

Es posible ordenar un arreglo de forma eficiente en tiempo $O(n \log n)$ usando algoritmos que no se limitan a intercambiar elementos consecutivos. Uno de tales algoritmos es el **ordenamiento por mezcla**¹, que se basa en la recursión.

El ordenamiento por mezcla ordena un subarreglo $\text{array}[a \dots b]$ de la siguiente manera:

1. Si $a = b$, no hagas nada, porque el subarreglo ya está ordenado.
2. Calcula la posición del elemento del medio: $k = \lfloor (a + b)/2 \rfloor$.
3. Ordena recursivamente el subarreglo $\text{array}[a \dots k]$.
4. Ordena recursivamente el subarreglo $\text{array}[k + 1 \dots b]$.
5. *Mezcla* los subarreglos ordenados $\text{array}[a \dots k]$ y $\text{array}[k + 1 \dots b]$ en un subarreglo ordenado $\text{array}[a \dots b]$.

El ordenamiento por mezcla es un algoritmo eficiente, porque reduce a la mitad el tamaño del subarreglo en cada paso. La recursión consta de $O(\log n)$ niveles, y procesar cada nivel toma $O(n)$ de tiempo. Mezclar los subarreglos $\text{array}[a \dots k]$ y $\text{array}[k + 1 \dots b]$ es posible en tiempo lineal, porque ya están ordenados.

Por ejemplo, considera ordenar el siguiente arreglo:

1	3	6	2	8	2	5	9
---	---	---	---	---	---	---	---

¹Según [47], el ordenamiento por mezcla fue inventado por J. von Neumann en 1945.

El arreglo se dividirá en dos subarreglos de la siguiente manera:

1	3	6	2	8	2	5	9
---	---	---	---	---	---	---	---

Luego, los subarreglos se ordenarán recursivamente de la siguiente manera:

1	2	3	6	2	5	8	9
---	---	---	---	---	---	---	---

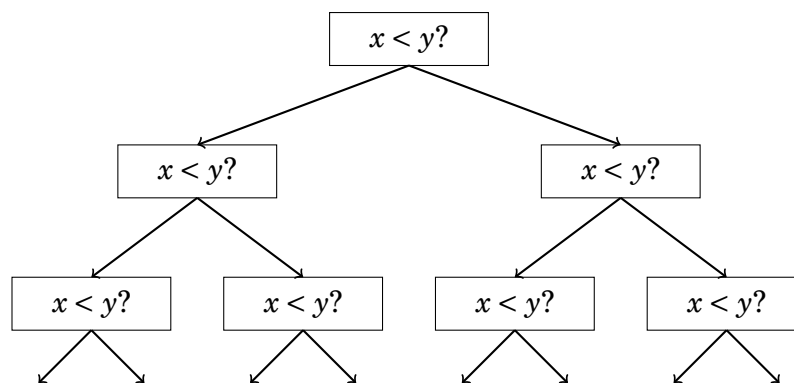
Finalmente, el algoritmo mezcla los subarreglos ordenados y crea el arreglo ordenado final:

1	2	2	3	5	6	8	9
---	---	---	---	---	---	---	---

Cota inferior del ordenamiento

¿Es posible ordenar un arreglo más rápido que en tiempo $O(n \log n)$? Resulta que esto *no* es posible cuando nos restringimos a algoritmos de ordenamiento que se basan en comparar elementos del arreglo.

La cota inferior de la complejidad temporal puede demostrarse considerando el ordenamiento como un proceso donde cada comparación de dos elementos da más información sobre el contenido del arreglo. El proceso crea el siguiente árbol:



Aquí " $x < y$?" significa que se comparan algunos elementos x y y . Si $x < y$, el proceso continúa hacia la izquierda, y en caso contrario hacia la derecha. Los resultados del proceso son las posibles formas de ordenar el arreglo, un total de $n!$ formas. Por esta razón, la altura del árbol debe ser al menos

$$\log_2(n!) = \log_2(1) + \log_2(2) + \cdots + \log_2(n).$$

Obtenemos una cota inferior para esta suma eligiendo los últimos $n/2$ elementos y cambiando el valor de cada elemento a $\log_2(n/2)$. Esto produce una estimación

$$\log_2(n!) \geq (n/2) \cdot \log_2(n/2),$$

así que la altura del árbol y el número mínimo posible de pasos en un algoritmo de ordenamiento en el peor caso es al menos $n \log n$.

Ordenamiento por conteo

La cota inferior $n \log n$ no se aplica a algoritmos que no comparan elementos del arreglo sino que usan alguna otra información. Un ejemplo de tal algoritmo es el **ordenamiento por conteo**, que ordena un arreglo en tiempo $O(n)$ suponiendo que todo elemento del arreglo es un entero entre $0 \dots c$ y $c = O(n)$.

El algoritmo crea un arreglo de *conteo*, cuyos índices son los elementos del arreglo original. El algoritmo recorre el arreglo original y calcula cuántas veces aparece cada elemento en el arreglo.

Por ejemplo, el arreglo

1	3	6	9	9	3	5	9
---	---	---	---	---	---	---	---

corresponde al siguiente arreglo de conteo:

1	2	3	4	5	6	7	8	9
1	0	2	0	1	1	0	0	3

Por ejemplo, el valor en la posición 3 del arreglo de conteo es 2, porque el elemento 3 aparece 2 veces en el arreglo original.

La construcción del arreglo de conteo toma $O(n)$ de tiempo. Después de esto, el arreglo ordenado puede crearse en tiempo $O(n)$ porque el número de apariciones de cada elemento puede recuperarse del arreglo de conteo. Así, la complejidad temporal total del ordenamiento por conteo es $O(n)$.

El ordenamiento por conteo es un algoritmo muy eficiente pero solo puede usarse cuando la constante c es lo bastante pequeña, de modo que los elementos del arreglo puedan usarse como índices en el arreglo de conteo.

3.2. Ordenamiento en C++

Casi nunca es buena idea usar un algoritmo de ordenamiento hecho en casa en un concurso, porque hay buenas implementaciones disponibles en los lenguajes de programación. Por ejemplo, la biblioteca estándar de C++ contiene la función `sort` que puede usarse fácilmente para ordenar arreglos y otras estructuras de datos.

Usar una función de biblioteca tiene muchos beneficios. Primero, ahorra tiempo porque no hay necesidad de implementar la función. Segundo, la implementación de la biblioteca es sin duda correcta y eficiente: no es probable que una función de ordenamiento hecha en casa sea mejor.

En esta sección veremos cómo usar la función `sort` de C++. El siguiente código ordena un vector en orden creciente:

```
vector<int> v = {4,2,5,3,5,8,3};
sort(v.begin(),v.end());
```

Después del ordenamiento, el contenido del vector será `[2,3,3,4,5,5,8]`. El orden de ordenamiento por defecto es creciente, pero un orden inverso es posible de la siguiente manera:

```
sort(v.rbegin(),v.rend());
```

Un arreglo ordinario puede ordenarse de la siguiente manera:

```
int n = 7; // array size
int a[] = {4,2,5,3,5,8,3};
sort(a,a+n);
```

El siguiente código ordena la cadena s:

```
string s = "monkey";  
sort(s.begin(), s.end());
```

Ordenar una cadena significa que los caracteres de la cadena se ordenan. Por ejemplo, la cadena "monkey" se convierte en "ekmnoy".

Operadores de comparación

La función `sort` requiere que se defina un **operador de comparación** para el tipo de dato de los elementos que se van a ordenar. Al ordenar, este operador se usará siempre que sea necesario averiguar el orden de dos elementos.

La mayoría de los tipos de datos de C++ tienen un operador de comparación integrado, y los elementos de esos tipos pueden ordenarse automáticamente. Por ejemplo, los números se ordenan según sus valores y las cadenas se ordenan en orden alfabético.

Los pares (`pair`) se ordenan principalmente según sus primeros elementos (`first`). Sin embargo, si los primeros elementos de dos pares son iguales, se ordenan según sus segundos elementos (`second`):

```
vector<pair<int,int>> v;  
v.push_back({1,5});  
v.push_back({2,3});  
v.push_back({1,2});  
sort(v.begin(), v.end());
```

Después de esto, el orden de los pares es (1,2), (1,5) y (2,3).

De manera similar, las tuplas (`tuple`) se ordenan principalmente por el primer elemento, secundariamente por el segundo elemento, etc.²:

```
vector<tuple<int,int,int>> v;  
v.push_back({2,1,4});  
v.push_back({1,5,3});  
v.push_back({2,1,3});  
sort(v.begin(), v.end());
```

Después de esto, el orden de las tuplas es (1,5,3), (2,1,3) y (2,1,4).

Structs definidos por el usuario

Los structs definidos por el usuario no tienen un operador de comparación automáticamente. El operador debe definirse dentro del struct como una función `operator<`, cuyo parámetro es otro elemento del mismo tipo. El operador debe devolver `true` si el elemento es menor que el parámetro, y `false` en caso contrario.

²Nota que en algunos compiladores más antiguos, hay que usar la función `make_tuple` para crear una tupla en lugar de llaves (por ejemplo, `make_tuple(2,1,4)` en vez de `{2,1,4}`).

Por ejemplo, el siguiente struct P contiene las coordenadas x e y de un punto. El operador de comparación se define de modo que los puntos se ordenen principalmente por la coordenada x y secundariamente por la coordenada y.

```
struct P {
    int x, y;
    bool operator<(const P &p) {
        if (x != p.x) return x < p.x;
        else return y < p.y;
    }
};
```

Funciones de comparación

También es posible dar una **función de comparación** externa a la función sort como una función de retorno (callback). Por ejemplo, la siguiente función de comparación comp ordena cadenas principalmente por longitud y secundariamente por orden alfabético:

```
bool comp(string a, string b) {
    if (a.size() != b.size()) return a.size() < b.size();
    return a < b;
}
```

Ahora un vector de cadenas puede ordenarse de la siguiente manera:

```
sort(v.begin(), v.end(), comp);
```

3.3. Búsqueda binaria

Un método general para buscar un elemento en un arreglo es usar un bucle for que recorra los elementos del arreglo. Por ejemplo, el siguiente código busca un elemento x en un arreglo:

```
for (int i = 0; i < n; i++) {
    if (array[i] == x) {
        // x found at index i
    }
}
```

La complejidad temporal de este enfoque es $O(n)$, porque en el peor caso, es necesario revisar todos los elementos del arreglo. Si el orden de los elementos es arbitrario, este es también el mejor enfoque posible, porque no hay información adicional disponible sobre en qué parte del arreglo deberíamos buscar el elemento x .

Sin embargo, si el arreglo está *ordenado*, la situación es distinta. En este caso es posible realizar la búsqueda mucho más rápido, porque el orden de los elementos del arreglo guía la búsqueda. El siguiente algoritmo de **búsqueda binaria** busca eficientemente un elemento en un arreglo ordenado en tiempo $O(\log n)$.

Método 1

La forma habitual de implementar la búsqueda binaria se asemeja a buscar una palabra en un diccionario. La búsqueda mantiene una región activa en el arreglo, que inicialmente contiene todos los elementos del arreglo. Luego, se realiza una serie de pasos, cada uno de los cuales reduce a la mitad el tamaño de la región.

En cada paso, la búsqueda revisa el elemento del medio de la región activa. Si el elemento del medio es el elemento buscado, la búsqueda termina. En caso contrario, la búsqueda continúa recursivamente hacia la mitad izquierda o derecha de la región, según el valor del elemento del medio.

La idea anterior puede implementarse de la siguiente manera:

```
int a = 0, b = n-1;
while (a <= b) {
    int k = (a+b)/2;
    if (array[k] == x) {
        // x found at index k
    }
    if (array[k] > x) b = k-1;
    else a = k+1;
}
```

En esta implementación, la región activa es $a \dots b$, e inicialmente la región es $0 \dots n-1$. El algoritmo reduce a la mitad el tamaño de la región en cada paso, así que la complejidad temporal es $O(\log n)$.

Método 2

Un método alternativo para implementar la búsqueda binaria se basa en una forma eficiente de recorrer los elementos del arreglo. La idea es hacer saltos y reducir la velocidad cuando nos acercamos al elemento buscado.

La búsqueda recorre el arreglo de izquierda a derecha, y la longitud inicial del salto es $n/2$. En cada paso, la longitud del salto se reduce a la mitad: primero $n/4$, luego $n/8$, $n/16$, etc., hasta que finalmente la longitud es 1. Después de los saltos, o bien se ha encontrado el elemento buscado o sabemos que no aparece en el arreglo.

El siguiente código implementa la idea anterior:

```
int k = 0;
for (int b = n/2; b >= 1; b /= 2) {
    while (k+b < n && array[k+b] <= x) k += b;
```

```

}
if (array[k] == x) {
    // x found at index k
}

```

Durante la búsqueda, la variable b contiene la longitud actual del salto. La complejidad temporal del algoritmo es $O(\log n)$, porque el código del bucle `while` se ejecuta a lo sumo dos veces por cada longitud de salto.

Funciones de C++

La biblioteca estándar de C++ contiene las siguientes funciones que se basan en la búsqueda binaria y funcionan en tiempo logarítmico:

- `lower_bound` devuelve un puntero al primer elemento del arreglo cuyo valor es al menos x .
- `upper_bound` devuelve un puntero al primer elemento del arreglo cuyo valor es mayor que x .
- `equal_range` devuelve ambos punteros anteriores.

Las funciones suponen que el arreglo está ordenado. Si no existe tal elemento, el puntero apunta al elemento posterior al último elemento del arreglo. Por ejemplo, el siguiente código averigua si un arreglo contiene un elemento con valor x :

```

auto k = lower_bound(array, array+n, x) - array;
if (k < n && array[k] == x) {
    // x found at index k
}

```

Luego, el siguiente código cuenta el número de elementos cuyo valor es x :

```

auto a = lower_bound(array, array+n, x);
auto b = upper_bound(array, array+n, x);
cout << b-a << "\n";

```

Usando `equal_range`, el código se vuelve más corto:

```

auto r = equal_range(array, array+n, x);
cout << r.second-r.first << "\n";

```

Encontrar la solución más pequeña

Un uso importante de la búsqueda binaria es encontrar la posición donde cambia el valor de una *función*. Supongamos que deseamos encontrar el valor más pequeño k que es una solución válida para un problema. Se nos da una

función $ok(x)$ que devuelve true si x es una solución válida y false en caso contrario. Además, sabemos que $ok(x)$ es false cuando $x < k$ y true cuando $x \geq k$. La situación se ve de la siguiente manera:

x	0	1	...	$k-1$	k	$k+1$	...
$ok(x)$	false	false	...	false	true	true	...

Ahora, el valor de k puede encontrarse usando búsqueda binaria:

```
int x = -1;
for (int b = z; b >= 1; b /= 2) {
    while (!ok(x+b)) x += b;
}
int k = x+1;
```

La búsqueda encuentra el mayor valor de x para el cual $ok(x)$ es false. Así, el siguiente valor $k = x + 1$ es el menor valor posible para el cual $ok(k)$ es true. La longitud inicial del salto z tiene que ser lo bastante grande, por ejemplo algún valor para el cual sepamos de antemano que $ok(z)$ es true.

El algoritmo llama a la función ok $O(\log z)$ veces, así que la complejidad temporal total depende de la función ok . Por ejemplo, si la función funciona en tiempo $O(n)$, la complejidad temporal total es $O(n \log z)$.

Encontrar el valor máximo

La búsqueda binaria también puede usarse para encontrar el valor máximo de una función que es primero creciente y luego decreciente. Nuestra tarea es encontrar una posición k tal que

- $f(x) < f(x+1)$ cuando $x < k$, y
- $f(x) > f(x+1)$ cuando $x \geq k$.

La idea es usar búsqueda binaria para encontrar el mayor valor de x para el cual $f(x) < f(x+1)$. Esto implica que $k = x + 1$ porque $f(x+1) > f(x+2)$. El siguiente código implementa la búsqueda:

```
int x = -1;
for (int b = z; b >= 1; b /= 2) {
    while (f(x+b) < f(x+b+1)) x += b;
}
int k = x+1;
```

Nota que a diferencia de la búsqueda binaria ordinaria, aquí no se permite que valores consecutivos de la función sean iguales. En ese caso no sería posible saber cómo continuar la búsqueda.

Capítulo 4

Estructuras de datos

Una **estructura de datos** es una forma de almacenar datos en la memoria de una computadora. Es importante elegir una estructura de datos apropiada para un problema, porque cada estructura de datos tiene sus propias ventajas y desventajas. La pregunta crucial es: ¿qué operaciones son eficientes en la estructura de datos elegida?

Este capítulo presenta las estructuras de datos más importantes de la biblioteca estándar de C++. Es buena idea usar la biblioteca estándar siempre que sea posible, porque ahorrará mucho tiempo. Más adelante en el libro aprenderemos sobre estructuras de datos más sofisticadas que no están disponibles en la biblioteca estándar.

4.1. Arreglos dinámicos

Un **arreglo dinámico** es un arreglo cuyo tamaño puede cambiarse durante la ejecución del programa. El arreglo dinámico más popular en C++ es la estructura `vector`, que puede usarse casi como un arreglo ordinario.

El siguiente código crea un vector vacío y le agrega tres elementos:

```
vector<int> v;  
v.push_back(3); // [3]  
v.push_back(2); // [3,2]  
v.push_back(5); // [3,2,5]
```

Después de esto, se puede acceder a los elementos como en un arreglo ordinario:

```
cout << v[0] << "\n"; // 3  
cout << v[1] << "\n"; // 2  
cout << v[2] << "\n"; // 5
```

La función `size` devuelve el número de elementos del vector. El siguiente código recorre el vector e imprime todos sus elementos:

```
for (int i = 0; i < v.size(); i++) {
```

```
    cout << v[i] << "\n";  
}
```

Una forma más corta de recorrer un vector es la siguiente:

```
for (auto x : v) {  
    cout << x << "\n";  
}
```

La función `back` devuelve el último elemento del vector, y la función `pop_back` elimina el último elemento:

```
vector<int> v;  
v.push_back(5);  
v.push_back(2);  
cout << v.back() << "\n"; // 2  
v.pop_back();  
cout << v.back() << "\n"; // 5
```

El siguiente código crea un vector con cinco elementos:

```
vector<int> v = {2,4,2,5,1};
```

Otra forma de crear un vector es dar el número de elementos y el valor inicial de cada elemento:

```
// size 10, initial value 0  
vector<int> v(10);
```

```
// size 10, initial value 5  
vector<int> v(10, 5);
```

La implementación interna de un vector usa un arreglo ordinario. Si el tamaño del vector aumenta y el arreglo se vuelve demasiado pequeño, se reserva un nuevo arreglo y todos los elementos se mueven al nuevo arreglo. Sin embargo, esto no ocurre a menudo y la complejidad temporal promedio de `push_back` es $O(1)$.

La estructura `string` es también un arreglo dinámico que puede usarse casi como un vector. Además, hay una sintaxis especial para las cadenas que no está disponible en otras estructuras de datos. Las cadenas pueden combinarse usando el símbolo `+`. La función `substr(k,x)` devuelve la subcadena que comienza en la posición *k* y tiene longitud *x*, y la función `find(t)` encuentra la posición de la primera aparición de una subcadena *t*.

El siguiente código presenta algunas operaciones con cadenas:

```
string a = "hatti";  
string b = a+a;  
cout << b << "\n"; // hattihatti  
b[5] = 'v';
```

```
cout << b << "\n"; // hattivatti
string c = b.substr(3,4);
cout << c << "\n"; // tiva
```

4.2. Estructuras de conjunto

Un **conjunto** es una estructura de datos que mantiene una colección de elementos. Las operaciones básicas de los conjuntos son la inserción, la búsqueda y la eliminación de elementos.

La biblioteca estándar de C++ contiene dos implementaciones de conjunto: La estructura `set` se basa en un árbol binario balanceado y sus operaciones funcionan en tiempo $O(\log n)$. La estructura `unordered_set` usa hashing, y sus operaciones funcionan en tiempo $O(1)$ en promedio.

La elección de qué implementación de conjunto usar suele ser cuestión de gustos. El beneficio de la estructura `set` es que mantiene el orden de los elementos y ofrece funciones que no están disponibles en `unordered_set`. Por otro lado, `unordered_set` puede ser más eficiente.

El siguiente código crea un conjunto que contiene enteros, y muestra algunas de las operaciones. La función `insert` agrega un elemento al conjunto, la función `count` devuelve el número de apariciones de un elemento en el conjunto, y la función `erase` elimina un elemento del conjunto.

```
set<int> s;
s.insert(3);
s.insert(2);
s.insert(5);
cout << s.count(3) << "\n"; // 1
cout << s.count(4) << "\n"; // 0
s.erase(3);
s.insert(4);
cout << s.count(3) << "\n"; // 0
cout << s.count(4) << "\n"; // 1
```

Un conjunto puede usarse en su mayoría como un vector, pero no es posible acceder a los elementos usando la notación `[]`. El siguiente código crea un conjunto, imprime el número de elementos que contiene, y luego recorre todos los elementos:

```
set<int> s = {2,5,6,8};
cout << s.size() << "\n"; // 4
for (auto x : s) {
    cout << x << "\n";
}
```

Una propiedad importante de los conjuntos es que todos sus elementos son *distintos*. Así, la función `count` siempre devuelve 0 (el elemento no está en el

conjunto) o 1 (el elemento está en el conjunto), y la función `insert` nunca agrega un elemento al conjunto si ya está ahí. El siguiente código ilustra esto:

```
set<int> s;
s.insert(5);
s.insert(5);
s.insert(5);
cout << s.count(5) << "\n"; // 1
```

C++ también contiene las estructuras `multiset` y `unordered_multiset` que por lo demás funcionan como `set` y `unordered_set` pero pueden contener varias instancias de un elemento. Por ejemplo, en el siguiente código las tres instancias del número 5 se agregan a un `multiset`:

```
multiset<int> s;
s.insert(5);
s.insert(5);
s.insert(5);
cout << s.count(5) << "\n"; // 3
```

La función `erase` elimina todas las instancias de un elemento de un `multiset`:

```
s.erase(5);
cout << s.count(5) << "\n"; // 0
```

A menudo, solo debería eliminarse una instancia, lo que puede hacerse de la siguiente manera:

```
s.erase(s.find(5));
cout << s.count(5) << "\n"; // 2
```

4.3. Estructuras de mapa

Un **mapa** es un arreglo generalizado que consta de pares clave-valor. Mientras que las claves en un arreglo ordinario son siempre los enteros consecutivos $0, 1, \dots, n-1$, donde n es el tamaño del arreglo, las claves en un mapa pueden ser de cualquier tipo de dato y no tienen que ser valores consecutivos.

La biblioteca estándar de C++ contiene dos implementaciones de mapa que corresponden a las implementaciones de conjunto: la estructura `map` se basa en un árbol binario balanceado y acceder a los elementos toma $O(\log n)$ de tiempo, mientras que la estructura `unordered_map` usa hashing y acceder a los elementos toma $O(1)$ de tiempo en promedio.

El siguiente código crea un mapa donde las claves son cadenas y los valores son enteros:

```
map<string, int> m;
m["monkey"] = 4;
```



```
m["banana"] = 3;
m["harpsichord"] = 9;
cout << m["banana"] << "\n"; // 3
```

Si se solicita el valor de una clave pero el mapa no la contiene, la clave se agrega automáticamente al mapa con un valor por defecto. Por ejemplo, en el siguiente código, la clave "aybaltu" con valor 0 se agrega al mapa.

```
map<string,int> m;
cout << m["aybaltu"] << "\n"; // 0
```

La función `count` comprueba si una clave existe en un mapa:

```
if (m.count("aybaltu")) {
    // key exists
}
```

El siguiente código imprime todas las claves y valores de un mapa:

```
for (auto x : m) {
    cout << x.first << " " << x.second << "\n";
}
```

4.4. Iteradores y rangos

Muchas funciones de la biblioteca estándar de C++ operan con iteradores. Un **iterador** es una variable que apunta a un elemento de una estructura de datos.

Los iteradores de uso frecuente `begin` y `end` definen un rango que contiene todos los elementos de una estructura de datos. El iterador `begin` apunta al primer elemento de la estructura de datos, y el iterador `end` apunta a la posición *posterior* al último elemento. La situación se ve de la siguiente manera:

```

{ 3, 4, 6, 8, 12, 13, 14, 17 }
  ↑                               ↑
  s.begin()                       s.end()

```

Nota la asimetría en los iteradores: `s.begin()` apunta a un elemento de la estructura de datos, mientras que `s.end()` apunta fuera de la estructura de datos. Así, el rango definido por los iteradores es *semiabierto*.

Trabajar con rangos

Los iteradores se usan en las funciones de la biblioteca estándar de C++ a las que se les da un rango de elementos de una estructura de datos. Normalmente, queremos procesar todos los elementos de una estructura de datos, así que se le dan a la función los iteradores `begin` y `end`.

Por ejemplo, el siguiente código ordena un vector usando la función `sort`, luego invierte el orden de los elementos usando la función `reverse`, y finalmente mezcla el orden de los elementos usando la función `random_shuffle`.

```
sort(v.begin(), v.end());  
reverse(v.begin(), v.end());  
random_shuffle(v.begin(), v.end());
```

Estas funciones también pueden usarse con un arreglo ordinario. En este caso, a las funciones se les dan punteros al arreglo en lugar de iteradores:

```
sort(a, a+n);
reverse(a, a+n);
random_shuffle(a, a+n);
```

Iteradores de conjunto

Los iteradores se usan a menudo para acceder a los elementos de un conjunto. El siguiente código crea un iterador `it` que apunta al elemento más pequeño de un conjunto:

```
set<int>::iterator it = s.begin();
```

Una forma más corta de escribir el código es la siguiente:

```
auto it = s.begin();
```

Al elemento al que apunta un iterador se puede acceder usando el símbolo `*`. Por ejemplo, el siguiente código imprime el primer elemento del conjunto:

```
auto it = s.begin();
cout << *it << "\n";
```

Los iteradores pueden moverse usando los operadores `++` (hacia adelante) y `--` (hacia atrás), lo que significa que el iterador se mueve al siguiente o al anterior elemento del conjunto.

El siguiente código imprime todos los elementos en orden creciente:

```
for (auto it = s.begin(); it != s.end(); it++) {
    cout << *it << "\n";
}
```

El siguiente código imprime el elemento más grande del conjunto:

```
auto it = s.end(); it--;
cout << *it << "\n";
```

La función `find(x)` devuelve un iterador que apunta a un elemento cuyo valor es `x`. Sin embargo, si el conjunto no contiene `x`, el iterador será `end`.

```
auto it = s.find(x);
if (it == s.end()) {
    // x is not found
}
```

La función `lower_bound(x)` devuelve un iterador al elemento más pequeño del conjunto cuyo valor es *al menos* `x`, y la función `upper_bound(x)` devuelve un iterador al elemento más pequeño del conjunto cuyo valor es *mayor que* `x`. En ambas funciones, si tal elemento no existe, el valor devuelto es `end`. Estas

funciones no son compatibles con la estructura `unordered_set` que no mantiene el orden de los elementos.

Por ejemplo, el siguiente código encuentra el elemento más cercano a x :

```
auto it = s.lower_bound(x);
if (it == s.begin()) {
    cout << *it << "\n";
} else if (it == s.end()) {
    it--;
    cout << *it << "\n";
} else {
    int a = *it; it--;
    int b = *it;
    if (x-b < a-x) cout << b << "\n";
    else cout << a << "\n";
}
```

El código supone que el conjunto no está vacío, y recorre todos los casos posibles usando un iterador `it`. Primero, el iterador apunta al elemento más pequeño cuyo valor es al menos x . Si `it` es igual a `begin`, el elemento correspondiente es el más cercano a x . Si `it` es igual a `end`, el elemento más grande del conjunto es el más cercano a x . Si ninguno de los casos anteriores se cumple, el elemento más cercano a x es o bien el elemento que corresponde a `it` o el elemento anterior.

4.5. Otras estructuras

Bitset

Un **bitset** es un arreglo cuyo cada valor es 0 o 1. Por ejemplo, el siguiente código crea un `bitset` que contiene 10 elementos:

```
bitset<10> s;
s[1] = 1;
s[3] = 1;
s[4] = 1;
s[7] = 1;
cout << s[4] << "\n"; // 1
cout << s[5] << "\n"; // 0
```

El beneficio de usar `bitsets` es que requieren menos memoria que los arreglos ordinarios, porque cada elemento de un `bitset` solo usa un bit de memoria. Por ejemplo, si se almacenan n bits en un arreglo de `int`, se usarán $32n$ bits de memoria, pero un `bitset` correspondiente solo requiere n bits de memoria. Además, los valores de un `bitset` pueden manipularse de forma eficiente usando operadores de bits, lo que hace posible optimizar algoritmos usando `bitsets`.

El siguiente código muestra otra forma de crear el `bitset` anterior:

```
bitset<10> s(string("0010011010")); // from right to left
```

```
cout << s[4] << "\n"; // 1
cout << s[5] << "\n"; // 0
```

La función `count` devuelve el número de unos del `bitset`:

```
bitset<10> s(string("0010011010"));
cout << s.count() << "\n"; // 4
```

El siguiente código muestra ejemplos del uso de operaciones de bits:

```
bitset<10> a(string("0010110110"));
bitset<10> b(string("1011011000"));
cout << (a&b) << "\n"; // 0010010000
cout << (a|b) << "\n"; // 1011111110
cout << (a^b) << "\n"; // 1001101110
```

Deque

Un **deque** es un arreglo dinámico cuyo tamaño puede cambiarse de forma eficiente en ambos extremos del arreglo. Como un vector, un deque ofrece las funciones `push_back` y `pop_back`, pero también incluye las funciones `push_front` y `pop_front` que no están disponibles en un vector.

Un deque puede usarse de la siguiente manera:

```
deque<int> d;
d.push_back(5); // [5]
d.push_back(2); // [5,2]
d.push_front(3); // [3,5,2]
d.pop_back(); // [3,5]
d.pop_front(); // [5]
```

La implementación interna de un deque es más compleja que la de un vector, y por esta razón, un deque es más lento que un vector. Aun así, tanto agregar como eliminar elementos toma $O(1)$ de tiempo en promedio en ambos extremos.

Pila

Una **pila** es una estructura de datos que ofrece dos operaciones de tiempo $O(1)$: agregar un elemento en la cima, y eliminar un elemento de la cima. Solo es posible acceder al elemento de la cima de una pila.

El siguiente código muestra cómo puede usarse una pila:

```
stack<int> s;
s.push(3);
s.push(2);
s.push(5);
cout << s.top(); // 5
s.pop();
```

```
cout << s.top(); // 2
```

Cola

Una **cola** también ofrece dos operaciones de tiempo $O(1)$: agregar un elemento al final de la cola, y eliminar el primer elemento de la cola. Solo es posible acceder al primer y al último elemento de una cola.

El siguiente código muestra cómo puede usarse una cola:

```
queue<int> q;  
q.push(3);  
q.push(2);  
q.push(5);  
cout << q.front(); // 3  
q.pop();  
cout << q.front(); // 2
```

Cola de prioridad

Una **cola de prioridad** mantiene un conjunto de elementos. Las operaciones admitidas son la inserción y, según el tipo de la cola, la recuperación y eliminación del elemento mínimo o máximo. La inserción y la eliminación toman $O(\log n)$ de tiempo, y la recuperación toma $O(1)$ de tiempo.

Aunque un conjunto ordenado admite eficientemente todas las operaciones de una cola de prioridad, el beneficio de usar una cola de prioridad es que tiene factores constantes más pequeños. Una cola de prioridad se implementa normalmente usando una estructura de montículo (heap) que es mucho más simple que un árbol binario balanceado usado en un conjunto ordenado.

Por defecto, los elementos de una cola de prioridad de C++ se ordenan en orden decreciente, y es posible encontrar y eliminar el elemento más grande de la cola. El siguiente código ilustra esto:

```
priority_queue<int> q;  
q.push(3);  
q.push(5);  
q.push(7);  
q.push(2);  
cout << q.top() << "\n"; // 7  
q.pop();  
cout << q.top() << "\n"; // 5  
q.pop();  
q.push(6);  
cout << q.top() << "\n"; // 6  
q.pop();
```

Si queremos crear una cola de prioridad que admita encontrar y eliminar el elemento más pequeño, podemos hacerlo de la siguiente manera:

```
priority_queue<int,vector<int>,greater<int>> q;
```

Estructuras de datos basadas en políticas

El compilador g++ también admite algunas estructuras de datos que no forman parte de la biblioteca estándar de C++. Tales estructuras se llaman estructuras de datos *basadas en políticas* (policy-based). Para usar estas estructuras, deben agregarse las siguientes líneas al código:

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
```

Después de esto, podemos definir una estructura de datos `indexed_set` que es como set pero puede indexarse como un arreglo. La definición para valores `int` es la siguiente:

```
typedef tree<int,null_type,less<int>,rb_tree_tag,
            tree_order_statistics_node_update> indexed_set;
```

Ahora podemos crear un conjunto de la siguiente manera:

```
indexed_set s;
s.insert(2);
s.insert(3);
s.insert(7);
s.insert(9);
```

La particularidad de este conjunto es que tenemos acceso a los índices que los elementos tendrían en un arreglo ordenado. La función `find_by_order` devuelve un iterador al elemento en una posición dada:

```
auto x = s.find_by_order(2);
cout << *x << "\n"; // 7
```

Y la función `order_of_key` devuelve la posición de un elemento dado:

```
cout << s.order_of_key(7) << "\n"; // 2
```

Si el elemento no aparece en el conjunto, obtenemos la posición que el elemento tendría en el conjunto:

```
cout << s.order_of_key(6) << "\n"; // 2
cout << s.order_of_key(8) << "\n"; // 3
```

Ambas funciones funcionan en tiempo logarítmico.

4.6. Comparación con el ordenamiento

A menudo es posible resolver un problema usando estructuras de datos o usando ordenamiento. A veces hay diferencias notables en la eficiencia real de estos enfoques, que pueden estar ocultas en sus complejidades temporales.

Consideremos un problema donde se nos dan dos listas A y B que ambas contienen n elementos. Nuestra tarea es calcular el número de elementos que pertenecen a ambas listas. Por ejemplo, para las listas

$$A = [5, 2, 8, 9] \quad \text{y} \quad B = [3, 2, 9, 5],$$

la respuesta es 3 porque los números 2, 5 y 9 pertenecen a ambas listas.

Una solución directa al problema es recorrer todos los pares de elementos en tiempo $O(n^2)$, pero a continuación nos centraremos en algoritmos más eficientes.

Algoritmo 1

Construimos un conjunto de los elementos que aparecen en A , y después de esto, recorremos los elementos de B y comprobamos para cada elemento si también pertenece a A . Esto es eficiente porque los elementos de A están en un conjunto. Usando la estructura `set`, la complejidad temporal del algoritmo es $O(n \log n)$.

Algoritmo 2

No es necesario mantener un conjunto ordenado, así que en lugar de la estructura `set` también podemos usar la estructura `unordered_set`. Esta es una forma fácil de hacer el algoritmo más eficiente, porque solo tenemos que cambiar la estructura de datos subyacente. La complejidad temporal del nuevo algoritmo es $O(n)$.

Algoritmo 3

En lugar de estructuras de datos, podemos usar ordenamiento. Primero, ordenamos ambas listas A y B . Después de esto, recorremos ambas listas al mismo tiempo y encontramos los elementos comunes. La complejidad temporal del ordenamiento es $O(n \log n)$, y el resto del algoritmo funciona en tiempo $O(n)$, así que la complejidad temporal total es $O(n \log n)$.

Comparación de eficiencia

La siguiente tabla muestra qué tan eficientes son los algoritmos anteriores cuando n varía y los elementos de las listas son enteros aleatorios entre $1 \dots 10^9$:

n	Algoritmo 1	Algoritmo 2	Algoritmo 3
10^6	1.5 s	0.3 s	0.2 s
$2 \cdot 10^6$	3.7 s	0.8 s	0.3 s
$3 \cdot 10^6$	5.7 s	1.3 s	0.5 s
$4 \cdot 10^6$	7.7 s	1.7 s	0.7 s
$5 \cdot 10^6$	10.0 s	2.3 s	0.9 s

Los Algoritmos 1 y 2 son iguales excepto que usan estructuras de conjunto distintas. En este problema, esta elección tiene un efecto importante en el tiempo de ejecución, porque el Algoritmo 2 es 4–5 veces más rápido que el Algoritmo 1.

Sin embargo, el algoritmo más eficiente es el Algoritmo 3 que usa ordenamiento. Solo usa la mitad del tiempo comparado con el Algoritmo 2. Curiosamente, la complejidad temporal tanto del Algoritmo 1 como del Algoritmo 3 es $O(n \log n)$, pero a pesar de esto, el Algoritmo 3 es diez veces más rápido. Esto puede explicarse por el hecho de que el ordenamiento es un procedimiento simple y se realiza solo una vez al principio del Algoritmo 3, y el resto del algoritmo funciona en tiempo lineal. Por otro lado, el Algoritmo 1 mantiene un complejo árbol binario balanceado durante todo el algoritmo.

Capítulo 5

Búsqueda completa

La **búsqueda completa** es un método general que puede usarse para resolver casi cualquier problema de algoritmos. La idea es generar todas las soluciones posibles al problema por fuerza bruta, y luego seleccionar la mejor solución o contar el número de soluciones, según el problema.

La búsqueda completa es una buena técnica si hay tiempo suficiente para recorrer todas las soluciones, porque la búsqueda suele ser fácil de implementar y siempre da la respuesta correcta. Si la búsqueda completa es demasiado lenta, pueden necesitarse otras técnicas, como los algoritmos voraces o la programación dinámica.

5.1. Generar subconjuntos

Primero consideramos el problema de generar todos los subconjuntos de un conjunto de n elementos. Por ejemplo, los subconjuntos de $\{0, 1, 2\}$ son $\emptyset$, $\{0\}$, $\{1\}$, $\{2\}$, $\{0, 1\}$, $\{0, 2\}$, $\{1, 2\}$ y $\{0, 1, 2\}$. Hay dos métodos comunes para generar subconjuntos: podemos realizar una búsqueda recursiva o aprovechar la representación en bits de los enteros.

Método 1

Una forma elegante de recorrer todos los subconjuntos de un conjunto es usar recursión. La siguiente función `search` genera los subconjuntos del conjunto $\{0, 1, \dots, n-1\}$. La función mantiene un vector `subset` que contendrá los elementos de cada subconjunto. La búsqueda comienza cuando la función se llama con el parámetro 0.

```
void search(int k) {  
    if (k == n) {  
        // process subset  
    } else {  
        search(k+1);  
        subset.push_back(k);  
        search(k+1);  
        subset.pop_back();  
    }  
}
```

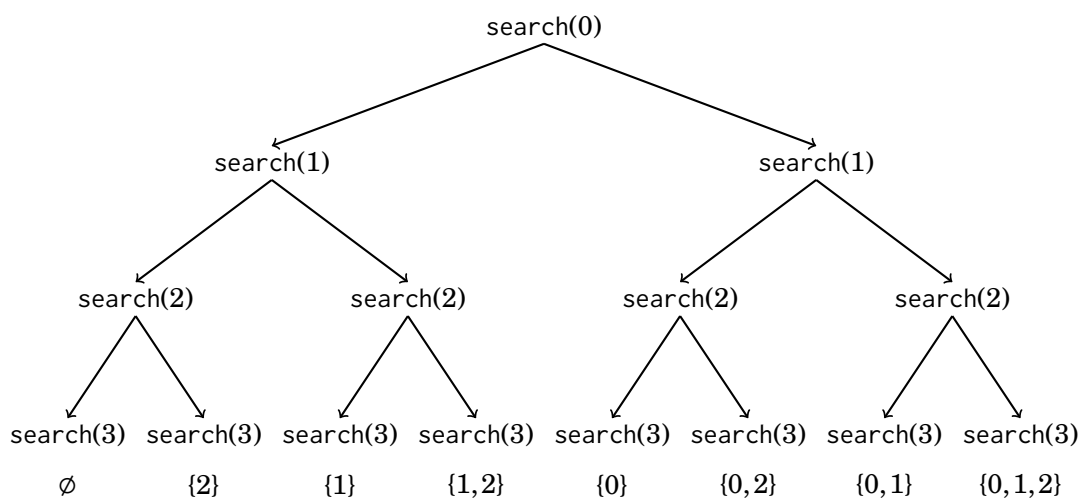
```

    }
}

```

Cuando la función `search` se llama con el parámetro k , decide si incluir el elemento k en el subconjunto o no, y en ambos casos, luego se llama a sí misma con el parámetro $k + 1$. Sin embargo, si $k = n$, la función se da cuenta de que todos los elementos han sido procesados y se ha generado un subconjunto.

El siguiente árbol ilustra las llamadas a la función cuando $n = 3$. Siempre podemos elegir la rama izquierda (k no se incluye en el subconjunto) o la rama derecha (k se incluye en el subconjunto).



Método 2

Otra forma de generar subconjuntos se basa en la representación en bits de los enteros. Cada subconjunto de un conjunto de n elementos puede representarse como una secuencia de n bits, que corresponde a un entero entre $0 \dots 2^n - 1$. Los unos en la secuencia de bits indican qué elementos se incluyen en el subconjunto.

La convención usual es que el último bit corresponde al elemento 0, el penúltimo bit corresponde al elemento 1, y así sucesivamente. Por ejemplo, la representación en bits de 25 es 11001, que corresponde al subconjunto $\{0, 3, 4\}$.

El siguiente código recorre los subconjuntos de un conjunto de n elementos

```

for (int b = 0; b < (1<<n); b++) {
    // process subset
}

```

El siguiente código muestra cómo podemos encontrar los elementos de un subconjunto que corresponde a una secuencia de bits. Al procesar cada subconjunto, el código construye un vector que contiene los elementos del subconjunto.

```

for (int b = 0; b < (1<<n); b++) {
    vector<int> subset;
    for (int i = 0; i < n; i++) {
        if (b & (1<<i)) subset.push_back(i);
    }
}

```

```
}  
}
```

5.2. Generar permutaciones

A continuación consideramos el problema de generar todas las permutaciones de un conjunto de n elementos. Por ejemplo, las permutaciones de $\{0, 1, 2\}$ son $(0, 1, 2)$, $(0, 2, 1)$, $(1, 0, 2)$, $(1, 2, 0)$, $(2, 0, 1)$ y $(2, 1, 0)$. De nuevo, hay dos enfoques: podemos usar recursión o recorrer las permutaciones de forma iterativa.

Método 1

Como los subconjuntos, las permutaciones pueden generarse usando recursión. La siguiente función `search` recorre las permutaciones del conjunto $\{0, 1, \dots, n-1\}$. La función construye un vector `permutation` que contiene la permutación, y la búsqueda comienza cuando la función se llama sin parámetros.

```
void search() {  
    if (permutation.size() == n) {  
        // process permutation  
    } else {  
        for (int i = 0; i < n; i++) {  
            if (chosen[i]) continue;  
            chosen[i] = true;  
            permutation.push_back(i);  
            search();  
            chosen[i] = false;  
            permutation.pop_back();  
        }  
    }  
}
```

Cada llamada a la función agrega un nuevo elemento a `permutation`. El arreglo `chosen` indica qué elementos ya están incluidos en la permutación. Si el tamaño de `permutation` es igual al tamaño del conjunto, se ha generado una permutación.

Método 2

Otro método para generar permutaciones es comenzar con la permutación $\{0, 1, \dots, n-1\}$ y usar repetidamente una función que construye la siguiente permutación en orden creciente. La biblioteca estándar de C++ contiene la función `next_permutation` que puede usarse para esto:

```
vector<int> permutation;  
for (int i = 0; i < n; i++) {
```

```

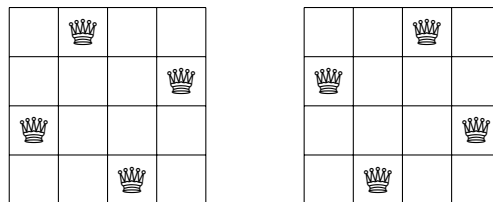
    permutation.push_back(i);
}
do {
    // process permutation
} while (next_permutation(permutation.begin(), permutation.end()));

```

5.3. Backtracking

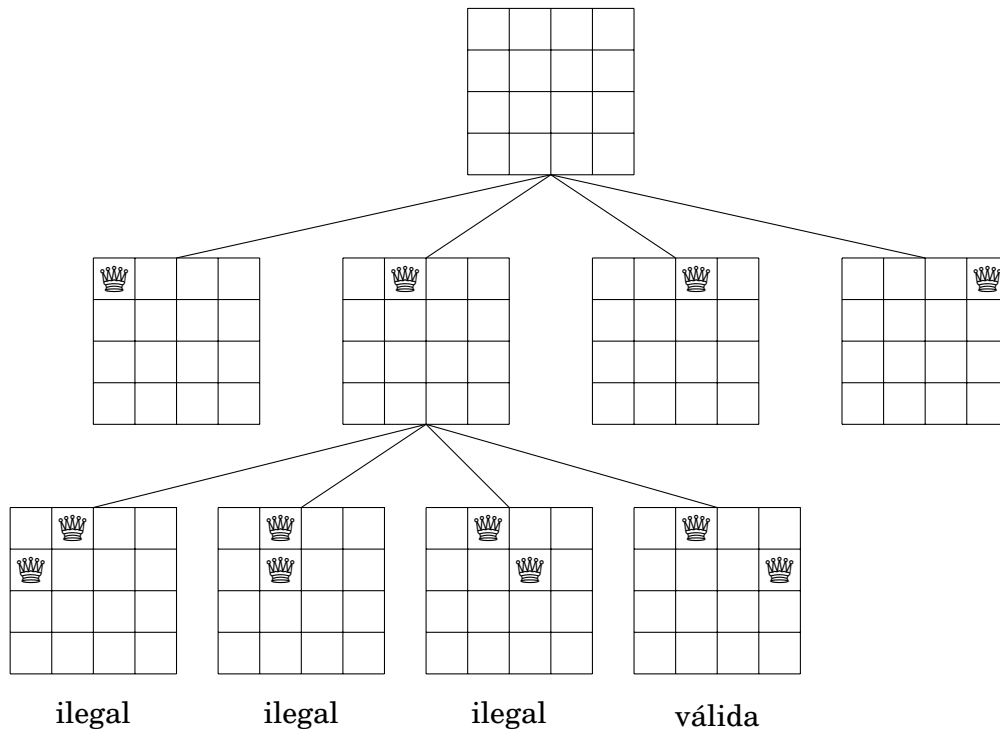
Un algoritmo de **backtracking** (vuelta atrás) comienza con una solución vacía y extiende la solución paso a paso. La búsqueda recorre recursivamente todas las formas distintas en que puede construirse una solución.

Como ejemplo, considera el problema de calcular el número de formas en que n reinas pueden colocarse en un tablero de ajedrez de $n \times n$ de modo que ninguna reina ataque a otra. Por ejemplo, cuando $n = 4$, hay dos soluciones posibles:



El problema puede resolverse usando backtracking colocando reinas en el tablero fila por fila. Más precisamente, se colocará exactamente una reina en cada fila de modo que ninguna reina ataque a ninguna de las reinas colocadas antes. Se ha encontrado una solución cuando todas las n reinas han sido colocadas en el tablero.

Por ejemplo, cuando $n = 4$, algunas soluciones parciales generadas por el algoritmo de backtracking son las siguientes:



En el nivel inferior, las tres primeras configuraciones son ilegales, porque las reinas se atacan entre sí. Sin embargo, la cuarta configuración es válida y puede extenderse a una solución completa colocando dos reinas más en el tablero. Solo hay una forma de colocar las dos reinas restantes.

El algoritmo puede implementarse de la siguiente manera:

```
void search(int y) {
    if (y == n) {
        count++;
        return;
    }
    for (int x = 0; x < n; x++) {
        if (column[x] || diag1[x+y] || diag2[x-y+n-1]) continue;
        column[x] = diag1[x+y] = diag2[x-y+n-1] = 1;
        search(y+1);
        column[x] = diag1[x+y] = diag2[x-y+n-1] = 0;
    }
}
```

La búsqueda comienza llamando a `search(0)`. El tamaño del tablero es $n \times n$, y el código calcula el número de soluciones en `count`.

El código supone que las filas y columnas del tablero se numeran de 0 a $n - 1$. Cuando la función `search` se llama con el parámetro y , coloca una reina en la fila y y luego se llama a sí misma con el parámetro $y + 1$. Luego, si $y = n$, se ha encontrado una solución y la variable `count` se incrementa en uno.

El arreglo `column` lleva registro de las columnas que contienen una reina, y los arreglos `diag1` y `diag2` llevan registro de las diagonales. No se permite agregar

otra reina a una columna o diagonal que ya contiene una reina. Por ejemplo, las columnas y diagonales del tablero de 4×4 se numeran de la siguiente manera:

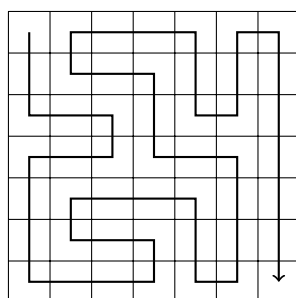
0	1	2	3	0	1	2	3	3	4	5	6
0	1	2	3	1	2	3	4	2	3	4	5
0	1	2	3	2	3	4	5	1	2	3	4
0	1	2	3	3	4	5	6	0	1	2	3
column				diag1				diag2			

Sea $q(n)$ el número de formas de colocar n reinas en un tablero de ajedrez de $n \times n$. El algoritmo de backtracking anterior nos dice que, por ejemplo, $q(8) = 92$. Cuando n aumenta, la búsqueda se vuelve rápidamente lenta, porque el número de soluciones aumenta exponencialmente. Por ejemplo, calcular $q(16) = 14772512$ usando el algoritmo anterior ya toma aproximadamente un minuto en una computadora moderna¹.

5.4. Podar la búsqueda

A menudo podemos optimizar el backtracking podando el árbol de búsqueda. La idea es agregar "inteligencia" al algoritmo para que note tan pronto como sea posible si una solución parcial no puede extenderse a una solución completa. Tales optimizaciones pueden tener un efecto tremendo en la eficiencia de la búsqueda.

Consideremos el problema de calcular el número de caminos en una cuadrícula de $n \times n$ desde la esquina superior izquierda hasta la esquina inferior derecha tales que el camino visite cada casilla exactamente una vez. Por ejemplo, en una cuadrícula de 7×7 , hay 111712 de tales caminos. Uno de los caminos es el siguiente:



Nos centramos en el caso de 7×7 , porque su nivel de dificultad es apropiado para nuestras necesidades. Comenzamos con un algoritmo de backtracking directo, y luego lo optimizamos paso a paso usando observaciones de cómo puede podarse la búsqueda. Después de cada optimización, medimos el tiempo de ejecución del algoritmo y el número de llamadas recursivas, para ver claramente el efecto de cada optimización en la eficiencia de la búsqueda.

¹No se conoce ninguna forma de calcular eficientemente valores más grandes de $q(n)$. El récord actual es $q(27) = 234907967154122528$, calculado en 2016 [55].

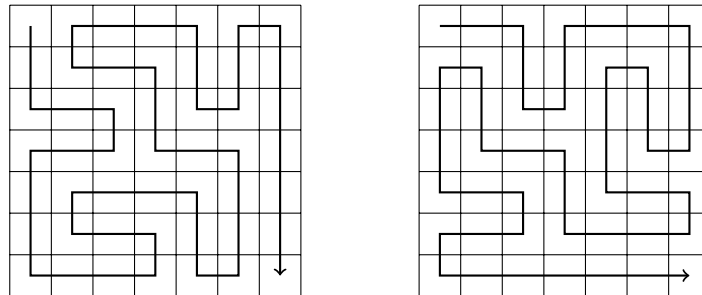
Algoritmo básico

La primera versión del algoritmo no contiene ninguna optimización. Simplemente usamos backtracking para generar todos los caminos posibles desde la esquina superior izquierda hasta la esquina inferior derecha y contamos el número de tales caminos.

- tiempo de ejecución: 483 segundos
- número de llamadas recursivas: 76 mil millones

Optimización 1

En cualquier solución, primero nos movemos un paso hacia abajo o hacia la derecha. Siempre hay dos caminos que son simétricos respecto a la diagonal de la cuadrícula después del primer paso. Por ejemplo, los siguientes caminos son simétricos:

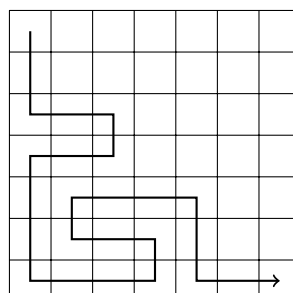


Por lo tanto, podemos decidir que siempre nos movemos primero un paso hacia abajo (o hacia la derecha), y finalmente multiplicar el número de soluciones por dos.

- tiempo de ejecución: 244 segundos
- número de llamadas recursivas: 38 mil millones

Optimización 2

Si el camino llega a la casilla inferior derecha antes de haber visitado todas las demás casillas de la cuadrícula, está claro que no será posible completar la solución. Un ejemplo de esto es el siguiente camino:

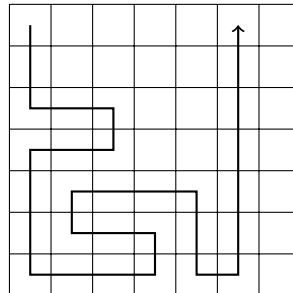


Usando esta observación, podemos terminar la búsqueda de inmediato si llegamos a la casilla inferior derecha demasiado pronto.

- tiempo de ejecución: 119 segundos
- número de llamadas recursivas: 20 mil millones

Optimización 3

Si el camino toca una pared y puede girar a la izquierda o a la derecha, la cuadrícula se divide en dos partes que contienen casillas no visitadas. Por ejemplo, en la siguiente situación, el camino puede girar a la izquierda o a la derecha:

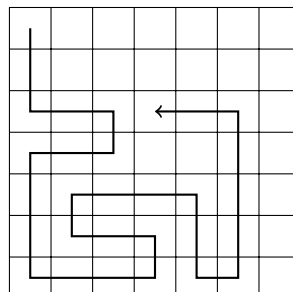


En este caso, ya no podemos visitar todas las casillas, así que podemos terminar la búsqueda. Esta optimización es muy útil:

- tiempo de ejecución: 1.8 segundos
- número de llamadas recursivas: 221 millones

Optimización 4

La idea de la Optimización 3 puede generalizarse: si el camino no puede continuar hacia adelante pero puede girar a la izquierda o a la derecha, la cuadrícula se divide en dos partes que ambas contienen casillas no visitadas. Por ejemplo, considera el siguiente camino:



Está claro que ya no podemos visitar todas las casillas, así que podemos terminar la búsqueda. Después de esta optimización, la búsqueda es muy eficiente:

- tiempo de ejecución: 0.6 segundos

- número de llamadas recursivas: 69 millones

Ahora es un buen momento para dejar de optimizar el algoritmo y ver lo que hemos logrado. El tiempo de ejecución del algoritmo original era de 483 segundos, y ahora, después de las optimizaciones, el tiempo de ejecución es de solo 0.6 segundos. Así, el algoritmo se volvió casi 1000 veces más rápido después de las optimizaciones.

Este es un fenómeno habitual en el backtracking, porque el árbol de búsqueda suele ser grande y hasta observaciones simples pueden podar la búsqueda de forma efectiva. Especialmente útiles son las optimizaciones que ocurren durante los primeros pasos del algoritmo, es decir, en la parte superior del árbol de búsqueda.

5.5. Encuentro en el medio

El **encuentro en el medio** (meet in the middle) es una técnica donde el espacio de búsqueda se divide en dos partes de tamaño aproximadamente igual. Se realiza una búsqueda separada para cada una de las partes, y finalmente se combinan los resultados de las búsquedas.

La técnica puede usarse si hay una forma eficiente de combinar los resultados de las búsquedas. En tal situación, las dos búsquedas pueden requerir menos tiempo que una sola búsqueda grande. Normalmente, podemos convertir un factor de 2^n en un factor de $2^{n/2}$ usando la técnica del encuentro en el medio.

Como ejemplo, considera un problema donde se nos da una lista de n números y un número x , y queremos averiguar si es posible elegir algunos números de la lista de modo que su suma sea x . Por ejemplo, dada la lista $[2, 4, 5, 9]$ y $x = 15$, podemos elegir los números $[2, 4, 9]$ para obtener $2 + 4 + 9 = 15$. Sin embargo, si $x = 10$ para la misma lista, no es posible formar la suma.

Un algoritmo simple para el problema es recorrer todos los subconjuntos de los elementos y comprobar si la suma de alguno de los subconjuntos es x . El tiempo de ejecución de tal algoritmo es $O(2^n)$, porque hay 2^n subconjuntos. Sin embargo, usando la técnica del encuentro en el medio, podemos lograr un algoritmo más eficiente de tiempo $O(2^{n/2})^2$. Nota que $O(2^n)$ y $O(2^{n/2})$ son complejidades distintas porque $2^{n/2}$ es igual a $\sqrt{2^n}$.

La idea es dividir la lista en dos listas A y B tales que ambas listas contengan aproximadamente la mitad de los números. La primera búsqueda genera todos los subconjuntos de A y almacena sus sumas en una lista S_A . Correspondientemente, la segunda búsqueda crea una lista S_B a partir de B . Después de esto, basta con comprobar si es posible elegir un elemento de S_A y otro elemento de S_B tales que su suma sea x . Esto es posible exactamente cuando hay una forma de formar la suma x usando los números de la lista original.

Por ejemplo, supongamos que la lista es $[2, 4, 5, 9]$ y $x = 15$. Primero, dividimos la lista en $A = [2, 4]$ y $B = [5, 9]$. Después de esto, creamos las listas $S_A = [0, 2, 4, 6]$

²Esta idea fue introducida en 1974 por E. Horowitz y S. Sahni [39].

y $S_B = [0, 5, 9, 14]$. En este caso, la suma $x = 15$ es posible de formar, porque S_A contiene la suma 6, S_B contiene la suma 9, y $6 + 9 = 15$. Esto corresponde a la solución $[2, 4, 9]$.

Podemos implementar el algoritmo de modo que su complejidad temporal sea $O(2^{n/2})$. Primero, generamos listas *ordenadas* S_A y S_B , lo que puede hacerse en tiempo $O(2^{n/2})$ usando una técnica similar a la mezcla. Después de esto, como las listas están ordenadas, podemos comprobar en tiempo $O(2^{n/2})$ si la suma x puede crearse a partir de S_A y S_B .

Capítulo 6

Algoritmos voraces

Un **algoritmo voraz** construye una solución al problema tomando siempre la elección que parece la mejor en ese momento. Un algoritmo voraz nunca deshace sus elecciones, sino que construye directamente la solución final. Por esta razón, los algoritmos voraces suelen ser muy eficientes.

La dificultad al diseñar algoritmos voraces es encontrar una estrategia voraz que siempre produzca una solución óptima al problema. Las elecciones localmente óptimas en un algoritmo voraz deberían ser también globalmente óptimas. A menudo es difícil argumentar que un algoritmo voraz funciona.

6.1. Problema de las monedas

Como primer ejemplo, consideramos un problema donde se nos da un conjunto de monedas y nuestra tarea es formar una cantidad de dinero n usando las monedas. Los valores de las monedas son $\text{coins} = \{c_1, c_2, \dots, c_k\}$, y cada moneda puede usarse tantas veces como queramos. ¿Cuál es el número mínimo de monedas necesarias?

Por ejemplo, si las monedas son las monedas de euro (en céntimos)

$$\{1, 2, 5, 10, 20, 50, 100, 200\}$$

y $n = 520$, necesitamos al menos cuatro monedas. La solución óptima es seleccionar las monedas $200 + 200 + 100 + 20$ cuya suma es 520.

Algoritmo voraz

Un algoritmo voraz simple para el problema siempre selecciona la moneda más grande posible, hasta que se haya construido la cantidad de dinero requerida. Este algoritmo funciona en el caso del ejemplo, porque primero seleccionamos dos monedas de 200 céntimos, luego una moneda de 100 céntimos y finalmente una moneda de 20 céntimos. Pero ¿este algoritmo siempre funciona?

Resulta que si las monedas son las monedas de euro, el algoritmo voraz *siempre* funciona, es decir, siempre produce una solución con el menor número posible de monedas. La corrección del algoritmo puede demostrarse de la siguiente manera:

Primero, cada moneda 1, 5, 10, 50 y 100 aparece a lo sumo una vez en una solución óptima, porque si la solución contuviera dos de tales monedas, podríamos reemplazarlas por una moneda y obtener una solución mejor. Por ejemplo, si la solución contuviera las monedas 5 + 5, podríamos reemplazarlas por la moneda 10.

De la misma forma, las monedas 2 y 20 aparecen a lo sumo dos veces en una solución óptima, porque podríamos reemplazar las monedas 2 + 2 + 2 por las monedas 5 + 1 y las monedas 20 + 20 + 20 por las monedas 50 + 10. Además, una solución óptima no puede contener las monedas 2 + 2 + 1 o 20 + 20 + 10, porque podríamos reemplazarlas por las monedas 5 y 50.

Usando estas observaciones, podemos mostrar para cada moneda x que no es posible construir de forma óptima una suma x o cualquier suma mayor usando solo monedas que sean menores que x . Por ejemplo, si $x = 100$, la mayor suma óptima usando las monedas más pequeñas es $50 + 20 + 20 + 5 + 2 + 2 = 99$. Así, el algoritmo voraz que siempre selecciona la moneda más grande produce la solución óptima.

Este ejemplo muestra que puede ser difícil argumentar que un algoritmo voraz funciona, aunque el algoritmo en sí sea simple.

Caso general

En el caso general, el conjunto de monedas puede contener cualquier moneda y el algoritmo voraz *no* necesariamente produce una solución óptima.

Podemos demostrar que un algoritmo voraz no funciona mostrando un contraejemplo donde el algoritmo da una respuesta incorrecta. En este problema podemos encontrar fácilmente un contraejemplo: si las monedas son $\{1, 3, 4\}$ y la suma objetivo es 6, el algoritmo voraz produce la solución $4 + 1 + 1$ mientras que la solución óptima es $3 + 3$.

No se sabe si el problema general de las monedas puede resolverse usando algún algoritmo voraz¹. Sin embargo, como veremos en el Capítulo 7, en algunos casos, el problema general puede resolverse de forma eficiente usando un algoritmo de programación dinámica que siempre da la respuesta correcta.

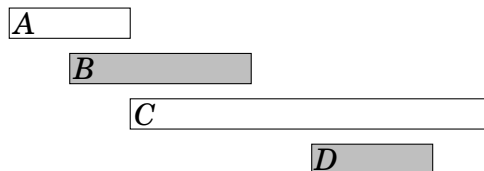
6.2. Planificación

Muchos problemas de planificación pueden resolverse usando algoritmos voraces. Un problema clásico es el siguiente: Dados n eventos con sus tiempos de inicio y fin, encontrar una planificación que incluya tantos eventos como sea posible. No es posible seleccionar un evento parcialmente. Por ejemplo, considera los siguientes eventos:

¹Sin embargo, es posible *comprobar* en tiempo polinómico si el algoritmo voraz presentado en este capítulo funciona para un conjunto dado de monedas [53].

evento	tiempo de inicio	tiempo de fin
<i>A</i>	1	3
<i>B</i>	2	5
<i>C</i>	3	9
<i>D</i>	6	8

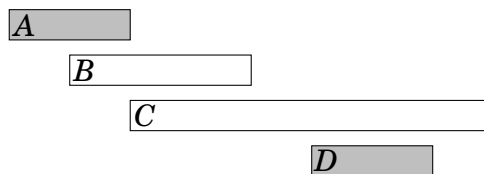
En este caso el número máximo de eventos es dos. Por ejemplo, podemos seleccionar los eventos *B* y *D* de la siguiente manera:



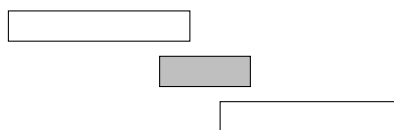
Es posible inventar varios algoritmos voraces para el problema, pero ¿cuál de ellos funciona en todos los casos?

Algoritmo 1

La primera idea es seleccionar los eventos tan *cortos* como sea posible. En el caso del ejemplo este algoritmo selecciona los siguientes eventos:



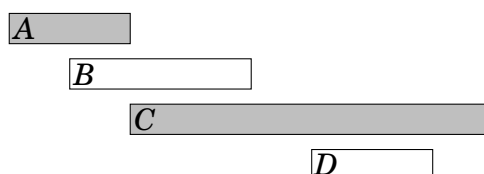
Sin embargo, seleccionar eventos cortos no siempre es una estrategia correcta. Por ejemplo, el algoritmo falla en el siguiente caso:



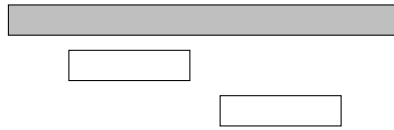
Si seleccionamos el evento corto, solo podemos seleccionar un evento. Sin embargo, sería posible seleccionar ambos eventos largos.

Algoritmo 2

Otra idea es seleccionar siempre el siguiente evento posible que *comience* lo más *temprano* posible. Este algoritmo selecciona los siguientes eventos:



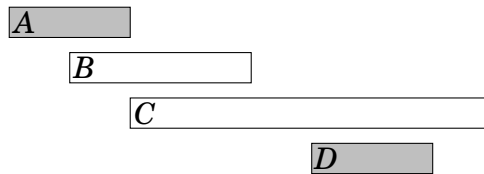
Sin embargo, podemos encontrar un contraejemplo también para este algoritmo. Por ejemplo, en el siguiente caso, el algoritmo solo selecciona un evento:



Si seleccionamos el primer evento, no es posible seleccionar ningún otro evento. Sin embargo, sería posible seleccionar los otros dos eventos.

Algoritmo 3

La tercera idea es seleccionar siempre el siguiente evento posible que *termine* lo más *temprano* posible. Este algoritmo selecciona los siguientes eventos:



Resulta que este algoritmo *siempre* produce una solución óptima. La razón de esto es que siempre es una elección óptima seleccionar primero un evento que termine lo más temprano posible. Después de esto, es una elección óptima seleccionar el siguiente evento usando la misma estrategia, etc., hasta que no podamos seleccionar más eventos.

Una forma de argumentar que el algoritmo funciona es considerar qué ocurre si primero seleccionamos un evento que termina más tarde que el evento que termina lo más temprano posible. Ahora, tendremos a lo sumo un número igual de opciones de cómo podemos seleccionar el siguiente evento. Por lo tanto, seleccionar un evento que termina más tarde nunca puede dar una solución mejor, y el algoritmo voraz es correcto.

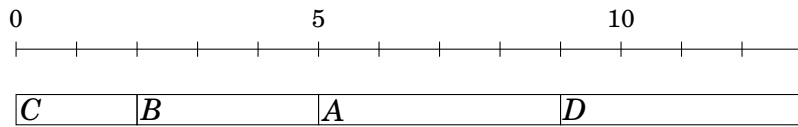
6.3. Tareas y fechas límite

Consideremos ahora un problema donde se nos dan n tareas con duraciones y fechas límite y nuestra tarea es elegir un orden para realizar las tareas. Por cada tarea, ganamos $d - x$ puntos donde d es la fecha límite de la tarea y x es el momento en que terminamos la tarea. ¿Cuál es la mayor puntuación total posible que podemos obtener?

Por ejemplo, supongamos que las tareas son las siguientes:

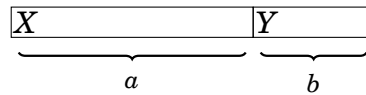
tarea	duración	fecha límite
<i>A</i>	4	2
<i>B</i>	3	5
<i>C</i>	2	7
<i>D</i>	4	5

En este caso, una planificación óptima de las tareas es la siguiente:

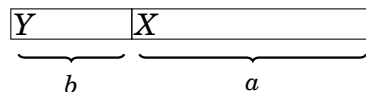


En esta solución, C da 5 puntos, B da 0 puntos, A da -7 puntos y D da -8 puntos, así que la puntuación total es -10 .

Sorprendentemente, la solución óptima al problema no depende para nada de las fechas límite, sino que una estrategia voraz correcta es simplemente realizar las tareas *ordenadas por sus duraciones* en orden creciente. La razón de esto es que si alguna vez realizamos dos tareas una tras otra de modo que la primera tarea tome más tiempo que la segunda tarea, podemos obtener una solución mejor si intercambiamos las tareas. Por ejemplo, considera la siguiente planificación:



Aquí $a > b$, así que deberíamos intercambiar las tareas:



Ahora X da b puntos menos e Y da a puntos más, así que la puntuación total aumenta en $a - b > 0$. En una solución óptima, para cualesquiera dos tareas consecutivas, debe cumplirse que la tarea más corta va antes que la tarea más larga. Así, las tareas deben realizarse ordenadas por sus duraciones.

6.4. Minimizar sumas

A continuación consideramos un problema donde se nos dan n números $a_1, a_2, \dots, a_n$ y nuestra tarea es encontrar un valor x que minimice la suma

$$|a_1 - x|^c + |a_2 - x|^c + \dots + |a_n - x|^c.$$

Nos centramos en los casos $c = 1$ y $c = 2$.

Caso $c = 1$

En este caso, debemos minimizar la suma

$$|a_1 - x| + |a_2 - x| + \dots + |a_n - x|.$$

Por ejemplo, si los números son $[1, 2, 9, 2, 6]$, la mejor solución es seleccionar $x = 2$ que produce la suma

$$|1 - 2| + |2 - 2| + |9 - 2| + |2 - 2| + |6 - 2| = 12.$$

En el caso general, la mejor elección para x es la *mediana* de los números, es decir, el número del medio después de ordenar. Por ejemplo, la lista $[1, 2, 9, 2, 6]$ se convierte en $[1, 2, 2, 6, 9]$ después de ordenar, así que la mediana es 2.

La mediana es una elección óptima, porque si x es menor que la mediana, la suma se hace más pequeña al aumentar x , y si x es mayor que la mediana, la suma se hace más pequeña al disminuir x . Por lo tanto, la solución óptima es que x sea la mediana. Si n es par y hay dos medianas, ambas medianas y todos los valores entre ellas son elecciones óptimas.

Caso $c = 2$

En este caso, debemos minimizar la suma

$$(a_1 - x)^2 + (a_2 - x)^2 + \cdots + (a_n - x)^2.$$

Por ejemplo, si los números son $[1, 2, 9, 2, 6]$, la mejor solución es seleccionar $x = 4$ que produce la suma

$$(1 - 4)^2 + (2 - 4)^2 + (9 - 4)^2 + (2 - 4)^2 + (6 - 4)^2 = 46.$$

En el caso general, la mejor elección para x es el *promedio* de los números. En el ejemplo el promedio es $(1 + 2 + 9 + 2 + 6)/5 = 4$. Este resultado puede deducirse presentando la suma de la siguiente manera:

$$nx^2 - 2x(a_1 + a_2 + \cdots + a_n) + (a_1^2 + a_2^2 + \cdots + a_n^2)$$

La última parte no depende de x , así que podemos ignorarla. Las partes restantes forman una función $nx^2 - 2xs$ donde $s = a_1 + a_2 + \cdots + a_n$. Esta es una parábola que abre hacia arriba con raíces $x = 0$ y $x = 2s/n$, y el valor mínimo es el promedio de las raíces $x = s/n$, es decir, el promedio de los números $a_1, a_2, \dots, a_n$.

6.5. Compresión de datos

Un **código binario** asigna a cada carácter de una cadena una **palabra de código** que consta de bits. Podemos *comprimir* la cadena usando el código binario reemplazando cada carácter por la palabra de código correspondiente. Por ejemplo, el siguiente código binario asigna palabras de código a los caracteres A–D:

carácter	palabra de código
A	00
B	01
C	10
D	11

Este es un código de **longitud constante** lo que significa que la longitud de cada palabra de código es la misma. Por ejemplo, podemos comprimir la cadena AABACDACA de la siguiente manera:

000001001011001000

Usando este código, la longitud de la cadena comprimida es de 18 bits. Sin embargo, podemos comprimir la cadena mejor si usamos un código de **longitud variable** donde las palabras de código pueden tener longitudes distintas. Entonces podemos dar palabras de código cortas para los caracteres que aparecen a menudo y palabras de código largas para los caracteres que aparecen rara vez. Resulta que un código **óptimo** para la cadena anterior es el siguiente:

carácter	palabra de código
A	0
B	110
C	10
D	111

Un código óptimo produce una cadena comprimida que es lo más corta posible. En este caso, la cadena comprimida usando el código óptimo es

001100101110100,

así que solo se necesitan 15 bits en lugar de 18 bits. Así, gracias a un código mejor fue posible ahorrar 3 bits en la cadena comprimida.

Requerimos que ninguna palabra de código sea un prefijo de otra palabra de código. Por ejemplo, no se permite que un código contenga ambas palabras de código 10 y 1011. La razón de esto es que queremos poder generar la cadena original a partir de la cadena comprimida. Si una palabra de código pudiera ser un prefijo de otra palabra de código, esto no siempre sería posible. Por ejemplo, el siguiente código *no* es válido:

carácter	palabra de código
A	10
B	11
C	1011
D	111

Usando este código, no sería posible saber si la cadena comprimida 1011 corresponde a la cadena AB o a la cadena C.

Codificación de Huffman

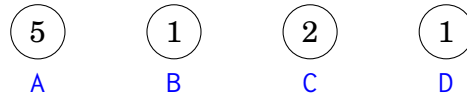
La **codificación de Huffman**² es un algoritmo voraz que construye un código óptimo para comprimir una cadena dada. El algoritmo construye un árbol binario basado en las frecuencias de los caracteres de la cadena, y la palabra de código de cada carácter puede leerse siguiendo un camino desde la raíz hasta el nodo correspondiente. Un movimiento hacia la izquierda corresponde al bit 0, y un movimiento hacia la derecha corresponde al bit 1.

Inicialmente, cada carácter de la cadena está representado por un nodo cuyo peso es el número de veces que el carácter aparece en la cadena. Luego en cada

²D. A. Huffman descubrió este método al resolver una tarea de un curso universitario y publicó el algoritmo en 1952 [40].

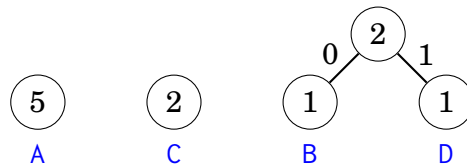
paso se combinan dos nodos con pesos mínimos creando un nuevo nodo cuyo peso es la suma de los pesos de los nodos originales. El proceso continúa hasta que todos los nodos han sido combinados.

A continuación veremos cómo la codificación de Huffman crea el código óptimo para la cadena AABACDACA. Inicialmente, hay cuatro nodos que corresponden a los caracteres de la cadena:

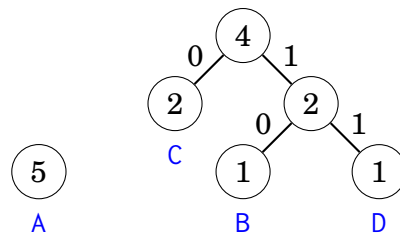


El nodo que representa el carácter A tiene peso 5 porque el carácter A aparece 5 veces en la cadena. Los otros pesos se han calculado de la misma manera.

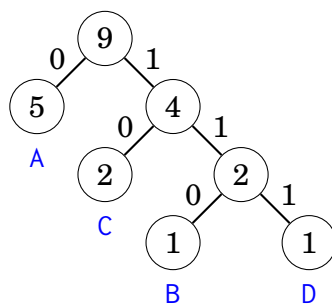
El primer paso es combinar los nodos que corresponden a los caracteres B y D, ambos con peso 1. El resultado es:



Después de esto, se combinan los nodos con peso 2:



Finalmente, se combinan los dos nodos restantes:



Ahora todos los nodos están en el árbol, así que el código está listo. Las siguientes palabras de código pueden leerse del árbol:

carácter	palabra de código
A	0
B	110
C	10
D	111

Capítulo 7

Programación dinámica

La **programación dinámica** es una técnica que combina la corrección de la búsqueda completa y la eficiencia de los algoritmos voraces. La programación dinámica puede aplicarse si el problema puede dividirse en subproblemas superpuestos que pueden resolverse de forma independiente.

Hay dos usos para la programación dinámica:

- **Encontrar una solución óptima:** Queremos encontrar una solución que sea lo más grande posible o lo más pequeña posible.
- **Contar el número de soluciones:** Queremos calcular el número total de soluciones posibles.

Primero veremos cómo puede usarse la programación dinámica para encontrar una solución óptima, y luego usaremos la misma idea para contar las soluciones.

Entender la programación dinámica es un hito en la carrera de todo programador competitivo. Aunque la idea básica es simple, el reto es cómo aplicar la programación dinámica a distintos problemas. Este capítulo presenta un conjunto de problemas clásicos que son un buen punto de partida.

7.1. Problema de las monedas

Primero nos centramos en un problema que ya hemos visto en el Capítulo 6: Dado un conjunto de valores de monedas $\text{coins} = \{c_1, c_2, \dots, c_k\}$ y una suma de dinero objetivo n , nuestra tarea es formar la suma n usando la menor cantidad de monedas posible.

En el Capítulo 6, resolvimos el problema usando un algoritmo voraz que siempre elige la moneda más grande posible. El algoritmo voraz funciona, por ejemplo, cuando las monedas son las monedas de euro, pero en el caso general el algoritmo voraz no necesariamente produce una solución óptima.

Ahora es momento de resolver el problema de forma eficiente usando programación dinámica, de modo que el algoritmo funcione para cualquier conjunto de monedas. El algoritmo de programación dinámica se basa en una función recursiva que recorre todas las posibilidades de cómo formar la suma, como un algoritmo de fuerza bruta. Sin embargo, el algoritmo de programación dinámica

es eficiente porque usa *memoización* y calcula la respuesta de cada subproblema solo una vez.

Formulación recursiva

La idea en la programación dinámica es formular el problema de forma recursiva de modo que la solución del problema pueda calcularse a partir de soluciones de subproblemas más pequeños. En el problema de las monedas, un problema recursivo natural es el siguiente: ¿cuál es el menor número de monedas requerido para formar una suma x ?

Sea $\text{solve}(x)$ el número mínimo de monedas requerido para una suma x . Los valores de la función dependen de los valores de las monedas. Por ejemplo, si $\text{coins} = \{1, 3, 4\}$, los primeros valores de la función son los siguientes:

$\text{solve}(0)$	$=$	0
$\text{solve}(1)$	$=$	1
$\text{solve}(2)$	$=$	2
$\text{solve}(3)$	$=$	1
$\text{solve}(4)$	$=$	1
$\text{solve}(5)$	$=$	2
$\text{solve}(6)$	$=$	2
$\text{solve}(7)$	$=$	2
$\text{solve}(8)$	$=$	2
$\text{solve}(9)$	$=$	3
$\text{solve}(10)$	$=$	3

Por ejemplo, $\text{solve}(10) = 3$, porque se necesitan al menos 3 monedas para formar la suma 10. La solución óptima es $3 + 3 + 4 = 10$.

La propiedad esencial de solve es que sus valores pueden calcularse recursivamente a partir de sus valores más pequeños. La idea es centrarse en la *primera* moneda que elegimos para la suma. Por ejemplo, en el escenario anterior, la primera moneda puede ser 1, 3 o 4. Si primero elegimos la moneda 1, la tarea restante es formar la suma 9 usando el número mínimo de monedas, que es un subproblema del problema original. Por supuesto, lo mismo aplica a las monedas 3 y 4. Así, podemos usar la siguiente fórmula recursiva para calcular el número mínimo de monedas:

$$\begin{aligned}\text{solve}(x) = \min(&\text{solve}(x - 1) + 1, \\ &\text{solve}(x - 3) + 1, \\ &\text{solve}(x - 4) + 1).\end{aligned}$$

El caso base de la recursión es $\text{solve}(0) = 0$, porque no se necesitan monedas para formar una suma vacía. Por ejemplo,

$$\text{solve}(10) = \text{solve}(7) + 1 = \text{solve}(4) + 2 = \text{solve}(0) + 3 = 3.$$

Ahora estamos listos para dar una función recursiva general que calcula el

número mínimo de monedas necesarias para formar una suma x :

$$\text{solve}(x) = \begin{cases} \infty & x < 0 \\ 0 & x = 0 \\ \min_{c \in \text{coins}} \text{solve}(x - c) + 1 & x > 0 \end{cases}$$

Primero, si $x < 0$, el valor es ∞ , porque es imposible formar una suma de dinero negativa. Luego, si $x = 0$, el valor es 0, porque no se necesitan monedas para formar una suma vacía. Finalmente, si $x > 0$, la variable c recorre todas las posibilidades de cómo elegir la primera moneda de la suma.

Una vez que se ha encontrado una función recursiva que resuelve el problema, podemos implementar directamente una solución en C++ (la constante INF denota infinito):

```
int solve(int x) {
    if (x < 0) return INF;
    if (x == 0) return 0;
    int best = INF;
    for (auto c : coins) {
        best = min(best, solve(x-c)+1);
    }
    return best;
}
```

Aun así, esta función no es eficiente, porque puede haber un número exponencial de formas de construir la suma. Sin embargo, a continuación veremos cómo hacer la función eficiente usando una técnica llamada memoización.

Usar memoización

La idea de la programación dinámica es usar **memoización** para calcular de forma eficiente los valores de una función recursiva. Esto significa que los valores de la función se almacenan en un arreglo después de calcularlos. Para cada parámetro, el valor de la función se calcula recursivamente solo una vez, y después de esto, el valor puede recuperarse directamente del arreglo.

En este problema, usamos los arreglos

```
bool ready[N];
int value[N];
```

donde `ready[x]` indica si el valor de `solve(x)` ha sido calculado, y si lo ha sido, `value[x]` contiene este valor. La constante N se ha elegido de modo que todos los valores requeridos quepan en los arreglos.

Ahora la función puede implementarse de forma eficiente de la siguiente manera:

```
int solve(int x) {
    if (x < 0) return INF;
```

```

    if (x == 0) return 0;
    if (ready[x]) return value[x];
    int best = INF;
    for (auto c : coins) {
        best = min(best, solve(x-c)+1);
    }
    value[x] = best;
    ready[x] = true;
    return best;
}

```

La función maneja los casos base $x < 0$ y $x = 0$ como antes. Luego la función comprueba desde `ready[x]` si `solve(x)` ya ha sido almacenado en `value[x]`, y si lo ha sido, la función lo devuelve directamente. En caso contrario la función calcula el valor de `solve(x)` recursivamente y lo almacena en `value[x]`.

Esta función funciona de forma eficiente, porque la respuesta para cada parámetro x se calcula recursivamente solo una vez. Después de que un valor de `solve(x)` se ha almacenado en `value[x]`, puede recuperarse de forma eficiente cada vez que la función sea llamada de nuevo con el parámetro x . La complejidad temporal del algoritmo es $O(nk)$, donde n es la suma objetivo y k es el número de monedas.

Nota que también podemos construir *iterativamente* el arreglo `value` usando un bucle que simplemente calcula todos los valores de `solve` para los parámetros $0 \dots n$:

```

value[0] = 0;
for (int x = 1; x <= n; x++) {
    value[x] = INF;
    for (auto c : coins) {
        if (x-c >= 0) {
            value[x] = min(value[x], value[x-c]+1);
        }
    }
}

```

De hecho, la mayoría de los programadores competitivos prefieren esta implementación, porque es más corta y tiene factores constantes más bajos. De ahora en adelante, también usamos implementaciones iterativas en nuestros ejemplos. Aun así, a menudo es más fácil pensar en soluciones de programación dinámica en términos de funciones recursivas.

Construir una solución

A veces se nos pide tanto encontrar el valor de una solución óptima como dar un ejemplo de cómo puede construirse tal solución. En el problema de las monedas, por ejemplo, podemos declarar otro arreglo que indique para cada suma de dinero la primera moneda de una solución óptima:


```
int first[N];
```

Luego, podemos modificar el algoritmo de la siguiente manera:

```
value[0] = 0;
for (int x = 1; x <= n; x++) {
    value[x] = INF;
    for (auto c : coins) {
        if (x-c >= 0 && value[x-c]+1 < value[x]) {
            value[x] = value[x-c]+1;
            first[x] = c;
        }
    }
}
```

Después de esto, el siguiente código puede usarse para imprimir las monedas que aparecen en una solución óptima para la suma n :

```
while (n > 0) {
    cout << first[n] << "\n";
    n -= first[n];
}
```

Contar el número de soluciones

Consideremos ahora otra versión del problema de las monedas donde nuestra tarea es calcular el número total de formas de producir una suma x usando las monedas. Por ejemplo, si $\text{coins} = \{1, 3, 4\}$ y $x = 5$, hay un total de 6 formas:

- | | |
|---------------|-----------|
| ■ $1+1+1+1+1$ | ■ $3+1+1$ |
| ■ $1+1+3$ | ■ $1+4$ |
| ■ $1+3+1$ | ■ $4+1$ |

De nuevo, podemos resolver el problema recursivamente. Sea $\text{solve}(x)$ el número de formas en que podemos formar la suma x . Por ejemplo, si $\text{coins} = \{1, 3, 4\}$, entonces $\text{solve}(5) = 6$ y la fórmula recursiva es

$$\begin{aligned}\text{solve}(x) = & \text{solve}(x-1) + \\ & \text{solve}(x-3) + \\ & \text{solve}(x-4).\end{aligned}$$

Luego, la función recursiva general es la siguiente:

$$\text{solve}(x) = \begin{cases} 0 & x < 0 \\ 1 & x = 0 \\ \sum_{c \in \text{coins}} \text{solve}(x-c) & x > 0 \end{cases}$$

Si $x < 0$, el valor es 0, porque no hay soluciones. Si $x = 0$, el valor es 1, porque solo hay una forma de formar una suma vacía. En caso contrario calculamos la suma de todos los valores de la forma $\text{solve}(x - c)$ donde c está en `coins`.

El siguiente código construye un arreglo `count` tal que `count[x]` es igual al valor de `solve(x)` para $0 \leq x \leq n$:

```
count[0] = 1;
for (int x = 1; x <= n; x++) {
    for (auto c : coins) {
        if (x-c >= 0) {
            count[x] += count[x-c];
        }
    }
}
```

A menudo el número de soluciones es tan grande que no se requiere calcular el número exacto sino que basta con dar la respuesta módulo m donde, por ejemplo, $m = 10^9 + 7$. Esto puede hacerse cambiando el código de modo que todos los cálculos se hagan módulo m . En el código anterior, basta con agregar la línea

```
count[x] %= m;
```

después de la línea

```
count[x] += count[x-c];
```

Ahora hemos discutido todas las ideas básicas de la programación dinámica. Como la programación dinámica puede usarse en muchas situaciones distintas, ahora recorreremos un conjunto de problemas que muestran más ejemplos sobre las posibilidades de la programación dinámica.

7.2. Subsecuencia creciente más larga

Nuestro primer problema es encontrar la **subsecuencia creciente más larga** en un arreglo de n elementos. Esta es una secuencia de longitud máxima de elementos del arreglo que va de izquierda a derecha, y cada elemento de la secuencia es mayor que el elemento anterior. Por ejemplo, en el arreglo

0	1	2	3	4	5	6	7
6	2	5	1	7	4	8	3

la subsecuencia creciente más larga contiene 4 elementos:

0	1	2	3	4	5	6	7
6	2	5	1	7	4	8	3



Sea $\text{length}(k)$ la longitud de la subsecuencia creciente más larga que termina en la posición k . Así, si calculamos todos los valores de $\text{length}(k)$ donde $0 \leq k \leq n - 1$, averiguaremos la longitud de la subsecuencia creciente más larga. Por ejemplo, los valores de la función para el arreglo anterior son los siguientes:

```
length(0) = 1
length(1) = 1
length(2) = 2
length(3) = 1
length(4) = 3
length(5) = 2
length(6) = 4
length(7) = 2
```

Por ejemplo, $\text{length}(6) = 4$, porque la subsecuencia creciente más larga que termina en la posición 6 consta de 4 elementos.

Para calcular un valor de $\text{length}(k)$, debemos encontrar una posición $i < k$ para la cual $\text{array}[i] < \text{array}[k]$ y $\text{length}(i)$ sea lo más grande posible. Entonces sabemos que $\text{length}(k) = \text{length}(i) + 1$, porque esta es una forma óptima de agregar $\text{array}[k]$ a una subsecuencia. Sin embargo, si no hay tal posición i , entonces $\text{length}(k) = 1$, lo que significa que la subsecuencia solo contiene $\text{array}[k]$.

Como todos los valores de la función pueden calcularse a partir de sus valores más pequeños, podemos usar programación dinámica. En el siguiente código, los valores de la función se almacenarán en un arreglo `length`.

```
for (int k = 0; k < n; k++) {
    length[k] = 1;
    for (int i = 0; i < k; i++) {
        if (array[i] < array[k]) {
            length[k] = max(length[k], length[i] + 1);
        }
    }
}
```

Este código funciona en tiempo $O(n^2)$, porque consta de dos bucles anidados. Sin embargo, también es posible implementar el cálculo de programación dinámica de forma más eficiente en tiempo $O(n \log n)$. ¿Puedes encontrar una forma de hacer esto?

7.3. Caminos en una cuadrícula

Nuestro siguiente problema es encontrar un camino desde la esquina superior izquierda hasta la esquina inferior derecha de una cuadrícula de $n \times n$, tal que solo nos movamos hacia abajo y hacia la derecha. Cada casilla contiene un entero positivo, y el camino debe construirse de modo que la suma de los valores a lo largo del camino sea lo más grande posible.

La siguiente figura muestra un camino óptimo en una cuadrícula:

3	7	9	2	7
9	8	3	5	5
1	7	9	8	5
3	8	6	4	10
6	3	9	7	8

La suma de los valores en el camino es 67, y esta es la mayor suma posible en un camino desde la esquina superior izquierda hasta la esquina inferior derecha.

Supongamos que las filas y columnas de la cuadrícula se numeran de 1 a n , y $\text{value}[y][x]$ es igual al valor de la casilla (y, x) . Sea $\text{sum}(y, x)$ la suma máxima en un camino desde la esquina superior izquierda hasta la casilla (y, x) . Ahora $\text{sum}(n, n)$ nos dice la suma máxima desde la esquina superior izquierda hasta la esquina inferior derecha. Por ejemplo, en la cuadrícula anterior, $\text{sum}(5, 5) = 67$.

Podemos calcular recursivamente las sumas de la siguiente manera:

$$\text{sum}(y, x) = \max(\text{sum}(y, x - 1), \text{sum}(y - 1, x)) + \text{value}[y][x]$$

La fórmula recursiva se basa en la observación de que un camino que termina en la casilla (y, x) puede venir o bien de la casilla $(y, x - 1)$ o de la casilla $(y - 1, x)$:

			↓	
		→		

Así, seleccionamos la dirección que maximiza la suma. Suponemos que $\text{sum}(y, x) = 0$ si $y = 0$ o $x = 0$ (porque no existen tales caminos), así que la fórmula recursiva también funciona cuando $y = 1$ o $x = 1$.

Como la función sum tiene dos parámetros, el arreglo de programación dinámica también tiene dos dimensiones. Por ejemplo, podemos usar un arreglo

```
int sum[N][N];
```

y calcular las sumas de la siguiente manera:

```
for (int y = 1; y <= n; y++) {
    for (int x = 1; x <= n; x++) {
        sum[y][x] = max(sum[y][x-1], sum[y-1][x]) + value[y][x];
    }
}
```

La complejidad temporal del algoritmo es $O(n^2)$.

7.4. Problemas de la mochila

El término **mochila** (knapsack) se refiere a problemas donde se da un conjunto de objetos, y hay que encontrar subconjuntos con algunas propiedades. Los problemas de la mochila a menudo pueden resolverse usando programación dinámica.

En esta sección, nos centramos en el siguiente problema: Dada una lista de pesos $[w_1, w_2, \dots, w_n]$, determinar todas las sumas que pueden construirse usando los pesos. Por ejemplo, si los pesos son $[1, 3, 3, 5]$, las siguientes sumas son posibles:

0	1	2	3	4	5	6	7	8	9	10	11	12
X	X		X	X	X	X	X	X	X		X	X

En este caso, todas las sumas entre $0 \dots 12$ son posibles, excepto 2 y 10. Por ejemplo, la suma 7 es posible porque podemos seleccionar los pesos $[1, 3, 3]$.

Para resolver el problema, nos centramos en subproblemas donde solo usamos los primeros k pesos para construir sumas. Sea $\text{possible}(x, k) = \text{true}$ si podemos construir una suma x usando los primeros k pesos, y en caso contrario $\text{possible}(x, k) = \text{false}$. Los valores de la función pueden calcularse recursivamente de la siguiente manera:

$$\text{possible}(x, k) = \text{possible}(x - w_k, k - 1) \vee \text{possible}(x, k - 1)$$

La fórmula se basa en el hecho de que podemos usar o no usar el peso w_k en la suma. Si usamos w_k , la tarea restante es formar la suma $x - w_k$ usando los primeros $k - 1$ pesos, y si no usamos w_k , la tarea restante es formar la suma x usando los primeros $k - 1$ pesos. Como casos base,

$$\text{possible}(x, 0) = \begin{cases} \text{true} & x = 0 \\ \text{false} & x \neq 0 \end{cases}$$

porque si no se usan pesos, solo podemos formar la suma 0.

La siguiente tabla muestra todos los valores de la función para los pesos $[1, 3, 3, 5]$ (el símbolo "X" indica los valores verdaderos):

$k \backslash x$	0	1	2	3	4	5	6	7	8	9	10	11	12
0	X												
1	X	X											
2	X	X		X	X								
3	X	X		X	X		X	X					
4	X	X		X	X	X	X	X	X	X		X	X

Después de calcular esos valores, $\text{possible}(x, n)$ nos dice si podemos construir una suma x usando *todos* los pesos.

Sea W la suma total de los pesos. La siguiente solución de programación dinámica de tiempo $O(nW)$ corresponde a la función recursiva:

```
possible[0][0] = true;
for (int k = 1; k <= n; k++) {
    for (int x = 0; x <= W; x++) {
        if (x-w[k] >= 0) possible[x][k] |= possible[x-w[k]][k-1];
        possible[x][k] |= possible[x][k-1];
    }
}
```

Sin embargo, aquí hay una mejor implementación que solo usa un arreglo unidimensional `possible[x]` que indica si podemos construir un subconjunto con suma x . El truco es actualizar el arreglo de derecha a izquierda para cada nuevo peso:

```
possible[0] = true;
for (int k = 1; k <= n; k++) {
    for (int x = W; x >= 0; x--) {
        if (possible[x]) possible[x+w[k]] = true;
    }
}
```

Nota que la idea general presentada aquí puede usarse en muchos problemas de la mochila. Por ejemplo, si se nos dan objetos con pesos y valores, podemos determinar para cada suma de pesos la suma máxima de valores de un subconjunto.

7.5. Distancia de edición

La **distancia de edición** o **distancia de Levenshtein**¹ es el número mínimo de operaciones de edición necesarias para transformar una cadena en otra cadena. Las operaciones de edición permitidas son las siguientes:

- insertar un carácter (p. ej. $ABC \rightarrow ABCA$)
- eliminar un carácter (p. ej. $ABC \rightarrow AC$)
- modificar un carácter (p. ej. $ABC \rightarrow ADC$)

Por ejemplo, la distancia de edición entre LOVE y MOVIE es 2, porque podemos primero realizar la operación $LOVE \rightarrow MOVE$ (modificar) y luego la operación $MOVE \rightarrow MOVIE$ (insertar). Este es el menor número posible de operaciones, porque está claro que solo una operación no es suficiente.

Supongamos que se nos da una cadena x de longitud n y una cadena y de longitud m , y queremos calcular la distancia de edición entre x e y . Para resolver el problema, definimos una función $distance(a, b)$ que da la distancia de edición

¹La distancia lleva el nombre de V. I. Levenshtein, quien la estudió en relación con los códigos binarios [49].

entre los prefijos $x[0 \dots a]$ e $y[0 \dots b]$. Así, usando esta función, la distancia de edición entre x e y es igual a $\text{distance}(n-1, m-1)$.

Podemos calcular los valores de distance de la siguiente manera:

$$\begin{aligned} \text{distance}(a, b) = \min(&\text{distance}(a, b-1) + 1, \\ &\text{distance}(a-1, b) + 1, \\ &\text{distance}(a-1, b-1) + \text{cost}(a, b)). \end{aligned}$$

Aquí $\text{cost}(a, b) = 0$ si $x[a] = y[b]$, y en caso contrario $\text{cost}(a, b) = 1$. La fórmula considera las siguientes formas de editar la cadena x :

- $\text{distance}(a, b-1)$: insertar un carácter al final de x
- $\text{distance}(a-1, b)$: eliminar el último carácter de x
- $\text{distance}(a-1, b-1)$: coincidir o modificar el último carácter de x

En los dos primeros casos, se necesita una operación de edición (insertar o eliminar). En el último caso, si $x[a] = y[b]$, podemos hacer coincidir los últimos caracteres sin editar, y en caso contrario se necesita una operación de edición (modificar).

La siguiente tabla muestra los valores de distance en el caso del ejemplo:

		M	O	V	I	E
L O V E	0	1	2	3	4	5
	1	1	2	3	4	5
	2	2	1	2	3	4
	3	3	2	1	2	3
	4	4	3	2	2	2

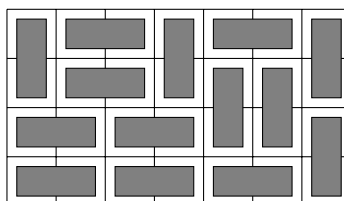
La esquina inferior derecha de la tabla nos dice que la distancia de edición entre LOVE y MOVIE es 2. La tabla también muestra cómo construir la secuencia más corta de operaciones de edición. En este caso el camino es el siguiente:

		M	O	V	I	E
	0	1	2	3	4	5
L	1	1	2	3	4	5
O	2	2	1	2	3	4
V	3	3	2	1	2	3
E	4	4	3	2	2	2

Los últimos caracteres de LOVE y MOVIE son iguales, así que la distancia de edición entre ellos es igual a la distancia de edición entre LOV y MOVI. Podemos usar una operación de edición para eliminar el carácter I de MOVI. Así, la distancia de edición es uno mayor que la distancia de edición entre LOV y MOV, etc.

7.6. Contar teselados

A veces los estados de una solución de programación dinámica son más complejos que combinaciones fijas de números. Como ejemplo, considera el problema de calcular el número de formas distintas de llenar una cuadrícula de $n \times m$ usando fichas de tamaño 1×2 y 2×1 . Por ejemplo, una solución válida para la cuadrícula de 4×7 es



y el número total de soluciones es 781.

El problema puede resolverse usando programación dinámica recorriendo la cuadrícula fila por fila. Cada fila de una solución puede representarse como una cadena que contiene m caracteres del conjunto $\{\sqcap, \sqcup, \sqsubset, \sqsupset\}$. Por ejemplo, la solución anterior consta de cuatro filas que corresponden a las siguientes cadenas:

- $\sqcap \sqsubset \sqcap \sqsubset \sqcap \sqcap$
- $\sqcup \sqsubset \sqcup \sqcap \sqcap \sqcup$
- $\sqsubset \sqsubset \sqsubset \sqcup \sqcup \sqcap$
- $\sqsubset \sqsubset \sqsubset \sqsubset \sqsubset \sqcup$

Sea $\text{count}(k, x)$ el número de formas de construir una solución para las filas $1 \dots k$ de la cuadrícula tal que la cadena x corresponde a la fila k . Es posible usar programación dinámica aquí, porque el estado de una fila está restringido solo por el estado de la fila anterior.

Una solución es válida si la fila 1 no contiene el carácter $\sqcup$, la fila n no contiene el carácter $\sqcap$, y todas las filas consecutivas son *compatibles*. Por ejemplo, las filas $\sqcup \sqsubset \sqcup \sqcap \sqcap \sqcup$ y $\sqsubset \sqsubset \sqsubset \sqcup \sqcup \sqcap$ son compatibles, mientras que las filas $\sqcap \sqsubset \sqcap \sqsubset \sqcap \sqcap$ y $\sqsubset \sqsubset \sqsubset \sqsubset \sqsubset \sqcup$ no son compatibles.

Como una fila consta de m caracteres y hay cuatro opciones para cada carácter, el número de filas distintas es a lo sumo 4^m . Así, la complejidad temporal de la solución es $O(n4^{2m})$ porque podemos recorrer los $O(4^m)$ estados posibles para cada fila, y para cada estado, hay $O(4^m)$ estados posibles para la fila anterior. En la práctica, es buena idea rotar la cuadrícula de modo que el lado más corto tenga longitud m , porque el factor 4^{2m} domina la complejidad temporal.

Es posible hacer la solución más eficiente usando una representación más compacta para las filas. Resulta que basta con saber qué columnas de la fila anterior contienen la casilla superior de una ficha vertical. Así, podemos representar una fila usando solo los caracteres $\sqcap$ y $\square$, donde $\square$ es una combinación de los caracteres $\sqcup$, $\sqsubset$ y $\sqsupset$. Usando esta representación, solo hay 2^m filas distintas y la complejidad temporal es $O(n2^{2m})$.

Como nota final, también hay una sorprendente fórmula directa para calcular el número de teselados²:

$$\prod_{a=1}^{\lceil n/2 \rceil} \prod_{b=1}^{\lceil m/2 \rceil} 4 \cdot \left(\cos^2 \frac{\pi a}{n+1} + \cos^2 \frac{\pi b}{m+1} \right)$$

Esta fórmula es muy eficiente, porque calcula el número de teselados en tiempo $O(nm)$, pero como la respuesta es un producto de números reales, un problema al usar la fórmula es cómo almacenar los resultados intermedios con precisión.

²Sorprendentemente, esta fórmula fue descubierta en 1961 por dos equipos de investigación [43, 67] que trabajaron de forma independiente.

Capítulo 8

Análisis amortizado

La complejidad temporal de un algoritmo suele ser fácil de analizar simplemente examinando la estructura del algoritmo: qué bucles contiene el algoritmo y cuántas veces se ejecutan los bucles. Sin embargo, a veces un análisis directo no da una imagen real de la eficiencia del algoritmo.

El **análisis amortizado** puede usarse para analizar algoritmos que contienen operaciones cuya complejidad temporal varía. La idea es estimar el tiempo total usado en todas esas operaciones durante la ejecución del algoritmo, en lugar de centrarse en operaciones individuales.

8.1. Método de los dos punteros

En el **método de los dos punteros**, se usan dos punteros para recorrer los valores del arreglo. Ambos punteros pueden moverse en una sola dirección, lo que asegura que el algoritmo funcione de forma eficiente. A continuación discutimos dos problemas que pueden resolverse usando el método de los dos punteros.

Suma de subarreglo

Como primer ejemplo, considera un problema donde se nos da un arreglo de n enteros positivos y una suma objetivo x , y queremos encontrar un subarreglo cuya suma sea x o reportar que no existe tal subarreglo.

Por ejemplo, el arreglo

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

contiene un subarreglo cuya suma es 8:

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

Este problema puede resolverse en tiempo $O(n)$ usando el método de los dos punteros. La idea es mantener punteros que apunten al primer y último valor de un subarreglo. En cada turno, el puntero izquierdo se mueve un paso a la derecha, y el puntero derecho se mueve a la derecha mientras la suma del subarreglo

resultante sea a lo sumo x . Si la suma se vuelve exactamente x , se ha encontrado una solución.

Como ejemplo, considera el siguiente arreglo y una suma objetivo $x = 8$:

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

El subarreglo inicial contiene los valores 1, 3 y 2 cuya suma es 6:

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

Luego, el puntero izquierdo se mueve un paso a la derecha. El puntero derecho no se mueve, porque de lo contrario la suma del subarreglo excedería x .

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

De nuevo, el puntero izquierdo se mueve un paso a la derecha, y esta vez el puntero derecho se mueve tres pasos a la derecha. La suma del subarreglo es $2 + 5 + 1 = 8$, así que se ha encontrado un subarreglo cuya suma es x .

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

El tiempo de ejecución del algoritmo depende del número de pasos que se mueve el puntero derecho. Aunque no hay una cota superior útil sobre cuántos pasos puede moverse el puntero en un *solo* turno, sabemos que el puntero se mueve *un total de* $O(n)$ pasos durante el algoritmo, porque solo se mueve a la derecha.

Como tanto el puntero izquierdo como el derecho se mueven $O(n)$ pasos durante el algoritmo, el algoritmo funciona en tiempo $O(n)$.

Problema 2SUM

Otro problema que puede resolverse usando el método de los dos punteros es el siguiente problema, también conocido como el **problema 2SUM**: dado un arreglo de n números y una suma objetivo x , encontrar dos valores del arreglo tales que su suma sea x , o reportar que no existen tales valores.

Para resolver el problema, primero ordenamos los valores del arreglo en orden creciente. Después de eso, recorremos el arreglo usando dos punteros. El puntero izquierdo comienza en el primer valor y se mueve un paso a la derecha en cada turno. El puntero derecho comienza en el último valor y siempre se mueve a la izquierda hasta que la suma del valor izquierdo y derecho sea a lo sumo x . Si la suma es exactamente x , se ha encontrado una solución.

Por ejemplo, considera el siguiente arreglo y una suma objetivo $x = 12$:

1	4	5	6	7	9	9	10
---	---	---	---	---	---	---	----

Las posiciones iniciales de los punteros son las siguientes. La suma de los valores es $1 + 10 = 11$ que es menor que x .

1	4	5	6	7	9	9	10
↑							↑

Luego el puntero izquierdo se mueve un paso a la derecha. El puntero derecho se mueve tres pasos a la izquierda, y la suma se vuelve $4 + 7 = 11$.

1	4	5	6	7	9	9	10
	↑			↑			

Después de esto, el puntero izquierdo se mueve un paso a la derecha de nuevo. El puntero derecho no se mueve, y se ha encontrado una solución $5 + 7 = 12$.

1	4	5	6	7	9	9	10
		↑		↑			

El tiempo de ejecución del algoritmo es $O(n \log n)$, porque primero ordena el arreglo en tiempo $O(n \log n)$, y luego ambos punteros se mueven $O(n)$ pasos.

Nota que es posible resolver el problema de otra forma en tiempo $O(n \log n)$ usando búsqueda binaria. En tal solución, recorremos el arreglo y para cada valor del arreglo, intentamos encontrar otro valor que produzca la suma x . Esto puede hacerse realizando n búsquedas binarias, cada una de las cuales toma $O(\log n)$ de tiempo.

Un problema más difícil es el **problema 3SUM** que pide encontrar *tres* valores del arreglo cuya suma sea x . Usando la idea del algoritmo anterior, este problema puede resolverse en tiempo $O(n^2)$ ¹. ¿Puedes ver cómo?

8.2. Elementos menores más cercanos

El análisis amortizado se usa a menudo para estimar el número de operaciones realizadas sobre una estructura de datos. Las operaciones pueden estar distribuidas de forma desigual de modo que la mayoría de las operaciones ocurran durante una cierta fase del algoritmo, pero el número total de las operaciones sea limitado.

Como ejemplo, considera el problema de encontrar para cada elemento del arreglo el **elemento menor más cercano**, es decir, el primer elemento menor que precede al elemento en el arreglo. Es posible que no exista tal elemento, en cuyo caso el algoritmo debería reportar esto. A continuación veremos cómo el problema puede resolverse de forma eficiente usando una estructura de pila.

¹Durante mucho tiempo, se pensó que resolver el problema 3SUM de forma más eficiente que en tiempo $O(n^2)$ no sería posible. Sin embargo, en 2014, resultó [30] que no es así.

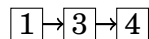
Recorremos el arreglo de izquierda a derecha y mantenemos una pila de elementos del arreglo. En cada posición del arreglo, eliminamos elementos de la pila hasta que el elemento de la cima sea menor que el elemento actual, o la pila esté vacía. Luego, reportamos que el elemento de la cima es el elemento menor más cercano del elemento actual, o si la pila está vacía, no hay tal elemento. Finalmente, agregamos el elemento actual a la pila.

Como ejemplo, considera el siguiente arreglo:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---

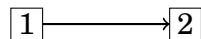
Primero, los elementos 1, 3 y 4 se agregan a la pila, porque cada elemento es mayor que el elemento anterior. Así, el elemento menor más cercano de 4 es 3, y el elemento menor más cercano de 3 es 1.

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



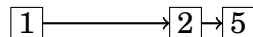
El siguiente elemento 2 es menor que los dos elementos de la cima en la pila. Así, los elementos 3 y 4 se eliminan de la pila, y luego el elemento 2 se agrega a la pila. Su elemento menor más cercano es 1:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



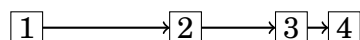
Luego, el elemento 5 es mayor que el elemento 2, así que se agregará a la pila, y su elemento menor más cercano es 2:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



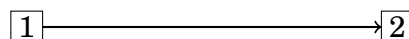
Después de esto, el elemento 5 se elimina de la pila y los elementos 3 y 4 se agregan a la pila:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



Finalmente, todos los elementos excepto 1 se eliminan de la pila y el último elemento 2 se agrega a la pila:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



La eficiencia del algoritmo depende del número total de operaciones sobre la pila. Si el elemento actual es mayor que el elemento de la cima en la pila, se agrega directamente a la pila, lo cual es eficiente. Sin embargo, a veces la pila puede contener varios elementos mayores y toma tiempo eliminarlos. Aun así, cada elemento se agrega *exactamente una vez* a la pila y se elimina *a lo sumo una vez* de la pila. Así, cada elemento provoca $O(1)$ operaciones sobre la pila, y el algoritmo funciona en tiempo $O(n)$.

8.3. Mínimo de ventana deslizante

Una **ventana deslizante** es un subarreglo de tamaño constante que se mueve de izquierda a derecha a través del arreglo. En cada posición de la ventana, queremos calcular alguna información sobre los elementos dentro de la ventana. En esta sección, nos centramos en el problema de mantener el **mínimo de ventana deslizante**, lo que significa que debemos reportar el valor más pequeño dentro de cada ventana.

El mínimo de ventana deslizante puede calcularse usando una idea similar a la que usamos para calcular los elementos menores más cercanos. Mantenemos una cola donde cada elemento es mayor que el elemento anterior, y el primer elemento siempre corresponde al elemento mínimo dentro de la ventana. Después de cada movimiento de la ventana, eliminamos elementos del final de la cola hasta que el último elemento de la cola sea menor que el nuevo elemento de la ventana, o la cola quede vacía. También eliminamos el primer elemento de la cola si ya no está dentro de la ventana. Finalmente, agregamos el nuevo elemento de la ventana al final de la cola.

Como ejemplo, considera el siguiente arreglo:

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

Supongamos que el tamaño de la ventana deslizante es 4. En la primera posición de la ventana, el valor más pequeño es 1:

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1	→	4	→	5
---	---	---	---	---

Luego la ventana se mueve un paso a la derecha. El nuevo elemento 3 es menor que los elementos 4 y 5 en la cola, así que los elementos 4 y 5 se eliminan de la cola y el elemento 3 se agrega a la cola. El valor más pequeño sigue siendo 1.

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1	→	3
---	---	---

Después de esto, la ventana se mueve de nuevo, y el elemento más pequeño 1 ya no pertenece a la ventana. Así, se elimina de la cola y el valor más pequeño ahora es 3. También el nuevo elemento 4 se agrega a la cola.

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

3	→	4
---	---	---

El siguiente nuevo elemento 1 es menor que todos los elementos de la cola. Así, todos los elementos se eliminan de la cola y esta solo contendrá el elemento 1:

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1

Finalmente la ventana llega a su última posición. El elemento 2 se agrega a la cola, pero el valor más pequeño dentro de la ventana sigue siendo 1.

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1 → 2

Como cada elemento del arreglo se agrega a la cola exactamente una vez y se elimina de la cola a lo sumo una vez, el algoritmo funciona en tiempo $O(n)$.

Capítulo 9

Consultas de rango

En este capítulo, discutimos estructuras de datos que nos permiten procesar de forma eficiente consultas de rango. En una **consulta de rango**, nuestra tarea es calcular un valor basado en un subarreglo de un arreglo. Las consultas de rango típicas son:

- $\text{sum}_q(a, b)$: calcular la suma de los valores en el rango $[a, b]$
- $\text{min}_q(a, b)$: encontrar el valor mínimo en el rango $[a, b]$
- $\text{max}_q(a, b)$: encontrar el valor máximo en el rango $[a, b]$

Por ejemplo, considera el rango $[3, 6]$ en el siguiente arreglo:

0	1	2	3	4	5	6	7
1	3	8	4	6	1	3	4

En este caso, $\text{sum}_q(3, 6) = 14$, $\text{min}_q(3, 6) = 1$ y $\text{max}_q(3, 6) = 6$.

Una forma simple de procesar consultas de rango es usar un bucle que recorra todos los valores del arreglo en el rango. Por ejemplo, la siguiente función puede usarse para procesar consultas de suma en un arreglo:

```
int sum(int a, int b) {  
    int s = 0;  
    for (int i = a; i <= b; i++) {  
        s += array[i];  
    }  
    return s;  
}
```

Esta función funciona en tiempo $O(n)$, donde n es el tamaño del arreglo. Así, podemos procesar q consultas en tiempo $O(nq)$ usando la función. Sin embargo, si tanto n como q son grandes, este enfoque es lento. Afortunadamente, resulta que hay formas de procesar consultas de rango mucho más eficientemente.

9.1. Consultas en un arreglo estático

Primero nos centramos en una situación donde el arreglo es *estático*, es decir, los valores del arreglo nunca se actualizan entre las consultas. En este caso, basta con construir una estructura de datos estática que nos diga la respuesta para cualquier consulta posible.

Consultas de suma

Podemos procesar fácilmente consultas de suma en un arreglo estático construyendo un **arreglo de sumas de prefijos**. Cada valor del arreglo de sumas de prefijos es igual a la suma de los valores del arreglo original hasta esa posición, es decir, el valor en la posición k es $\text{sum}_q(0, k)$. El arreglo de sumas de prefijos puede construirse en tiempo $O(n)$.

Por ejemplo, considera el siguiente arreglo:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

El arreglo de sumas de prefijos correspondiente es el siguiente:

0	1	2	3	4	5	6	7
1	4	8	16	22	23	27	29

Como el arreglo de sumas de prefijos contiene todos los valores de $\text{sum}_q(0, k)$, podemos calcular cualquier valor de $\text{sum}_q(a, b)$ en tiempo $O(1)$ de la siguiente manera:

$$\text{sum}_q(a, b) = \text{sum}_q(0, b) - \text{sum}_q(0, a - 1)$$

Definiendo $\text{sum}_q(0, -1) = 0$, la fórmula anterior también se cumple cuando $a = 0$.

Por ejemplo, considera el rango $[3, 6]$:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

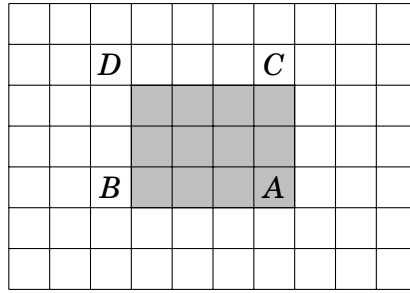
En este caso $\text{sum}_q(3, 6) = 8 + 6 + 1 + 4 = 19$. Esta suma puede calcularse a partir de dos valores del arreglo de sumas de prefijos:

0	1	2	3	4	5	6	7
1	4	8	16	22	23	27	29

Así, $\text{sum}_q(3, 6) = \text{sum}_q(0, 6) - \text{sum}_q(0, 2) = 27 - 8 = 19$.

También es posible generalizar esta idea a dimensiones más altas. Por ejemplo, podemos construir un arreglo bidimensional de sumas de prefijos que puede usarse para calcular la suma de cualquier subarreglo rectangular en tiempo $O(1)$. Cada suma en tal arreglo corresponde a un subarreglo que comienza en la esquina superior izquierda del arreglo.

La siguiente figura ilustra la idea:



La suma del subarreglo gris puede calcularse usando la fórmula

$$S(A) - S(B) - S(C) + S(D),$$

donde $S(X)$ denota la suma de los valores en un subarreglo rectangular desde la esquina superior izquierda hasta la posición de X .

Consultas de mínimo

Las consultas de mínimo son más difíciles de procesar que las consultas de suma. Aun así, hay un método de preprocesamiento bastante simple de tiempo $O(n \log n)$ después del cual podemos responder cualquier consulta de mínimo en tiempo $O(1)$ ¹. Nota que como las consultas de mínimo y máximo pueden procesarse de forma similar, podemos centrarnos en las consultas de mínimo.

La idea es precalcular todos los valores de $\min_q(a, b)$ donde $b - a + 1$ (la longitud del rango) es una potencia de dos. Por ejemplo, para el arreglo

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

se calculan los siguientes valores:

a	b	$\min_q(a, b)$	a	b	$\min_q(a, b)$	a	b	$\min_q(a, b)$
0	0	1	0	1	1	0	3	1
1	1	3	1	2	3	1	4	3
2	2	4	2	3	4	2	5	1
3	3	8	3	4	6	3	6	1
4	4	6	4	5	1	4	7	1
5	5	1	5	6	1	0	7	1
6	6	4	6	7	2			
7	7	2						

¹Esta técnica se presentó en [7] y a veces se llama el método de la **tabla dispersa** (sparse table). También hay técnicas más sofisticadas [22] donde el tiempo de preprocesamiento es solo $O(n)$, pero tales algoritmos no se necesitan en la programación competitiva.

El número de valores precalculados es $O(n \log n)$, porque hay $O(\log n)$ longitudes de rango que son potencias de dos. Los valores pueden calcularse de forma eficiente usando la fórmula recursiva

$$\min_q(a, b) = \min(\min_q(a, a + w - 1), \min_q(a + w, b)),$$

donde $b - a + 1$ es una potencia de dos y $w = (b - a + 1)/2$. Calcular todos esos valores toma $O(n \log n)$ de tiempo.

Después de esto, cualquier valor de $\min_q(a, b)$ puede calcularse en tiempo $O(1)$ como el mínimo de dos valores precalculados. Sea k la mayor potencia de dos que no excede $b - a + 1$. Podemos calcular el valor de $\min_q(a, b)$ usando la fórmula

$$\min_q(a, b) = \min(\min_q(a, a + k - 1), \min_q(b - k + 1, b)).$$

En la fórmula anterior, el rango $[a, b]$ se representa como la unión de los rangos $[a, a + k - 1]$ y $[b - k + 1, b]$, ambos de longitud k .

Como ejemplo, considera el rango $[1, 6]$:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

La longitud del rango es 6, y la mayor potencia de dos que no excede 6 es 4. Así, el rango $[1, 6]$ es la unión de los rangos $[1, 4]$ y $[3, 6]$:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

Como $\min_q(1, 4) = 3$ y $\min_q(3, 6) = 1$, concluimos que $\min_q(1, 6) = 1$.

9.2. Árbol indexado binario

Un **árbol indexado binario** o **árbol de Fenwick**² puede verse como una variante dinámica de un arreglo de sumas de prefijos. Admite dos operaciones de tiempo $O(\log n)$ sobre un arreglo: procesar una consulta de suma de rango y actualizar un valor.

La ventaja de un árbol indexado binario es que nos permite actualizar de forma eficiente los valores del arreglo entre consultas de suma. Esto no sería posible usando un arreglo de sumas de prefijos, porque después de cada actualización, sería necesario construir de nuevo todo el arreglo de sumas de prefijos en tiempo $O(n)$.

²La estructura del árbol indexado binario fue presentada por P. M. Fenwick en 1994 [21].

Estructura

Aunque el nombre de la estructura es *árbol* indexado binario, normalmente se representa como un arreglo. En esta sección suponemos que todos los arreglos están indexados desde 1, porque hace la implementación más fácil.

Sea $p(k)$ la mayor potencia de dos que divide a k . Almacenamos un árbol indexado binario como un arreglo *tree* tal que

$$\text{tree}[k] = \text{sum}_q(k - p(k) + 1, k),$$

es decir, cada posición k contiene la suma de los valores en un rango del arreglo original cuya longitud es $p(k)$ y que termina en la posición k . Por ejemplo, como $p(6) = 2$, $\text{tree}[6]$ contiene el valor de $\text{sum}_q(5, 6)$.

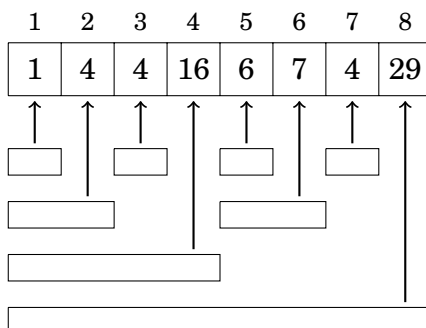
Por ejemplo, considera el siguiente arreglo:

1	2	3	4	5	6	7	8
1	3	4	8	6	1	4	2

El árbol indexado binario correspondiente es el siguiente:

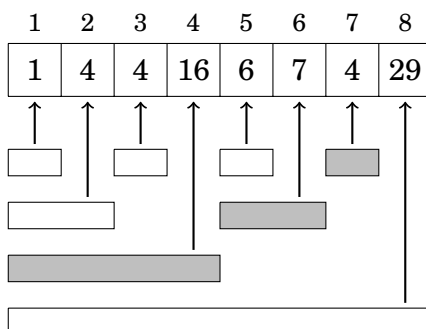
1	2	3	4	5	6	7	8
1	4	4	16	6	7	4	29

La siguiente figura muestra más claramente cómo cada valor del árbol indexado binario corresponde a un rango del arreglo original:



Usando un árbol indexado binario, cualquier valor de $\text{sum}_q(1, k)$ puede calcularse en tiempo $O(\log n)$, porque un rango $[1, k]$ siempre puede dividirse en $O(\log n)$ rangos cuyas sumas están almacenadas en el árbol.

Por ejemplo, el rango $[1, 7]$ consta de los siguientes rangos:



Así, podemos calcular la suma correspondiente de la siguiente manera:

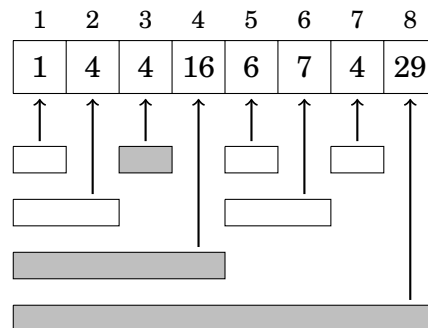
$$\text{sum}_q(1, 7) = \text{sum}_q(1, 4) + \text{sum}_q(5, 6) + \text{sum}_q(7, 7) = 16 + 7 + 4 = 27$$

Para calcular el valor de $\text{sum}_q(a, b)$ donde $a > 1$, podemos usar el mismo truco que usamos con los arreglos de sumas de prefijos:

$$\text{sum}_q(a, b) = \text{sum}_q(1, b) - \text{sum}_q(1, a - 1).$$

Como podemos calcular tanto $\text{sum}_q(1, b)$ como $\text{sum}_q(1, a - 1)$ en tiempo $O(\log n)$, la complejidad temporal total es $O(\log n)$.

Luego, después de actualizar un valor en el arreglo original, deberían actualizarse varios valores en el árbol indexado binario. Por ejemplo, si el valor en la posición 3 cambia, las sumas de los siguientes rangos cambian:



Como cada elemento del arreglo pertenece a $O(\log n)$ rangos en el árbol indexado binario, basta con actualizar $O(\log n)$ valores en el árbol.

Implementación

Las operaciones de un árbol indexado binario pueden implementarse de forma eficiente usando operaciones de bits. El hecho clave que se necesita es que podemos calcular cualquier valor de $p(k)$ usando la fórmula

$$p(k) = k \& -k.$$

La siguiente función calcula el valor de $\text{sum}_q(1, k)$:

```
int sum(int k) {
    int s = 0;
    while (k >= 1) {
        s += tree[k];
        k -= k & -k;
    }
    return s;
}
```

La siguiente función incrementa el valor del arreglo en la posición k en x (x puede ser positivo o negativo):

```

void add(int k, int x) {
    while (k <= n) {
        tree[k] += x;
        k += k&-k;
    }
}

```

La complejidad temporal de ambas funciones es $O(\log n)$, porque las funciones acceden a $O(\log n)$ valores en el árbol indexado binario, y cada movimiento a la siguiente posición toma $O(1)$ de tiempo.

9.3. Árbol de segmentos

Un **árbol de segmentos**³ es una estructura de datos que admite dos operaciones: procesar una consulta de rango y actualizar un valor del arreglo. Los árboles de segmentos pueden admitir consultas de suma, consultas de mínimo y máximo y muchas otras consultas de modo que ambas operaciones funcionen en tiempo $O(\log n)$.

Comparado con un árbol indexado binario, la ventaja de un árbol de segmentos es que es una estructura de datos más general. Mientras que los árboles indexados binarios solo admiten consultas de suma⁴, los árboles de segmentos también admiten otras consultas. Por otro lado, un árbol de segmentos requiere más memoria y es un poco más difícil de implementar.

Estructura

Un árbol de segmentos es un árbol binario tal que los nodos del nivel inferior del árbol corresponden a los elementos del arreglo, y los otros nodos contienen información necesaria para procesar consultas de rango.

En esta sección, suponemos que el tamaño del arreglo es una potencia de dos y se usa indexación desde cero, porque es conveniente construir un árbol de segmentos para tal arreglo. Si el tamaño del arreglo no es una potencia de dos, siempre podemos agregarle elementos adicionales.

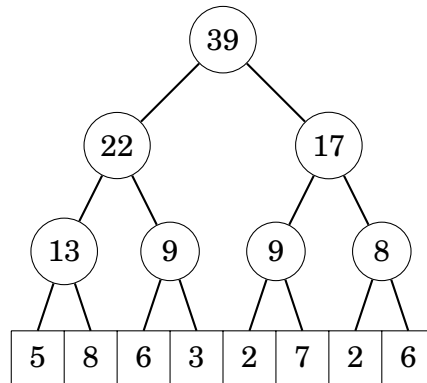
Primero discutiremos árboles de segmentos que admiten consultas de suma. Como ejemplo, considera el siguiente arreglo:

0	1	2	3	4	5	6	7
5	8	6	3	2	7	2	6

El árbol de segmentos correspondiente es el siguiente:

³La implementación de abajo hacia arriba de este capítulo corresponde a la de [62]. Estructuras similares se usaron a finales de los años 70 para resolver problemas geométricos [9].

⁴De hecho, usando *dos* árboles indexados binarios es posible admitir consultas de mínimo [16], pero esto es más complicado que usar un árbol de segmentos.

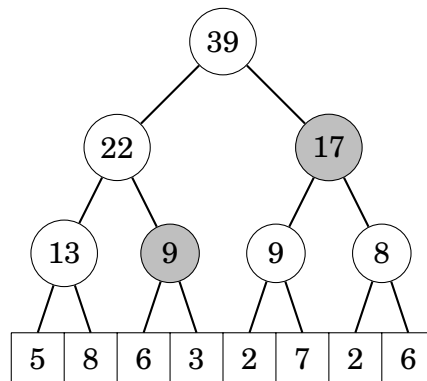


Cada nodo interno del árbol corresponde a un rango del arreglo cuyo tamaño es una potencia de dos. En el árbol anterior, el valor de cada nodo interno es la suma de los valores correspondientes del arreglo, y puede calcularse como la suma de los valores de sus nodos hijos izquierdo y derecho.

Resulta que cualquier rango $[a, b]$ puede dividirse en $O(\log n)$ rangos cuyos valores están almacenados en nodos del árbol. Por ejemplo, considera el rango $[2, 7]$:

0	1	2	3	4	5	6	7
5	8	6	3	2	7	2	6

Aquí $\text{sum}_q(2, 7) = 6 + 3 + 2 + 7 + 2 + 6 = 26$. En este caso, los siguientes dos nodos del árbol corresponden al rango:

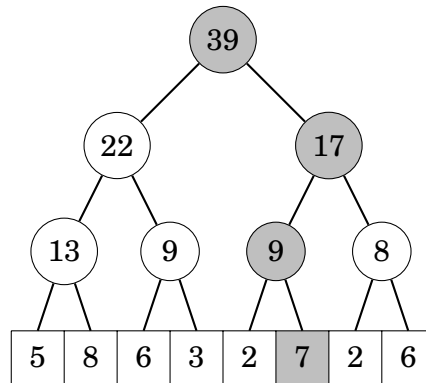


Así, otra forma de calcular la suma es $9 + 17 = 26$.

Cuando la suma se calcula usando nodos ubicados lo más alto posible en el árbol, se necesitan a lo sumo dos nodos en cada nivel del árbol. Por lo tanto, el número total de nodos es $O(\log n)$.

Después de una actualización del arreglo, deberíamos actualizar todos los nodos cuyo valor dependa del valor actualizado. Esto puede hacerse recorriendo el camino desde el elemento actualizado del arreglo hasta el nodo superior y actualizando los nodos a lo largo del camino.

La siguiente figura muestra qué nodos del árbol cambian si el valor 7 del arreglo cambia:

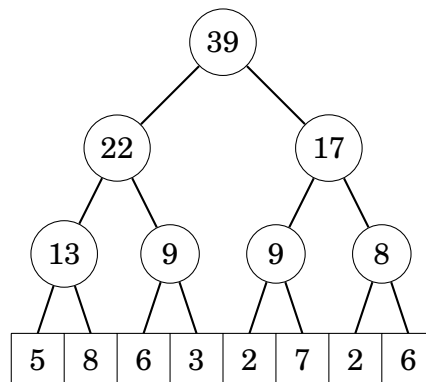


El camino de abajo hacia arriba siempre consta de $O(\log n)$ nodos, así que cada actualización cambia $O(\log n)$ nodos en el árbol.

Implementación

Almacenamos un árbol de segmentos como un arreglo de $2n$ elementos donde n es el tamaño del arreglo original y una potencia de dos. Los nodos del árbol se almacenan de arriba hacia abajo: $\text{tree}[1]$ es el nodo superior, $\text{tree}[2]$ y $\text{tree}[3]$ son sus hijos, y así sucesivamente. Finalmente, los valores desde $\text{tree}[n]$ hasta $\text{tree}[2n - 1]$ corresponden a los valores del arreglo original en el nivel inferior del árbol.

Por ejemplo, el árbol de segmentos



se almacena de la siguiente manera:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
39	22	17	13	9	9	8	5	8	6	3	2	7	2	6

Usando esta representación, el padre de $\text{tree}[k]$ es $\text{tree}[\lfloor k/2 \rfloor]$, y sus hijos son $\text{tree}[2k]$ y $\text{tree}[2k + 1]$. Nota que esto implica que la posición de un nodo es par si es un hijo izquierdo e impar si es un hijo derecho.

La siguiente función calcula el valor de $\text{sum}_q(a, b)$:

```

int sum(int a, int b) {
    a += n; b += n;
    int s = 0;

```

```

while (a <= b) {
    if (a%2 == 1) s += tree[a++];
    if (b%2 == 0) s += tree[b--];
    a /= 2; b /= 2;
}
return s;
}

```

La función mantiene un rango que inicialmente es $[a + n, b + n]$. Luego, en cada paso, el rango se mueve un nivel más arriba en el árbol, y antes de eso, los valores de los nodos que no pertenecen al rango superior se suman a la suma.

La siguiente función incrementa el valor del arreglo en la posición k en x :

```

void add(int k, int x) {
    k += n;
    tree[k] += x;
    for (k /= 2; k >= 1; k /= 2) {
        tree[k] = tree[2*k] + tree[2*k+1];
    }
}

```

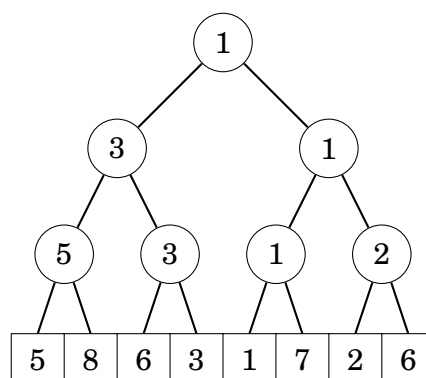
Primero la función actualiza el valor en el nivel inferior del árbol. Después de esto, la función actualiza los valores de todos los nodos internos del árbol, hasta que llega al nodo superior del árbol.

Ambas funciones anteriores funcionan en tiempo $O(\log n)$, porque un árbol de segmentos de n elementos consta de $O(\log n)$ niveles, y las funciones se mueven un nivel más arriba en el árbol en cada paso.

Otras consultas

Los árboles de segmentos pueden admitir todas las consultas de rango donde es posible dividir un rango en dos partes, calcular la respuesta por separado para ambas partes y luego combinar de forma eficiente las respuestas. Ejemplos de tales consultas son el mínimo y el máximo, el máximo común divisor, y las operaciones de bits and, or y xor.

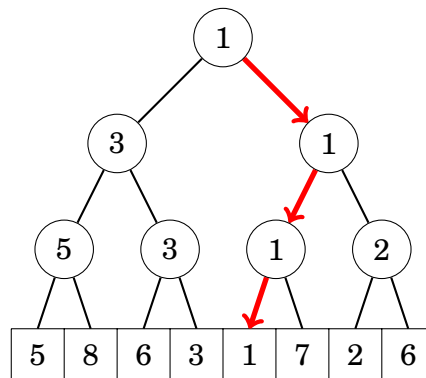
Por ejemplo, el siguiente árbol de segmentos admite consultas de mínimo:



En este caso, cada nodo del árbol contiene el valor más pequeño del rango correspondiente del arreglo. El nodo superior del árbol contiene el valor más pequeño de todo el arreglo. Las operaciones pueden implementarse como antes, pero en lugar de sumas, se calculan mínimos.

La estructura de un árbol de segmentos también nos permite usar búsqueda binaria para localizar elementos del arreglo. Por ejemplo, si el árbol admite consultas de mínimo, podemos encontrar la posición de un elemento con el valor más pequeño en tiempo $O(\log n)$.

Por ejemplo, en el árbol anterior, un elemento con el valor más pequeño 1 puede encontrarse recorriendo un camino hacia abajo desde el nodo superior:



9.4. Técnicas adicionales

Compresión de índices

Una limitación en las estructuras de datos que se construyen sobre un arreglo es que los elementos se indexan usando enteros consecutivos. Surgen dificultades cuando se necesitan índices grandes. Por ejemplo, si deseamos usar el índice 10^9 , el arreglo debería contener 10^9 elementos, lo que requeriría demasiada memoria.

Sin embargo, a menudo podemos sortear esta limitación usando **compresión de índices**, donde los índices originales se reemplazan con los índices 1, 2, 3, etc. Esto puede hacerse si conocemos de antemano todos los índices necesarios durante el algoritmo.

La idea es reemplazar cada índice original x con $c(x)$ donde c es una función que comprime los índices. Requerimos que el orden de los índices no cambie, así que si $a < b$, entonces $c(a) < c(b)$. Esto nos permite realizar consultas cómodamente aunque los índices estén comprimidos.

Por ejemplo, si los índices originales son 555, 10^9 y 8, los nuevos índices son:

$$\begin{aligned} c(8) &= 1 \\ c(555) &= 2 \\ c(10^9) &= 3 \end{aligned}$$

Actualizaciones de rango

Hasta ahora, hemos implementado estructuras de datos que admiten consultas de rango y actualizaciones de valores individuales. Consideremos ahora una situación opuesta, donde debemos actualizar rangos y recuperar valores individuales. Nos centramos en una operación que incrementa todos los elementos de un rango $[a, b]$ en x .

Sorprendentemente, podemos usar las estructuras de datos presentadas en este capítulo también en esta situación. Para hacer esto, construimos un **arreglo de diferencias** cuyos valores indican las diferencias entre valores consecutivos del arreglo original. Así, el arreglo original es el arreglo de sumas de prefijos del arreglo de diferencias. Por ejemplo, considera el siguiente arreglo:

0	1	2	3	4	5	6	7
3	3	1	1	1	5	2	2

El arreglo de diferencias para el arreglo anterior es el siguiente:

0	1	2	3	4	5	6	7
3	0	-2	0	0	4	-3	0

Por ejemplo, el valor 2 en la posición 6 del arreglo original corresponde a la suma $3 - 2 + 4 - 3 = 2$ en el arreglo de diferencias.

La ventaja del arreglo de diferencias es que podemos actualizar un rango en el arreglo original cambiando solo dos elementos en el arreglo de diferencias. Por ejemplo, si queremos incrementar en 5 los valores del arreglo original entre las posiciones 1 y 4, basta con incrementar el valor del arreglo de diferencias en la posición 1 en 5 y disminuir el valor en la posición 5 en 5. El resultado es el siguiente:

0	1	2	3	4	5	6	7
3	5	-2	0	0	-1	-3	0

Más en general, para incrementar los valores en el rango $[a, b]$ en x , incrementamos el valor en la posición a en x y disminuimos el valor en la posición $b + 1$ en x . Así, solo se necesita actualizar valores individuales y procesar consultas de suma, por lo que podemos usar un árbol indexado binario o un árbol de segmentos.

Un problema más difícil es admitir tanto consultas de rango como actualizaciones de rango. En el Capítulo 28 veremos que incluso esto es posible.

Manipulación de bits

10.1. Representación en bits

103

Por ejemplo, la representación en bits del número int `-43` es

1111111111111111111111111010101.

En una representación sin signo, solo pueden usarse números no negativos, pero la cota superior de los valores es mayor. Una variable sin signo de n bits puede contener cualquier entero entre 0 y $2^n - 1$. Por ejemplo, en C++, una variable `unsigned int` puede contener cualquier entero entre 0 y $2^{32} - 1$.

Hay una conexión entre las representaciones: un número con signo $-x$ es igual a un número sin signo $2^n - x$. Por ejemplo, el siguiente código muestra que el número con signo $x = -43$ es igual al número sin signo $y = 2^{32} - 43$:

```
int x = -43;
unsigned int y = x;
cout << x << "\n"; // -43
cout << y << "\n"; // 4294967253
```

Si un número es mayor que la cota superior de la representación en bits, el número se desbordará. En una representación con signo, el siguiente número después de $2^{n-1} - 1$ es -2^{n-1} , y en una representación sin signo, el siguiente número después de $2^n - 1$ es 0. Por ejemplo, considera el siguiente código:

```
int x = 2147483647
cout << x << "\n"; // 2147483647
x++;
cout << x << "\n"; // -2147483648
```

Inicialmente, el valor de x es $2^{31} - 1$. Este es el mayor valor que puede almacenarse en una variable `int`, así que el siguiente número después de $2^{31} - 1$ es -2^{31} .

10.2. Operaciones de bits

Operación and

La operación **and** x & y produce un número que tiene bits en uno en las posiciones donde tanto x como y tienen bits en uno. Por ejemplo, $22 \& 26 = 18$, porque

$$\begin{array}{rcl} & 10110 & (22) \\ \& & 11010 & (26) \\ \hline = & 10010 & (18) \end{array}$$

Usando la operación $\&$, podemos comprobar si un número x es par porque $x \& 1 = 0$ si x es par, y $x \& 1 = 1$ si x es impar. Más en general, x es divisible entre 2^k exactamente cuando $x \& (2^k - 1) = 0$.

Operación or

La operación **or** $x \mid y$ produce un número que tiene bits en uno en las posiciones donde al menos uno de x e y tiene bits en uno. Por ejemplo, $22 \mid 26 = 30$, porque

$$\begin{array}{r} 10110 \quad (22) \\ \mid \quad 11010 \quad (26) \\ \hline = \quad 11110 \quad (30) \end{array}$$

Operación xor

La operación **xor** $x \wedge y$ produce un número que tiene bits en uno en las posiciones donde exactamente uno de x e y tiene bits en uno. Por ejemplo, $22 \wedge 26 = 12$, porque

$$\begin{array}{r} 10110 \quad (22) \\ \wedge \quad 11010 \quad (26) \\ \hline = \quad 01100 \quad (12) \end{array}$$

Operación not

La operación **not** $\sim x$ produce un número donde todos los bits de x han sido invertidos. La fórmula $\sim x = -x - 1$ se cumple, por ejemplo, $\sim 29 = -30$.

El resultado de la operación not a nivel de bits depende de la longitud de la representación en bits, porque la operación invierte todos los bits. Por ejemplo, si los números son números int de 32 bits, el resultado es el siguiente:

$$\begin{array}{rcl} x & = & 29 \quad 000000000000000000000000011101 \\ \sim x & = & -30 \quad 11111111111111111111111111100010 \end{array}$$

Desplazamientos de bits

El desplazamiento de bits a la izquierda $x \ll k$ agrega k bits en cero al número, y el desplazamiento de bits a la derecha $x \gg k$ elimina los k últimos bits del número. Por ejemplo, $14 \ll 2 = 56$, porque 14 y 56 corresponden a 1110 y 111000. De manera similar, $49 \gg 3 = 6$, porque 49 y 6 corresponden a 110001 y 110.

Nota que $x \ll k$ corresponde a multiplicar x por 2^k , y $x \gg k$ corresponde a dividir x entre 2^k redondeado hacia abajo a un entero.

Aplicaciones

Un número de la forma $1 \ll k$ tiene un bit en uno en la posición k y todos los demás bits en cero, así que podemos usar tales números para acceder a bits individuales de números. En particular, el k -ésimo bit de un número está en uno exactamente cuando $x \& (1 \ll k)$ no es cero. El siguiente código imprime la representación en bits de un número int x :

```
for (int i = 31; i >= 0; i--) {
    if (x & (1 << i)) cout << "1";
    else cout << "0";
}
```

También es posible modificar bits individuales de números usando ideas similares. Por ejemplo, la fórmula $x \mid (1 \ll k)$ pone en uno el k -ésimo bit de x , la fórmula $x \& \sim(1 \ll k)$ pone en cero el k -ésimo bit de x , y la fórmula $x \wedge (1 \ll k)$ invierte el k -ésimo bit de x .

La fórmula $x \& (x - 1)$ pone en cero el último bit en uno de x , y la fórmula $x \& -x$ pone en cero todos los bits en uno, excepto el último bit en uno. La fórmula $x \mid (x - 1)$ invierte todos los bits después del último bit en uno. Nota también que un número positivo x es una potencia de dos exactamente cuando $x \& (x - 1) = 0$.

Funciones adicionales

El compilador g++ proporciona las siguientes funciones para contar bits:

- `__builtin_clz(x)`: el número de ceros al principio del número
- `__builtin_ctz(x)`: el número de ceros al final del número
- `__builtin_popcount(x)`: el número de unos en el número
- `__builtin_parity(x)`: la paridad (par o impar) del número de unos

Las funciones pueden usarse de la siguiente manera:

```
int x = 5328; // 000000000000000000001010011010000
cout << __builtin_clz(x) << "\n"; // 19
cout << __builtin_ctz(x) << "\n"; // 4
cout << __builtin_popcount(x) << "\n"; // 5
cout << __builtin_parity(x) << "\n"; // 1
```

Aunque las funciones anteriores solo admiten números `int`, también hay versiones `long long` de las funciones disponibles con el sufijo `ll`.

10.3. Representar conjuntos

Todo subconjunto de un conjunto $\{0, 1, 2, \dots, n - 1\}$ puede representarse como un entero de n bits cuyos bits en uno indican qué elementos pertenecen al subconjunto. Esta es una forma eficiente de representar conjuntos, porque cada elemento requiere solo un bit de memoria, y las operaciones de conjuntos pueden implementarse como operaciones de bits.

Por ejemplo, como `int` es un tipo de 32 bits, un número `int` puede representar cualquier subconjunto del conjunto $\{0, 1, 2, \dots, 31\}$. La representación en bits del conjunto $\{1, 3, 4, 8\}$ es

000000000000000000000000100011010,

que corresponde al número $2^8 + 2^4 + 2^3 + 2^1 = 282$.

Implementación de conjuntos

El siguiente código declara una variable `int x` que puede contener un subconjunto de $\{0, 1, 2, \dots, 31\}$. Después de esto, el código agrega los elementos 1, 3, 4 y 8 al conjunto e imprime el tamaño del conjunto.

```
int x = 0;
x |= (1<<1);
x |= (1<<3);
x |= (1<<4);
x |= (1<<8);
cout << __builtin_popcount(x) << "\n"; // 4
```

Luego, el siguiente código imprime todos los elementos que pertenecen al conjunto:

```
for (int i = 0; i < 32; i++) {
    if (x&(1<<i)) cout << i << " ";
}
// output: 1 3 4 8
```

Operaciones de conjuntos

Las operaciones de conjuntos pueden implementarse de la siguiente manera como operaciones de bits:

	sintaxis de conjuntos	sintaxis de bits
intersección	$a \cap b$	$a \& b$
unión	$a \cup b$	$a \mid b$
complemento	$\bar{a}$	$\sim a$
diferencia	$a \setminus b$	$a \& (\sim b)$

Por ejemplo, el siguiente código primero construye los conjuntos $x = \{1, 3, 4, 8\}$ e $y = \{3, 6, 8, 9\}$, y luego construye el conjunto $z = x \cup y = \{1, 3, 4, 6, 8, 9\}$:

```
int x = (1<<1)|(1<<3)|(1<<4)|(1<<8);
int y = (1<<3)|(1<<6)|(1<<8)|(1<<9);
int z = x|y;
cout << __builtin_popcount(z) << "\n"; // 6
```

Recorrer subconjuntos

El siguiente código recorre los subconjuntos de $\{0, 1, \dots, n-1\}$:

```
for (int b = 0; b < (1<<n); b++) {
    // process subset b
}
```

El siguiente código recorre los subconjuntos con exactamente k elementos:

```
for (int b = 0; b < (1<<n); b++) {
    if (__builtin_popcount(b) == k) {
        // process subset b
    }
}
```

El siguiente código recorre los subconjuntos de un conjunto x :

```
int b = 0;
do {
    // process subset b
} while (b=(b-x)&x);
```

10.4. Optimizaciones con bits

Muchos algoritmos pueden optimizarse usando operaciones de bits. Tales optimizaciones no cambian la complejidad temporal del algoritmo, pero pueden tener un gran impacto en el tiempo de ejecución real del código. En esta sección discutimos ejemplos de tales situaciones.

Distancias de Hamming

La **distancia de Hamming** $\text{hamming}(a, b)$ entre dos cadenas a y b de igual longitud es el número de posiciones donde las cadenas difieren. Por ejemplo,

$$\text{hamming}(01101, 11001) = 2.$$

Considera el siguiente problema: Dada una lista de n cadenas de bits, cada una de longitud k , calcular la distancia de Hamming mínima entre dos cadenas de la lista. Por ejemplo, la respuesta para $[00111, 01101, 11110]$ es 2, porque

- $\text{hamming}(00111, 01101) = 2$,
- $\text{hamming}(00111, 11110) = 3$, y
- $\text{hamming}(01101, 11110) = 3$.

Una forma directa de resolver el problema es recorrer todos los pares de cadenas y calcular sus distancias de Hamming, lo que produce un algoritmo de tiempo $O(n^2k)$. La siguiente función puede usarse para calcular distancias:

```
int hamming(string a, string b) {
    int d = 0;
```

```

for (int i = 0; i < k; i++) {
    if (a[i] != b[i]) d++;
}
return d;
}

```

Sin embargo, si k es pequeño, podemos optimizar el código almacenando las cadenas de bits como enteros y calculando las distancias de Hamming usando operaciones de bits. En particular, si $k \leq 32$, podemos simplemente almacenar las cadenas como valores `int` y usar la siguiente función para calcular distancias:

```

int hamming(int a, int b) {
    return __builtin_popcount(a^b);
}

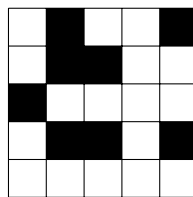
```

En la función anterior, la operación `xor` construye una cadena de bits que tiene bits en uno en las posiciones donde a y b difieren. Luego, el número de bits se calcula usando la función `__builtin_popcount`.

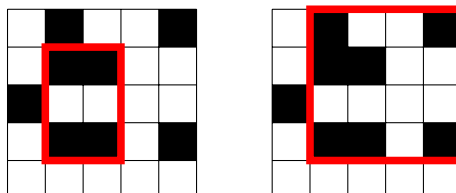
Para comparar las implementaciones, generamos una lista de 10000 cadenas de bits aleatorias de longitud 30. Usando el primer enfoque, la búsqueda tomó 13.5 segundos, y después de la optimización con bits, solo tomó 0.5 segundos. Así, el código optimizado con bits fue casi 30 veces más rápido que el código original.

Contar subcuadrículas

Como otro ejemplo, considera el siguiente problema: Dada una cuadrícula de $n \times n$ donde cada casilla es negra (1) o blanca (0), calcular el número de subcuadrículas cuyas cuatro esquinas son negras. Por ejemplo, la cuadrícula



contiene dos de tales subcuadrículas:



Hay un algoritmo de tiempo $O(n^3)$ para resolver el problema: recorrer todos los $O(n^2)$ pares de filas y para cada par (a, b) calcular el número de columnas que contienen una casilla negra en ambas filas en tiempo $O(n)$. El siguiente código supone que `color[y][x]` denota el color en la fila y y la columna x :

```

int count = 0;
for (int i = 0; i < n; i++) {
    if (color[a][i] == 1 && color[b][i] == 1) count++;
}

```

Luego, esas columnas representan $\text{count}(\text{count} - 1)/2$ subcuadrículas con esquinas negras, porque podemos elegir cualesquiera dos de ellas para formar una subcuadrícula.

Para optimizar este algoritmo, dividimos la cuadrícula en bloques de columnas de modo que cada bloque conste de N columnas consecutivas. Luego, cada fila se almacena como una lista de números de N bits que describen los colores de las casillas. Ahora podemos procesar N columnas al mismo tiempo usando operaciones de bits. En el siguiente código, $\text{color}[y][k]$ representa un bloque de N colores como bits.

```

int count = 0;
for (int i = 0; i <= n/N; i++) {
    count += __builtin_popcount(color[a][i]&color[b][i]);
}

```

El algoritmo resultante funciona en tiempo $O(n^3/N)$.

Generamos una cuadrícula aleatoria de tamaño 2500×2500 y comparamos la implementación original y la optimizada con bits. Mientras que el código original tomó 29.6 segundos, la versión optimizada con bits solo tomó 3.1 segundos con $N = 32$ (números int) y 1.7 segundos con $N = 64$ (números long long).

10.5. Programación dinámica

Las operaciones de bits ofrecen una forma eficiente y cómoda de implementar algoritmos de programación dinámica cuyos estados contienen subconjuntos de elementos, porque tales estados pueden almacenarse como enteros. A continuación discutimos ejemplos de combinar operaciones de bits y programación dinámica.

Selección óptima

Como primer ejemplo, considera el siguiente problema: Se nos dan los precios de k productos durante n días, y queremos comprar cada producto exactamente una vez. Sin embargo, se nos permite comprar a lo sumo un producto en un día. ¿Cuál es el precio total mínimo? Por ejemplo, considera el siguiente escenario ($k = 3$ y $n = 8$):

	0	1	2	3	4	5	6	7
producto 0	6	9	5	2	8	9	1	6
producto 1	8	2	6	2	7	5	7	2
producto 2	5	3	9	7	3	5	1	4

En este escenario, el precio total mínimo es 5:

	0	1	2	3	4	5	6	7
producto 0	6	9	5	2	8	9	1	6
producto 1	8	2	6	2	7	5	7	2
producto 2	5	3	9	7	3	5	1	4

Sea $\text{price}[x][d]$ el precio del producto x en el día d . Por ejemplo, en el escenario anterior $\text{price}[2][3] = 7$. Luego, sea $\text{total}(S, d)$ el precio total mínimo para comprar un subconjunto S de productos para el día d . Usando esta función, la solución al problema es $\text{total}(\{0 \dots k-1\}, n-1)$.

Primero, $\text{total}(\emptyset, d) = 0$, porque no cuesta nada comprar un conjunto vacío, y $\text{total}(\{x\}, 0) = \text{price}[x][0]$, porque hay una forma de comprar un producto el primer día. Luego, puede usarse la siguiente recurrencia:

$$\text{total}(S, d) = \min(\text{total}(S, d-1), \min_{x \in S} (\text{total}(S \setminus x, d-1) + \text{price}[x][d]))$$

Esto significa que o bien no compramos ningún producto el día d o compramos un producto x que pertenece a S . En el último caso, eliminamos x de S y agregamos el precio de x al precio total.

El siguiente paso es calcular los valores de la función usando programación dinámica. Para almacenar los valores de la función, declaramos un arreglo

```
int total[1<<K][N];
```

donde K y N son constantes suficientemente grandes. La primera dimensión del arreglo corresponde a una representación en bits de un subconjunto.

Primero, los casos donde $d = 0$ pueden procesarse de la siguiente manera:

```
for (int x = 0; x < k; x++) {
    total[1<<x][0] = price[x][0];
}
```

Luego, la recurrencia se traduce en el siguiente código:

```
for (int d = 1; d < n; d++) {
    for (int s = 0; s < (1<<k); s++) {
        total[s][d] = total[s][d-1];
        for (int x = 0; x < k; x++) {
            if (s & (1<<x)) {
                total[s][d] = min(total[s][d],
```

```

    }
    }
    }
    total[s^(1<<x)][d-1]+price[x][d]);
}

```

La complejidad temporal del algoritmo es $O(n2^k k)$.

De permutaciones a subconjuntos

Usando programación dinámica, a menudo es posible cambiar una iteración sobre permutaciones por una iteración sobre subconjuntos¹. El beneficio de esto es que $n!$, el número de permutaciones, es mucho mayor que 2^n , el número de subconjuntos. Por ejemplo, si $n = 20$, entonces $n! \approx 2.4 \cdot 10^{18}$ y $2^n \approx 10^6$. Así, para ciertos valores de n , podemos recorrer eficientemente los subconjuntos pero no las permutaciones.

Como ejemplo, considera el siguiente problema: Hay un ascensor con peso máximo x , y n personas con pesos conocidos que quieren ir de la planta baja al último piso. ¿Cuál es el número mínimo de viajes necesarios si las personas entran al ascensor en un orden óptimo?

Por ejemplo, supongamos que $x = 10$, $n = 5$ y los pesos son los siguientes:

persona	peso
0	2
1	3
2	3
3	5
4	6

En este caso, el número mínimo de viajes es 2. Un orden óptimo es $\{0, 2, 3, 1, 4\}$, que reparte a las personas en dos viajes: primero $\{0, 2, 3\}$ (peso total 10), y luego $\{1, 4\}$ (peso total 9).

El problema puede resolverse fácilmente en tiempo $O(n!n)$ probando todas las permutaciones posibles de n personas. Sin embargo, podemos usar programación dinámica para obtener un algoritmo más eficiente de tiempo $O(2^n n)$. La idea es calcular para cada subconjunto de personas dos valores: el número mínimo de viajes necesarios y el peso mínimo de las personas que viajan en el último grupo.

Sea $\text{weight}[p]$ el peso de la persona p . Definimos dos funciones: $\text{rides}(S)$ es el número mínimo de viajes para un subconjunto S , y $\text{last}(S)$ es el peso mínimo del último viaje. Por ejemplo, en el escenario anterior

$$\text{rides}(\{1, 3, 4\}) = 2 \quad \text{y} \quad \text{last}(\{1, 3, 4\}) = 5,$$

porque los viajes óptimos son $\{1, 4\}$ y $\{3\}$, y el segundo viaje tiene peso 5. Por supuesto, nuestro objetivo final es calcular el valor de $\text{rides}(\{0 \dots n-1\})$.

Podemos calcular los valores de las funciones recursivamente y luego aplicar programación dinámica. La idea es recorrer todas las personas que pertenecen a

¹Esta técnica fue introducida en 1962 por M. Held y R. M. Karp [34].

S y elegir de forma óptima la última persona p que entra al ascensor. Cada una de tales elecciones produce un subproblema para un subconjunto más pequeño de personas. Si $\text{last}(S \setminus p) + \text{weight}[p] \leq x$, podemos agregar p al último viaje. En caso contrario, tenemos que reservar un nuevo viaje que inicialmente solo contiene a p .

Para implementar la programación dinámica, declaramos un arreglo

```
pair<int,int> best[1<<N];
```

que contiene para cada subconjunto S un par $(\text{rides}(S), \text{last}(S))$. Establecemos el valor para un grupo vacío de la siguiente manera:

```
best[0] = {1,0};
```

Luego, podemos llenar el arreglo de la siguiente manera:

```
for (int s = 1; s < (1<<n); s++) {
    // initial value: n+1 rides are needed
    best[s] = {n+1,0};
    for (int p = 0; p < n; p++) {
        if (s&(1<<p)) {
            auto option = best[s^(1<<p)];
            if (option.second+weight[p] <= x) {
                // add p to an existing ride
                option.second += weight[p];
            } else {
                // reserve a new ride for p
                option.first++;
                option.second = weight[p];
            }
            best[s] = min(best[s], option);
        }
    }
}
```

Nota que el bucle anterior garantiza que para cualesquiera dos subconjuntos S_1 y S_2 tales que $S_1 \subset S_2$, procesamos S_1 antes que S_2 . Así, los valores de programación dinámica se calculan en el orden correcto.

Contar subconjuntos

Nuestro último problema de este capítulo es el siguiente: Sea $X = \{0 \dots n-1\}$, y a cada subconjunto $S \subset X$ se le asigna un entero $\text{value}[S]$. Nuestra tarea es calcular para cada S

$$\text{sum}(S) = \sum_{A \subset S} \text{value}[A],$$

es decir, la suma de los valores de los subconjuntos de S .

Por ejemplo, supongamos que $n = 3$ y los valores son los siguientes:

- $\text{value}[\emptyset] = 3$
- $\text{value}[\{0\}] = 1$
- $\text{value}[\{1\}] = 4$
- $\text{value}[\{0, 1\}] = 5$
- $\text{value}[\{2\}] = 5$
- $\text{value}[\{0, 2\}] = 1$
- $\text{value}[\{1, 2\}] = 3$
- $\text{value}[\{0, 1, 2\}] = 3$

En este caso, por ejemplo,

$$\begin{aligned}\text{sum}(\{0, 2\}) &= \text{value}[\emptyset] + \text{value}[\{0\}] + \text{value}[\{2\}] + \text{value}[\{0, 2\}] \\ &= 3 + 1 + 5 + 1 = 10.\end{aligned}$$

Como hay un total de 2^n subconjuntos, una posible solución es recorrer todos los pares de subconjuntos en tiempo $O(2^{2n})$. Sin embargo, usando programación dinámica, podemos resolver el problema en tiempo $O(2^n n)$. La idea es centrarnos en sumas donde los elementos que pueden eliminarse de S están restringidos.

Sea $\text{partial}(S, k)$ la suma de los valores de los subconjuntos de S con la restricción de que solo los elementos $0 \dots k$ pueden eliminarse de S . Por ejemplo,

$$\text{partial}(\{0, 2\}, 1) = \text{value}[\{2\}] + \text{value}[\{0, 2\}],$$

porque solo podemos eliminar los elementos $0 \dots 1$. Podemos calcular los valores de sum usando los valores de partial , porque

$$\text{sum}(S) = \text{partial}(S, n - 1).$$

Los casos base de la función son

$$\text{partial}(S, -1) = \text{value}[S],$$

porque en este caso no puede eliminarse ningún elemento de S . Luego, en el caso general podemos usar la siguiente recurrencia:

$$\text{partial}(S, k) = \begin{cases} \text{partial}(S, k - 1) & k \notin S \\ \text{partial}(S, k - 1) + \text{partial}(S \setminus \{k\}, k - 1) & k \in S \end{cases}$$

Aquí nos centramos en el elemento k . Si $k \in S$, tenemos dos opciones: podemos mantener k en S o eliminarlo de S .

Hay una forma particularmente ingeniosa de implementar el cálculo de las sumas. Podemos declarar un arreglo

```
int sum[1<<N];
```

que contendrá la suma de cada subconjunto. El arreglo se inicializa de la siguiente manera:

```
for (int s = 0; s < (1<<n); s++) {
    sum[s] = value[s];
}
```


Luego, podemos llenar el arreglo de la siguiente manera:

```
for (int k = 0; k < n; k++) {  
    for (int s = 0; s < (1<<n); s++) {  
        if (s&(1<<k)) sum[s] += sum[s^(1<<k)];  
    }  
}
```

Este código calcula los valores de $\text{partial}(S, k)$ para $k = 0 \dots n - 1$ en el arreglo `sum`. Como $\text{partial}(S, k)$ siempre se basa en $\text{partial}(S, k - 1)$, podemos reutilizar el arreglo `sum`, lo que produce una implementación muy eficiente.

Parte II

Algoritmos de grafos

Parte III

Temas avanzados

Bibliografia

- [1] A. V. Aho, J. E. Hopcroft and J. Ullman. *Data Structures and Algorithms*, Addison-Wesley, 1983.
- [2] R. K. Ahuja and J. B. Orlin. Distance directed augmenting path algorithms for maximum flow and parametric maximum flow problems. *Naval Research Logistics*, 38(3):413–430, 1991.
- [3] A. M. Andrew. Another efficient algorithm for convex hulls in two dimensions. *Information Processing Letters*, 9(5):216–219, 1979.
- [4] B. Aspvall, M. F. Plass and R. E. Tarjan. A linear-time algorithm for testing the truth of certain quantified boolean formulas. *Information Processing Letters*, 8(3):121–123, 1979.
- [5] R. Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.
- [6] M. Beck, E. Pine, W. Tarrat and K. Y. Jensen. New integer representations as the sum of three cubes. *Mathematics of Computation*, 76(259):1683–1690, 2007.
- [7] M. A. Bender and M. Farach-Colton. The LCA problem revisited. In *Latin American Symposium on Theoretical Informatics*, 88–94, 2000.
- [8] J. Bentley. *Programming Pearls*. Addison-Wesley, 1999 (2nd edition).
- [9] J. Bentley and D. Wood. An optimal worst case algorithm for reporting intersections of rectangles. *IEEE Transactions on Computers*, C-29(7):571–577, 1980.
- [10] C. L. Bouton. Nim, a game with a complete mathematical theory. *Annals of Mathematics*, 3(1/4):35–39, 1901.
- [11] Croatian Open Competition in Informatics, <http://hsin.hr/coci/>
- [12] Codeforces: On "Mo's algorithm", <http://codeforces.com/blog/entry/20032>
- [13] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein. *Introduction to Algorithms*, MIT Press, 2009 (3rd edition).
- [14] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.

- [15] K. Diks et al. *Looking for a Challenge? The Ultimate Problem Set from the University of Warsaw Programming Competitions*, University of Warsaw, 2012.
- [16] M. Dima and R. Ceterchi. Efficient range minimum queries using binary indexed trees. *Olympiad in Informatics*, 9(1):39–44, 2015.
- [17] J. Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17(3):449–467, 1965.
- [18] J. Edmonds and R. M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972.
- [19] S. Even, A. Itai and A. Shamir. On the complexity of time table and multi-commodity flow problems. *16th Annual Symposium on Foundations of Computer Science*, 184–193, 1975.
- [20] D. Fanding. A faster algorithm for shortest-path – SPFA. *Journal of Southwest Jiaotong University*, 2, 1994.
- [21] P. M. Fenwick. A new data structure for cumulative frequency tables. *Software: Practice and Experience*, 24(3):327–336, 1994.
- [22] J. Fischer and V. Heun. Theoretical and practical improvements on the RMQ-problem, with applications to LCA and LCE. In *Annual Symposium on Combinatorial Pattern Matching*, 36–48, 2006.
- [23] R. W. Floyd Algorithm 97: shortest path. *Communications of the ACM*, 5(6):345, 1962.
- [24] L. R. Ford. Network flow theory. RAND Corporation, Santa Monica, California, 1956.
- [25] L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8(3):399–404, 1956.
- [26] R. Freivalds. Probabilistic machines can use less running time. In *IFIP congress*, 839–842, 1977.
- [27] F. Le Gall. Powers of tensors and fast matrix multiplication. In *Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation*, 296–303, 2014.
- [28] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman and Company, 1979.
- [29] Google Code Jam Statistics (2017), <https://www.go-hero.net/jam/17>
- [30] A. Grønlund and S. Pettie. Threesomes, degenerates, and love triangles. In *Proceedings of the 55th Annual Symposium on Foundations of Computer Science*, 621–630, 2014.

- [31] P. M. Grundy. Mathematics and games. *Eureka*, 2(5):6–8, 1939.
- [32] D. Gusfield. *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*, Cambridge University Press, 1997.
- [33] S. Halim and F. Halim. *Competitive Programming 3: The New Lower Bound of Programming Contests*, 2013.
- [34] M. Held and R. M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962.
- [35] C. Hierholzer and C. Wiener. Über die Möglichkeit, einen Linienzug ohne Wiederholung und ohne Unterbrechung zu umfahren. *Mathematische Annalen*, 6(1), 30–32, 1873.
- [36] C. A. R. Hoare. Algorithm 64: Quicksort. *Communications of the ACM*, 4(7):321, 1961.
- [37] C. A. R. Hoare. Algorithm 65: Find. *Communications of the ACM*, 4(7):321–322, 1961.
- [38] J. E. Hopcroft and J. D. Ullman. A linear list merging algorithm. Technical report, Cornell University, 1971.
- [39] E. Horowitz and S. Sahni. Computing partitions with applications to the knapsack problem. *Journal of the ACM*, 21(2):277–292, 1974.
- [40] D. A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952.
- [41] The International Olympiad in Informatics Syllabus, <https://people.ksp.sk/~misof/ioi-syllabus/>
- [42] R. M. Karp and M. O. Rabin. Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development*, 31(2):249–260, 1987.
- [43] P. W. Kasteleyn. The statistics of dimers on a lattice: I. The number of dimer arrangements on a quadratic lattice. *Physica*, 27(12):1209–1225, 1961.
- [44] C. Kent, G. M. Landau and M. Ziv-Ukelson. On the complexity of sparse exon assembly. *Journal of Computational Biology*, 13(5):1013–1027, 2006.
- [45] J. Kleinberg and É. Tardos. *Algorithm Design*, Pearson, 2005.
- [46] D. E. Knuth. *The Art of Computer Programming. Volume 2: Seminumerical Algorithms*, Addison–Wesley, 1998 (3rd edition).
- [47] D. E. Knuth. *The Art of Computer Programming. Volume 3: Sorting and Searching*, Addison–Wesley, 1998 (2nd edition).

- [48] J. B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956.
- [49] V. I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet physics doklady*, 10(8):707–710, 1966.
- [50] M. G. Main and R. J. Lorentz. An $O(n \log n)$ algorithm for finding all repetitions in a string. *Journal of Algorithms*, 5(3):422–432, 1984.
- [51] J. Pachocki and J. Radoszewski. Where to use and how not to use polynomial string hashing. *Olympiads in Informatics*, 7(1):90–100, 2013.
- [52] I. Parberry. An efficient algorithm for the Knight’s tour problem. *Discrete Applied Mathematics*, 73(3):251–260, 1997.
- [53] D. Pearson. A polynomial-time algorithm for the change-making problem. *Operations Research Letters*, 33(3):231–234, 2005.
- [54] R. C. Prim. Shortest connection networks and some generalizations. *Bell System Technical Journal*, 36(6):1389–1401, 1957.
- [55] 27-Queens Puzzle: Massively Parallel Enumeration and Solution Counting. <https://github.com/preusser/q27>
- [56] M. I. Shamos and D. Hoey. Closest-point problems. In *Proceedings of the 16th Annual Symposium on Foundations of Computer Science*, 151–162, 1975.
- [57] M. Sharir. A strong-connectivity algorithm and its applications in data flow analysis. *Computers & Mathematics with Applications*, 7(1):67–72, 1981.
- [58] S. S. Skiena. *The Algorithm Design Manual*, Springer, 2008 (2nd edition).
- [59] S. S. Skiena and M. A. Revilla. *Programming Challenges: The Programming Contest Training Manual*, Springer, 2003.
- [60] SZKOpuł, <https://szkopul.edu.pl/>
- [61] R. Sprague. Über mathematische Kampfspiele. *Tohoku Mathematical Journal*, 41:438–444, 1935.
- [62] P. Stańczyk. *Algorytmika praktyczna w konkursach Informatycznych*, MSc thesis, University of Warsaw, 2006.
- [63] V. Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, 1969.
- [64] R. E. Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.
- [65] R. E. Tarjan. Applications of path compression on balanced trees. *Journal of the ACM*, 26(4):690–715, 1979.

- [66] R. E. Tarjan and U. Vishkin. Finding biconnected components and computing tree functions in logarithmic parallel time. In *Proceedings of the 25th Annual Symposium on Foundations of Computer Science*, 12–20, 1984.
- [67] H. N. V. Temperley and M. E. Fisher. Dimer problem in statistical mechanics – an exact result. *Philosophical Magazine*, 6(68):1061–1063, 1961.
- [68] USA Computing Olympiad, <http://www.usaco.org/>
- [69] H. C. von Warnsdorf. *Des Rösselsprunges einfachste und allgemeinste Lösung*. Schmalkalden, 1823.
- [70] S. Warshall. A theorem on boolean matrices. *Journal of the ACM*, 9(1):11–12, 1962.

